

COPILOTÉ PAR IA

Première édition • 2025

Construire un Système **RAG**

Production Ready

De l'ingénierie des embeddings à l'industrialisation complète

Loïc Laureote

À propos de l'auteur

Loïc Laureote est né et a grandi en Martinique, cette île des Caraïbes où la luxuriance de la nature façonne les esprits autant que les caractères. Développeur autodidacte depuis ses premières années, il a appris à programmer seul, armé d'une curiosité insatiable, d'une connexion Internet et d'une passion dévorante pour comprendre ce qui se cache derrière les machines.

Maker dans l'âme, il a construit, démonté et reconstruit des dizaines de projets allant des microcontrôleurs aux systèmes distribués en passant par l'impression 3D et l'électronique DIY. Ce goût du concret, de la chose que l'on peut toucher et transformer, a toujours orienté sa façon d'aborder le code : avant tout, un outil au service d'une intention.

Passionné d'informatique et d'intelligence artificielle depuis l'époque où les LLM n'existaient encore que dans des laboratoires de recherche confidentiels, Loïc a suivi l'explosion du deep learning avec la même ferveur qu'un alpiniste scrute un sommet encore non balisé. Il a plongé dans les transformers, exploré les embeddings, construit ses premiers RAG de prototype avant d'en faire des systèmes de production robustes.

Biohacker dans le sens le plus radical et le plus fascinant du terme, Loïc s'intéresse à la biologie en tant que terrain d'expérimentation et de détournement créatif. Captivé par la biologie synthétique et la génomique ouverte, il explore comment les micro-organismes — bactéries, levures, champignons — peuvent être reprogrammés pour produire des aliments, générer de l'énergie, fabriquer des biomatériaux ou concevoir des biosenseurs low-cost. Ce n'est pas la santé personnelle qui l'anime, mais la conviction que le vivant est le prochain grand terrain de l'ingénierie, et que les outils comme CRISPR, les cultures DIY en laboratoire ouvert, ou la fermentation dirigée ouvrent des possibilités que l'industrie traditionnelle n'a pas encore imaginées.

Amoureux de la nature et de la randonnée, Loïc retrouve dans les sentiers martiniquais — du Mont Pelé aux gorges de la Falaise — une forme de ressourcement fondamental. La forêt tropicale, la mer des Caraïbes, les traces oubliées : autant d'espaces où les idées se cristallisent et les problèmes techniques se résolvent souvent d'eux-mêmes, loin des écrans.

Il croit fermement que la prochaine révolution ne sera pas seulement technologique, mais humaine — celle d'individus capables d'augmenter leur propre intelligence, de maîtriser les outils d'IA et de les mettre au service de projets qui ont du sens. Ce livre est une contribution à cette vision.

Préface

Il y a quelque chose de particulièrement vertigineux dans le fait d'écrire un livre sur l'intelligence artificielle à l'aide de l'intelligence artificielle elle-même.

Ce projet est né d'une conviction simple : les systèmes RAG (Retrieval-Augmented Generation) représentent aujourd'hui l'une des architectures les plus puissantes et les plus sous-estimées du

paysage de l'IA applicative. Trop souvent relégués au rang de "simple wrapper autour d'un LLM", ils sont en réalité des infrastructures cognitives à part entière, capables de transformer la connaissance brute d'une organisation en réponses précises, traçables et industrialisables.

Mais pour les construire sérieusement — pour les déployer en production, les instrumenter, les évaluer et les maintenir — il faut une compréhension profonde de chaque composant. C'est l'ambition de ce livre.

Ce livre a été rédigé en collaboration étroite avec des modèles de langage avancés. Non pas pour déléguer la pensée, mais pour amplifier la capacité de structuration, d'explication et de synthèse. Chaque chapitre, chaque concept, chaque exemple de code a été guidé, relu, corrigé et validé par un regard humain — celui d'un praticien qui a construit ces systèmes, qui a rencontré leurs limites et qui en connaît les pièges.

Cette co-écriture est elle-même une démonstration de ce que l'IA, bien utilisée, permet : non pas remplacer l'expertise humaine, mais l'exprimer plus clairement, plus complètement, plus rapidement.

À vous, lecteur, qui tenez ce livre entre les mains — ou plutôt qui le lisez sur un écran —, je vous souhaite autant de plaisir à le parcourir que j'en ai eu à le construire.

Loïc Laureote

Martinique, 2025

Note d'intention

Ce livre s'adresse aux développeurs, ingénieurs et architectes qui souhaitent dépasser le stade du prototype et construire de véritables systèmes RAG en production. Il ne suppose aucune connaissance préalable des LLM, mais attend du lecteur une familiarité avec Python et les concepts de base de l'ingénierie logicielle.

Chaque chapitre alterne entre fondements théoriques, exemples de code commentés et conseils pratiques issus de déploiements réels. L'objectif n'est pas de vous donner une recette, mais de vous donner les clés pour prendre vos propres décisions architecturales en connaissance de cause.

Bonne lecture.

Partie I — Fondations solides du RAG industriel

Chapitre 1 — Comprendre l'architecture RAG en production

Section 1.1 — Définition d'un RAG industriel

Un **RAG** industriel (Retrieval-Augmented Generation en production) est une architecture logicielle qui combine un modèle de langage avec un système de recherche d'information externe, conçu non pas pour des démonstrations ou des prototypes, mais pour fonctionner de manière fiable, scalable et contrôlée dans un environnement réel.

Contrairement à un usage expérimental, où l'objectif est simplement de "répondre correctement à une question", un **RAG** industriel doit répondre à des contraintes strictes : latence faible, cohérence des réponses, traçabilité des sources, gestion de volumes importants de données et robustesse face aux erreurs et aux cas limites.

Dans une architecture **RAG** classique, le système suit une chaîne logique :

- Une question utilisateur est formulée.

Le système recherche des passages pertinents dans une base documentaire.

Ces passages sont transformés en représentation exploitable (souvent via **embeddings**).

Les documents les plus pertinents sont sélectionnés.

Le modèle de langage génère une réponse en s'appuyant sur ces éléments.

Dans un contexte industriel, cette chaîne devient plus complexe. Chaque étape est transformée en composant robuste et optimisé. L'objectif n'est plus seulement la pertinence, mais la fiabilité globale du système.

Un **RAG** industriel se distingue par plusieurs propriétés fondamentales.

D'abord, il est scalable. Il doit être capable de gérer des milliers, voire des millions de documents sans dégradation significative des performances. Cela implique l'utilisation d'index vectoriels optimisés, de stratégies de partitionnement (**sharding**) et de bases spécialisées comme Milvus ou Qdrant.

Ensuite, il est mesurable. Chaque composant du **pipeline** est évalué : qualité du **retrieval**, recall, précision, pertinence du **reranking**, taux d'hallucination du modèle génératif. Un **RAG** industriel n'est pas une boîte noire, mais un système instrumenté.

Il est également traçable. Chaque réponse doit pouvoir être reliée à des sources précises. Cela est essentiel dans les contextes professionnels (juridique, médical, industriel, support client), où une réponse sans justification n'a pas de valeur exploitable.

Un autre aspect essentiel est la robustesse. Le système doit continuer à fonctionner même en cas de données bruitées, de requêtes ambiguës ou de surcharge du système. Cela implique des mécanismes de fallback, de cache, de **reranking** et parfois de dégradation contrôlée des performances.

Enfin, un **RAG** industriel est intégré dans une architecture logicielle complète. Il n'existe pas comme un simple script Python, mais comme un ensemble de services : API de requête, **pipeline** d'ingestion, base vectorielle, système de monitoring, authentification, et souvent orchestration de conteneurs.

En résumé, un **RAG** industriel n'est pas une simple technique d'IA, mais une architecture de production complète pour transformer de la connaissance brute en réponse exploitable, fiable et industrialisable.

Section 1.2 — Pourquoi les LLM seuls ne suffisent pas

Les grands modèles de langage (**LLM**) constituent aujourd'hui une avancée majeure dans le domaine de l'intelligence artificielle. Ils sont capables de générer du texte cohérent, d'expliquer des concepts complexes et de simuler des raisonnements sur une grande variété de sujets. Cependant, dans un contexte de production, leur utilisation seule présente des limites structurelles importantes qui rendent leur usage insuffisant pour des systèmes fiables et industriels.

La première limite fondamentale est celle de la connaissance figée. Un **LLM** est entraîné sur un ensemble de données statiques, correspondant à un instant donné du monde numérique. Il ne dispose pas d'accès natif aux informations récentes ni aux données internes d'une organisation. Ainsi, toute information postérieure à sa date d'entraînement ou spécifique à une entreprise reste inaccessible, sauf intégration externe. Cela empêche toute mise à jour dynamique du savoir sans réentraînement coûteux.

La deuxième limite majeure concerne les **hallucinations**. Un **LLM** ne fonctionne pas comme une base de données vérifiant la vérité des faits. Il génère la suite de tokens la plus probable en fonction du contexte, ce qui peut conduire à des réponses plausibles mais incorrectes. En environnement de production, cette caractéristique est problématique car elle peut produire des informations erronées sans signal d'alerte explicite.

Une troisième limitation est l'absence de traçabilité des réponses. Un **LLM** ne fournit pas naturellement de source ou de justification pour ses affirmations. Il est donc impossible de relier une réponse à un document précis ou à une donnée vérifiable sans mécanisme additionnel. Dans les contextes professionnels (juridique, industriel, support client), cette absence de transparence est un obstacle majeur à l'adoption.

Par ailleurs, les **LLM** seuls ne disposent pas d'un accès direct aux données privées ou spécifiques à un domaine. Ils ne peuvent pas interroger une base documentaire interne, analyser des fichiers

métier ou exploiter des bases de connaissances propriétaires sans une architecture externe. Cela limite fortement leur capacité à répondre à des questions contextualisées à une organisation.

Une autre contrainte importante est liée au coût et à la latence. Pour compenser l'absence de mémoire externe, il serait nécessaire d'injecter un grand volume de contexte directement dans les prompts. Cette approche entraîne une augmentation significative du nombre de tokens traités, ce qui augmente à la fois le coût d'exécution et le temps de réponse.

Enfin, les **LLM** ne possèdent pas de mémoire évolutive propre. Ils ne s'adaptent pas automatiquement aux nouvelles informations d'une entreprise ou d'un système. Chaque évolution nécessite soit un réentraînement, soit une reconstruction du contexte à chaque requête.

En résumé, un **LLM** seul est un puissant générateur de texte, mais il reste fondamentalement limité en termes de fiabilité, d'actualisation des connaissances et d'intégration dans des systèmes réels. C'est précisément pour répondre à ces contraintes que les architectures de type **RAG** ont été développées, en introduisant un mécanisme de récupération d'information externe structuré et vérifiable.

Section 1.3 — Contraintes de production (latence, coût, scalabilité)

Dans un contexte industriel, un système **RAG** ne peut pas être évalué uniquement sur la qualité de ses réponses. Sa valeur réelle dépend de sa capacité à fonctionner sous contraintes opérationnelles strictes, propres aux environnements de production. Parmi ces contraintes, trois dimensions structurent l'ensemble de l'architecture : la latence, le coût et la **scalabilité**.

La latence correspond au temps nécessaire pour produire une réponse complète à partir d'une requête utilisateur. Dans un système **RAG**, cette latence ne dépend pas uniquement du modèle de langage, mais de l'ensemble de la chaîne de traitement : recherche vectorielle, éventuel **reranking**, construction du contexte et génération de la réponse. Chaque étape ajoute un coût temporel. En production, une latence trop élevée dégrade directement l'expérience utilisateur et limite les cas d'usage interactifs. Un système RAG industriel doit donc optimiser chaque maillon de la chaîne afin de garantir des temps de réponse compatibles avec une utilisation en temps réel ou quasi temps réel.

Le coût constitue une deuxième contrainte majeure. Contrairement à un prototype, où les requêtes sont peu nombreuses, un système en production peut traiter des milliers voire des millions d'appels par jour. Le coût est alors déterminé par plusieurs facteurs : utilisation des modèles d'**embedding**, appels aux modèles de langage, stockage des vecteurs et infrastructure de recherche. Sans optimisation, le coût peut croître de manière exponentielle avec le volume de données et le nombre d'utilisateurs. C'est pourquoi les architectures **RAG** industrielles intègrent souvent des mécanismes de cache, de réduction du contexte et de sélection intelligente des documents.

La **scalabilité** représente la capacité du système à absorber une augmentation de charge sans dégradation significative des performances. Elle concerne à la fois le volume de documents à indexer, le nombre d'utilisateurs simultanés et la fréquence des requêtes. Une architecture non scalable peut fonctionner correctement sur un petit corpus, mais devenir inutilisable dès que la base de connaissances atteint une certaine taille. Pour répondre à cette contrainte, les systèmes **RAG**

industriels reposent sur des bases vectorielles optimisées, des stratégies d'indexation approximative (**ANN**), ainsi que des architectures distribuées permettant de répartir la charge.

Ces trois contraintes sont fortement interdépendantes. Une optimisation de la latence peut augmenter le coût, une réduction du coût peut dégrader la qualité, et une amélioration de la **scalabilité** peut complexifier l'architecture globale. Le rôle de l'ingénierie **RAG** en production consiste précisément à trouver un équilibre entre ces dimensions, en fonction du cas d'usage cible.

Ainsi, un système **RAG** industriel ne se conçoit pas uniquement comme une chaîne de traitement intelligente, mais comme un système d'ingénierie complet, soumis aux mêmes exigences que les infrastructures logicielles critiques.

Section 1.4 — RAG vs Fine-tuning vs Agents

Dans les systèmes d'intelligence artificielle modernes, trois approches principales sont souvent utilisées pour adapter les grands modèles de langage à des cas d'usage concrets : le **RAG** (Retrieval-Augmented Generation), le **fine-tuning**, et les agents IA. Bien qu'elles puissent sembler similaires dans leur objectif — améliorer les performances d'un modèle — elles reposent sur des principes fondamentalement différents et répondent à des besoins distincts en production.

Le **RAG** repose sur une idée simple : au lieu de modifier le modèle, on lui fournit dynamiquement les informations nécessaires au moment de la requête. Le système récupère des documents pertinents depuis une base externe, puis les injecte dans le contexte du modèle pour générer une réponse. Cette approche permet de travailler avec des données à jour, privées ou spécifiques à un domaine, sans réentraîner le modèle. En production, le RAG est particulièrement adapté aux systèmes documentaires, aux assistants internes et aux cas où la traçabilité des sources est essentielle.

Le **fine-tuning**, à l'inverse, consiste à modifier directement les paramètres du modèle en l'entraînant sur un jeu de données spécifique. Cette approche permet d'adapter profondément le comportement du modèle à un domaine particulier, par exemple le juridique, le médical ou le support client. Cependant, elle présente des contraintes importantes en production : coût élevé d'entraînement, difficulté de mise à jour continue, risque d'oubli catastrophique et absence de flexibilité face à des données changeantes. Une fois fine-tuné, le modèle reste figé jusqu'à une nouvelle phase d'entraînement.

Les agents IA représentent une approche différente encore. Un agent est un système basé sur un **LLM** capable de planifier et d'exécuter plusieurs actions de manière autonome. Il peut appeler des outils, interroger des APIs, effectuer des recherches ou enchaîner plusieurs étapes de raisonnement. Contrairement au **RAG**, qui est centré sur la récupération de contexte, et au **fine-tuning**, qui agit sur les paramètres du modèle, l'agent introduit une couche de logique d'exécution dynamique. Il transforme le modèle en composant décisionnel capable d'interagir avec un environnement.

Dans un contexte industriel, ces trois approches ne sont pas exclusives mais complémentaires. Le **RAG** est souvent utilisé comme base pour fournir des connaissances fiables et actualisées. Le **fine-tuning** est réservé aux cas où un comportement très spécifique du modèle est nécessaire et

stable dans le temps. Les agents, quant à eux, sont utilisés pour orchestrer des workflows complexes impliquant plusieurs outils et étapes.

Le choix entre ces approches dépend principalement de trois facteurs : la nature des données, la stabilité des connaissances et la complexité des tâches à réaliser. Un système documentaire interne privilégiera généralement le **RAG**. Un modèle spécialisé dans un langage ou un style très particulier pourra nécessiter du **fine-tuning**. Un système automatisé multi-étapes (analyse, recherche, action) bénéficiera davantage d'une architecture agentique.

En pratique, les systèmes de production modernes combinent souvent ces trois approches dans une architecture hybride : un agent orchestre les actions, un **RAG** fournit la connaissance, et éventuellement un modèle fine-tuné améliore la qualité de génération dans un domaine précis.

Ainsi, plutôt que de les opposer, il est plus juste de considérer le **RAG**, le **fine-tuning** et les agents comme trois couches complémentaires d'une même évolution vers des systèmes d'intelligence artificielle plus robustes, adaptatifs et intégrés.

Chapitre 2 — Embeddings et représentation sémantique

Section 2.1 — Espace vectoriel et sémantique

Les systèmes de **RAG** reposent sur une transformation fondamentale du langage : au lieu de manipuler du texte brut, ils représentent les informations sous forme de vecteurs numériques dans un espace mathématique. Cette représentation permet de passer d'une logique symbolique (mots, phrases, documents) à une logique géométrique où la similarité entre concepts devient mesurable.

Un espace vectoriel sémantique est un espace multidimensionnel dans lequel chaque texte — qu'il s'agisse d'un mot, d'une phrase ou d'un document entier — est encodé sous la forme d'un vecteur. Ces vecteurs sont généralement produits par des modèles d'**embeddings** entraînés pour capturer le sens des textes plutôt que leur simple forme lexicale.

L'idée centrale est que des éléments sémantiquement proches dans le langage naturel se retrouvent proches dans cet espace vectoriel. Par exemple, les phrases "voiture électrique" et "véhicule à batterie" ne partagent pas nécessairement les mêmes mots, mais elles seront projetées dans des régions proches de l'espace vectoriel, car elles décrivent des concepts similaires.

Cette propriété est ce qui rend possible la recherche sémantique utilisée dans les systèmes **RAG**. Contrairement à une recherche classique basée sur des mots-clés, la recherche vectorielle permet de retrouver des documents pertinents même lorsque la formulation diffère fortement de la requête initiale.

Mathématiquement, chaque élément est représenté par un vecteur de dimension fixe :

- un mot peut être représenté dans un espace de 384, 768 ou 1024 dimensions
- un document est souvent représenté par un vecteur agrégé de ses segments

Ces dimensions n'ont pas de signification interprétable individuellement, mais collectivement elles encodent des relations complexes entre concepts, contextes et usages linguistiques.

Dans cet espace, la notion de distance devient centrale. Deux vecteurs proches correspondent à des contenus sémantiquement similaires, tandis que des vecteurs éloignés représentent des concepts non liés. Cette distance est généralement mesurée par des métriques telles que la **similarité cosinus**, qui évalue l'angle entre deux vecteurs plutôt que leur magnitude.

L'intérêt de cette représentation est qu'elle permet de transformer un problème linguistique complexe en un problème géométrique optimisable. La recherche d'information devient alors une opération de type "nearest neighbors", où l'on cherche les vecteurs les plus proches d'une requête dans un espace de grande dimension.

Dans le cadre des systèmes **RAG**, cet espace vectoriel est construit à partir de modèles d'**embeddings** entraînés sur de grands corpus textuels. Ces modèles apprennent implicitement des structures linguistiques profondes, telles que les synonymies, les relations hiérarchiques ou les contextes d'usage.

Ainsi, l'espace vectoriel sémantique constitue la base fondamentale du **retrieval** moderne. Il permet de dépasser les limites des approches lexicales traditionnelles pour accéder à une compréhension approximative mais robuste du sens, qui est essentielle pour connecter efficacement une question utilisateur à des connaissances pertinentes.

Section 2.2 — Similarité cosinus et métriques

Dans les systèmes de recherche vectorielle utilisés en **RAG**, la notion de similarité est centrale. Une fois les textes transformés en vecteurs dans un espace sémantique, il devient nécessaire de définir une mesure permettant d'évaluer à quel point deux vecteurs sont proches. Cette mesure est appelée métrique de similarité.

Parmi les différentes métriques existantes, la **similarité cosinus** est la plus utilisée dans les systèmes de recherche sémantique modernes. Elle mesure l'angle entre deux vecteurs plutôt que leur distance absolue, ce qui permet de comparer efficacement des représentations indépendamment de leur magnitude.

Mathématiquement, la **similarité cosinus** entre deux vecteurs A et B est définie par le rapport entre leur produit scalaire et le produit de leurs normes :

$$\cos(\theta) = \frac{A \cdot B}{|A| |B|}$$

Cette formule permet de produire une valeur comprise entre -1 et 1, où :

- 1 indique que les vecteurs sont parfaitement alignés (forte similarité sémantique)

0 indique une absence de corrélation directionnelle

-1 indique une opposition totale dans l'espace vectoriel

Dans les applications **RAG**, les **embeddings** sont généralement normalisés, ce qui fait que la **similarité cosinus** devient équivalente à une comparaison directionnelle pure. Cela simplifie les calculs et améliore la stabilité des résultats.

Cependant, la **similarité cosinus** n'est qu'une des nombreuses métriques possibles. D'autres mesures peuvent être utilisées selon les contraintes du système :

La distance euclidienne mesure la distance directe entre deux points dans l'espace vectoriel. Elle est intuitive mais sensible à la magnitude des vecteurs, ce qui peut introduire des biais si les **embeddings** ne sont pas normalisés.

La distance de Manhattan (ou L1) calcule la somme des différences absolues entre les composantes des vecteurs. Elle est parfois utilisée dans des contextes spécifiques mais reste moins adaptée aux **embeddings** sémantiques.

Dans les systèmes de recherche modernes, on utilise également des métriques optimisées pour les structures d'index approximatifs (**ANN**), où l'objectif n'est pas uniquement la précision mathématique, mais aussi la rapidité de recherche dans des espaces de très haute dimension.

Le choix de la métrique a un impact direct sur la qualité du **retrieval**. Une mauvaise métrique peut entraîner des résultats incohérents, même si les **embeddings** sont de bonne qualité. À l'inverse, une bonne métrique permet d'extraire efficacement les documents les plus pertinents, même dans des bases contenant des millions de vecteurs.

Ainsi, la **similarité cosinus** constitue le standard de facto des systèmes **RAG** modernes, car elle offre un compromis optimal entre simplicité, robustesse et efficacité dans les espaces vectoriels sémantiques.

Section 2.3 — Modèles d'embeddings (BGE, E5, SBERT)

Les modèles d'**embeddings** constituent le cœur du système de recherche vectorielle dans une architecture **RAG**. Leur rôle est de transformer des textes en représentations numériques denses, capables de capturer le sens sémantique au-delà des mots utilisés. Le choix du modèle d'**embedding** a un impact direct sur la qualité du **retrieval**, et donc sur la pertinence globale du système.

Dans les systèmes modernes, trois familles de modèles sont largement utilisées : Sentence-BERT (SBERT), E5, et BGE (BAAI General **embedding**). Chacune de ces approches représente une évolution dans la manière d'apprendre des représentations sémantiques efficaces pour la recherche.

Les modèles SBERT ont été parmi les premiers à rendre les **embeddings** exploitables à grande échelle pour la similarité de phrases. Ils reposent sur une adaptation de BERT, optimisée pour produire des embeddings comparables via des fonctions de distance comme la **similarité cosinus**. SBERT a introduit une approche simple mais efficace pour transformer des modèles de langage en encodeurs de phrases utilisables dans des systèmes de recherche.

Cependant, SBERT montre des limites dans les contextes industriels modernes, notamment en termes de précision sur des corpus hétérogènes et de robustesse sur des requêtes complexes.

Les modèles E5 (**embedding** from Encoder Enhanced) représentent une évolution importante. Ils introduisent une formulation plus structurée du problème d'embedding en intégrant explicitement le rôle de la requête et du document dans l'apprentissage. E5 améliore significativement la qualité du **retrieval** en optimisant directement la capacité du modèle à rapprocher requêtes et passages pertinents dans l'espace vectoriel. Cette approche est particulièrement efficace dans les systèmes **RAG** où la distinction entre query et document est fondamentale.

Les modèles BGE (BAAI General **embedding**) constituent aujourd'hui l'un des standards les plus performants pour la recherche sémantique. Ils sont entraînés sur de grands corpus avec des stratégies avancées de contrastive learning et sont souvent optimisés pour des scénarios de **retrieval** industriel. BGE offre généralement une meilleure précision que SBERT et une robustesse supérieure à E5 dans des contextes variés, notamment lorsque les données sont bruitées ou hétérogènes.

Un aspect important dans le choix d'un modèle d'**embedding** est le compromis entre performance et coût de calcul. Les modèles plus récents comme BGE offrent une meilleure qualité mais peuvent être plus lourds à exécuter, ce qui a un impact direct sur la latence et les ressources nécessaires en production. À l'inverse, des modèles plus légers comme certaines variantes de SBERT peuvent être privilégiés dans des systèmes contraints en ressources.

Dans un système **RAG** industriel, le choix du modèle d'**embedding** n'est jamais isolé. Il dépend du type de données, du volume de documents, des exigences de latence et du niveau de précision attendu. Dans de nombreux cas, les architectures hybrides combinent plusieurs modèles ou utilisent des **embeddings** rapides pour la pré-recherche, suivis de modèles plus précis pour le **reranking**.

Ainsi, les modèles d'**embeddings** constituent une brique essentielle du **RAG** moderne, car ils déterminent la qualité de la représentation sémantique et conditionnent directement l'efficacité du système de **retrieval**.

Section 2.4 — Impact du modèle sur la qualité RAG

Dans une architecture **RAG**, le modèle d'**embedding** n'est pas un simple composant technique interchangeable : il conditionne directement la qualité globale du système. En pratique, la performance d'un RAG dépend autant du modèle de récupération que du modèle de génération. Un **LLM** performant ne peut pas compenser un **retrieval** de mauvaise qualité, car il ne travaille qu'à partir du contexte qui lui est fourni.

L'impact du modèle d'**embedding** se manifeste d'abord sur la pertinence du **retrieval**. Un bon modèle permet de rapprocher correctement une requête utilisateur des passages réellement utiles dans la base documentaire. À l'inverse, un modèle insuffisant peut introduire du bruit en sélectionnant des documents sémantiquement proches mais non pertinents, ce qui dégrade directement la qualité de la réponse générée.

Cette étape est critique, car le **RAG** repose sur une hypothèse fondamentale : la qualité de la réponse est bornée par la qualité du contexte récupéré. Ainsi, si les documents extraits sont incomplets, mal classés ou hors sujet, le modèle de langage produira inévitablement une réponse approximative, voire erronée.

Un second impact important concerne la robustesse aux variations linguistiques. Les utilisateurs ne formulent pas toujours leurs questions de manière uniforme. Un bon modèle d'**embedding** doit être capable de comprendre des reformulations, des synonymes, des paraphrases et des structures syntaxiques différentes tout en ramenant les mêmes contenus pertinents. Les modèles modernes comme BGE ou E5 améliorent fortement cette capacité par rapport aux approches plus anciennes.

Le modèle influence également la densité informationnelle du **retrieval**. Dans un système **RAG**, le nombre de chunks envoyés au **LLM** est limité par la fenêtre de contexte. Un modèle d'**embedding** performant permet de maximiser la densité d'information utile dans cette fenêtre, en sélectionnant des passages à forte valeur sémantique plutôt que des fragments redondants ou faibles.

Un autre aspect critique est la stabilité des performances en environnement réel. En production, les données ne sont pas propres ni homogènes : elles contiennent du bruit, des formats variés, des documents incomplets ou mal structurés. Certains modèles d'**embeddings** sont plus sensibles à ces variations que d'autres. Un modèle robuste maintient une performance stable malgré la diversité des entrées, ce qui est essentiel pour un système industriel.

Le choix du modèle a également un impact direct sur la latence globale du système. Les modèles plus lourds produisent généralement des **embeddings** de meilleure qualité, mais augmentent le temps de calcul lors de l'ingestion et parfois lors des requêtes. Cela impose un compromis entre précision et performance, particulièrement dans les systèmes à grande échelle ou en temps réel.

Enfin, le modèle d'**embedding** influence indirectement le besoin de composants additionnels tels que le **reranking**. Un modèle de faible qualité nécessite souvent une couche supplémentaire de reranking pour corriger les erreurs de **retrieval**, tandis qu'un modèle plus avancé peut réduire fortement ce besoin.

En résumé, le modèle d'**embedding** agit comme un facteur multiplicatif sur la qualité globale du système **RAG**. Il ne détermine pas uniquement la performance du **retrieval**, mais conditionne l'efficacité de toute la chaîne : du contexte fourni au **LLM** jusqu'à la pertinence finale de la réponse. Dans un système industriel, le choix de ce modèle constitue donc une décision architecturale majeure, au même niveau que le choix du LLM ou de la base vectorielle.

Chapitre 3 — Indexation et bases vectorielles

Section 3.1 — Vector databases (Milvus, Qdrant, Weaviate)

Les bases de données vectorielles constituent un composant central des systèmes **RAG** modernes. Elles sont responsables du stockage, de l'indexation et de la recherche efficace de millions de représentations vectorielles issues des **embeddings**. Contrairement aux bases de données

traditionnelles, qui manipulent des données structurées ou textuelles, les vector databases sont conçues pour gérer des espaces de très haute dimension et permettre des recherches de similarité rapides et scalables.

Dans une architecture **RAG** industrielle, leur rôle est critique : elles assurent la transition entre la représentation sémantique des documents et la capacité du système à retrouver rapidement les informations pertinentes. Sans elles, la recherche vectorielle à grande échelle serait trop lente pour des usages en production.

Parmi les solutions les plus utilisées aujourd'hui, trois technologies dominent le marché : Milvus, **Qdrant** et Weaviate. Chacune propose une approche différente de l'indexation vectorielle et de la **scalabilité**.

Milvus est une base vectorielle conçue pour les environnements à très grande échelle. Elle est optimisée pour le traitement de millions voire de milliards de vecteurs, avec une architecture distribuée. Milvus repose sur des index **ANN** (Approximate Nearest Neighbors) comme **IVF** et **HNSW**, permettant de réduire drastiquement le coût de la recherche dans des espaces de haute dimension. Son principal avantage réside dans sa capacité à scaler horizontalement, ce qui en fait une solution adaptée aux systèmes industriels nécessitant une forte capacité de traitement.

Qdrant adopte une approche plus légère et orientée performance en temps réel. Elle est souvent utilisée dans des systèmes nécessitant une latence faible et une intégration rapide. Qdrant se distingue par sa simplicité d'utilisation, son API claire et son efficacité sur des volumes de données moyens à élevés. Elle est particulièrement adaptée aux architectures **RAG** nécessitant une mise en production rapide avec des contraintes de performance strictes.

Weaviate se positionne comme une solution hybride entre base vectorielle et base de connaissances. Elle intègre nativement des fonctionnalités de gestion de schémas, de filtres avancés et parfois même de modules d'**embeddings** intégrés. Weaviate est particulièrement intéressante dans les systèmes où les données ne sont pas uniquement vectorielles mais également fortement structurées et enrichies de métadonnées complexes.

Le choix d'une base vectorielle dépend fortement des contraintes du système. Les facteurs déterminants incluent le volume de données, les exigences de latence, la complexité des requêtes et les besoins en **scalabilité**. Dans un système **RAG** industriel, il est courant de privilégier Milvus pour les architectures à grande échelle, **Qdrant** pour des systèmes plus agiles et rapides à déployer, et Weaviate pour des cas d'usage nécessitant une forte intégration de la connaissance structurée.

Au-delà des différences fonctionnelles, toutes ces bases reposent sur un principe commun : l'utilisation d'algorithmes de recherche approximative (**ANN**). Ces algorithmes permettent de retrouver rapidement les vecteurs les plus proches sans effectuer une comparaison exhaustive, ce qui rend possible la recherche en temps réel dans des espaces contenant des millions d'éléments.

Ainsi, les vector databases ne sont pas simplement des outils de stockage, mais des composants d'infrastructure critiques qui conditionnent directement la performance, la **scalabilité** et la fiabilité d'un système **RAG** en production.

Section 3.2 — Index ANN (HNSW, IVF, PQ)

Dans les bases de données vectorielles, la recherche de similarité consiste à identifier les vecteurs les plus proches d'une requête dans un espace de très haute dimension. Cependant, une recherche exhaustive (brute force) devient rapidement impraticable dès que le volume de données atteint plusieurs centaines de milliers ou millions de vecteurs. Pour répondre à cette contrainte, les systèmes **RAG** industriels utilisent des structures d'index appelées **ANN** (Approximate Nearest Neighbors).

Les index **ANN** permettent de sacrifier une légère partie de précision au profit d'un gain considérable en performance. L'objectif n'est pas de trouver exactement le voisin le plus proche, mais de trouver rapidement un ensemble de candidats très proches, suffisamment pertinents pour les besoins du système.

Parmi les approches les plus utilisées en production, trois familles d'index dominent : **HNSW**, **IVF** et **PQ**. Chacune repose sur une stratégie différente pour réduire la complexité de la recherche dans l'espace vectoriel.

HNSW — Hierarchical Navigable Small World

L'algorithme **HNSW** (Hierarchical Navigable Small World) repose sur une structure de graphe hiérarchique. Les vecteurs sont organisés sous forme de nœuds connectés entre eux, permettant de naviguer efficacement dans l'espace de recherche.

L'idée principale est de construire plusieurs niveaux de graphes :

- les niveaux supérieurs contiennent des connexions globales et approximatives
- les niveaux inférieurs permettent une recherche plus fine et précise

Lors d'une requête, le système commence par les couches supérieures pour localiser rapidement une zone proche, puis descend progressivement vers des couches plus fines pour affiner les résultats.

HNSW est particulièrement performant en termes de latence et offre un excellent compromis entre précision et vitesse, ce qui en fait l'un des index les plus utilisés dans les systèmes **RAG** modernes.

IVF — Inverted File Index

L'**IVF** (Inverted File Index) repose sur une logique de clustering. L'espace vectoriel est divisé en plusieurs groupes (clusters) représentés par des centroïdes. Chaque vecteur est associé à un cluster spécifique.

Lors d'une requête, le système ne compare pas la requête à tous les vecteurs, mais uniquement aux clusters les plus proches. Cela réduit drastiquement le nombre de comparaisons nécessaires.

Cette approche est particulièrement efficace pour les très grands volumes de données, mais sa performance dépend fortement de la qualité du clustering initial. Un mauvais partitionnement peut dégrader la qualité du **retrieval**.

PQ — Product Quantization

Le PQ (Product Quantization) est une technique de compression des vecteurs. Au lieu de stocker des vecteurs complets en mémoire, ils sont approximés à l'aide de sous-espaces quantifiés.

Chaque vecteur est découpé en plusieurs sous-vecteurs, chacun étant remplacé par un code représentant une approximation. Cette compression permet de réduire considérablement l'usage mémoire et d'accélérer les calculs de distance.

Le PQ est souvent utilisé en combinaison avec **IVF** dans des architectures hybrides, afin de concilier compression et efficacité de recherche.

Compromis et choix d'index

Le choix d'un index **ANN** dépend des contraintes du système :

- **HNSW** est privilégié pour les systèmes nécessitant une faible latence et une haute précision

IVF est adapté aux très grandes bases de données nécessitant une bonne **scalabilité**

PQ est utilisé lorsque la mémoire est une contrainte critique

Dans les systèmes **RAG** industriels, ces index ne sont pas utilisés isolément mais souvent combinés dans des architectures hybrides pour optimiser simultanément la vitesse, la mémoire et la qualité de recherche.

Ainsi, les index **ANN** constituent un pilier fondamental de la recherche vectorielle moderne, rendant possible l'exploitation de bases de connaissances massives dans des conditions de production.

Section 3.3 — Scalabilité à grande échelle

Section 3.3 — Scalabilité à grande échelle

La **scalabilité** est l'une des contraintes les plus déterminantes dans la conception d'un système **RAG** industriel. Elle désigne la capacité d'une architecture à maintenir ses performances lorsque le volume de données, le nombre d'utilisateurs ou le débit de requêtes augmente de manière significative. Dans un contexte réel, un système RAG n'est jamais statique : il évolue constamment avec l'ajout de nouveaux documents, de nouveaux utilisateurs et de nouvelles exigences métiers.

Contrairement à un prototype qui fonctionne sur quelques milliers de documents, un système en production doit souvent gérer des volumes allant de plusieurs millions à plusieurs milliards de chunks vectorisés. Cette montée en charge impose des choix architecturaux spécifiques dès la conception du système.

La première dimension de la **scalabilité** concerne le stockage des vecteurs. Les **embeddings** occupent un espace mémoire important, surtout lorsque le corpus devient massif. Pour répondre à cette contrainte, les bases vectorielles comme Milvus, **Qdrant** ou Weaviate utilisent des mécanismes de partitionnement et de distribution des données. Le stockage est réparti sur plusieurs nœuds afin d'éviter les goulots d'étranglement et permettre une extension horizontale du système.

La deuxième dimension est celle du calcul de la recherche. Lorsqu'un utilisateur effectue une requête, le système doit comparer son **embedding** à des millions de vecteurs potentiels. Sans optimisation, cette opération serait trop lente pour des usages interactifs. Les index **ANN** (Approximate Nearest Neighbors) jouent ici un rôle essentiel en réduisant drastiquement le nombre de comparaisons nécessaires, permettant ainsi de conserver une latence acceptable même à grande échelle.

La troisième dimension est la gestion du débit de requêtes. Dans un système **RAG** en production, plusieurs utilisateurs peuvent interroger simultanément la base de connaissances. Cela nécessite des mécanismes de concurrence, de mise en file d'attente et parfois de cache pour éviter la surcharge du système. Les architectures modernes utilisent souvent des systèmes distribués capables de répartir la charge entre plusieurs instances de calcul.

Un autre aspect critique de la **scalabilité** est la mise à jour dynamique des données. Contrairement à une base statique, un système **RAG** doit intégrer en continu de nouveaux documents sans interrompre le service. Cela implique des pipelines d'ingestion capables de traiter les données en streaming, de générer les **embeddings** à la volée et de les insérer dans l'index sans réindexation complète.

La **scalabilité** ne se limite pas à la performance brute. Elle inclut également la stabilité du système dans le temps. À mesure que les données évoluent, les distributions vectorielles peuvent changer, ce qui peut impacter la qualité du **retrieval**. Il devient alors nécessaire de surveiller les performances du système et de réajuster les paramètres d'indexation ou de **reranking** si nécessaire.

Enfin, la **scalabilité** a également un impact économique direct. Plus le système grandit, plus les coûts liés au stockage, au calcul des **embeddings** et aux appels **LLM** augmentent. Une architecture **RAG** bien conçue doit donc intégrer des stratégies d'optimisation du coût, comme la compression des embeddings, le caching des résultats fréquents ou la réduction du nombre de requêtes vers les modèles de génération.

Ainsi, la **scalabilité** à grande échelle ne se limite pas à un problème technique isolé, mais constitue une contrainte transversale qui influence l'ensemble de l'architecture **RAG**. Elle conditionne à la fois les choix de stockage, de calcul, de **pipeline** et de monitoring, et représente un élément fondamental de toute conception orientée production.

Section 3.4 — Choix d'architecture en production

Le choix d'une architecture **RAG** en production ne consiste pas uniquement à assembler des composants techniques, mais à concevoir un système cohérent capable de répondre à des contraintes réelles de performance, de coût, de maintenance et d'évolution. Contrairement à un prototype, où l'objectif est de valider une idée, une architecture de production doit être robuste, observable et extensible.

La première décision structurante concerne le niveau de complexité de l'architecture. Trois grandes approches existent. La première est une architecture monolithique, où ingestion, indexation, **retrieval** et génération sont regroupés dans un même service. Cette approche est simple à mettre en place mais rapidement limitée en **scalabilité**. La deuxième est une architecture modulaire, où chaque composant (ingestion, vector DB, API, **LLM**) est séparé en services indépendants. La troisième, plus avancée, est une architecture distribuée orientée microservices et événements, capable de gérer des charges importantes et des flux de données continus.

Le deuxième facteur clé est le choix de la base vectorielle et de son mode de déploiement. Une base comme Milvus peut être utilisée en mode local pour des tests, mais en production elle est généralement déployée en cluster distribué. À l'inverse, des solutions plus légères comme **Qdrant** peuvent être utilisées en single-node pour des systèmes à faible ou moyenne charge, avec une évolution possible vers des déploiements horizontaux.

Un autre élément déterminant est la stratégie de **pipeline** d'ingestion. Dans une architecture simple, l'ingestion est synchrone : un document est traité et immédiatement indexé. Dans une architecture industrielle, l'ingestion est souvent asynchrone et pilotée par une file de messages. Cela permet de découpler la charge d'entrée des capacités de traitement et d'éviter les blocages lors de pics de volume.

Le choix du modèle de **retrieval** influence également l'architecture globale. Un système basé uniquement sur des **embeddings** nécessite une **pipeline** simple vector search → **LLM**. En revanche, un système hybride intégrant **BM25**, **reranking** et filtres métadonnées impose plusieurs couches supplémentaires dans le pipeline de requête. Cette complexité augmente la latence mais améliore significativement la qualité des résultats.

La question du déploiement et de l'infrastructure est également centrale. Les systèmes **RAG** modernes sont généralement conteneurisés via **Docker** et orchestrés via Docker Compose ou Kubernetes. Cette approche permet d'assurer la reproductibilité, la **scalabilité** et la résilience du système. Elle facilite également les mises à jour progressives et la gestion des versions des différents composants.

Un autre critère important est l'observabilité. En production, il est indispensable de monitorer les performances du système à plusieurs niveaux : latence des requêtes, taux de récupération, qualité des réponses, consommation mémoire et coût des appels **LLM**. Sans ces métriques, il devient impossible d'optimiser ou de diagnostiquer les problèmes.

Enfin, le choix d'architecture dépend fortement du cas d'usage métier. Un système **RAG** interne pour une PME n'a pas les mêmes contraintes qu'un assistant juridique à grande échelle ou qu'un moteur de recherche documentaire industriel. Le niveau de complexité doit toujours être

proportionné aux besoins réels, afin d'éviter une sur-architecture inutilement coûteuse.

Ainsi, choisir une architecture **RAG** en production revient à arbitrer en permanence entre simplicité, performance, coût et évolutivité. L'objectif n'est pas de construire le système le plus sophistiqué possible, mais celui qui répond le mieux aux contraintes opérationnelles tout en restant maintenable sur le long terme.

Chapitre 4 — Ingestion documentaire industrielle

Chapitre 4 — Ingestion documentaire industrielle

L'ingestion documentaire est l'une des briques les plus critiques d'un système **RAG** en production. Elle constitue le point d'entrée de toute la connaissance exploitable par le système. Si cette étape est mal conçue, même les meilleurs modèles d'**embeddings** et les architectures de **retrieval** les plus avancées ne pourront compenser la perte de qualité introduite en amont. L'ingestion n'est donc pas une simple étape technique, mais un véritable **pipeline** d'ingénierie de la donnée.

Section 4.1 — Pipeline d'ingestion robuste

Un **pipeline** d'ingestion robuste a pour objectif de transformer des documents bruts hétérogènes en unités d'information structurées, exploitables par un système de recherche vectorielle. Dans un contexte industriel, ces documents peuvent provenir de sources variées : PDF, pages web, fichiers Word, bases de données, logs ou encore documents scannés. Cette diversité impose une architecture capable de standardiser et normaliser les données avant leur indexation.

La première étape du **pipeline** consiste en l'acquisition des documents. Cette phase regroupe la collecte des données depuis différentes sources et leur centralisation dans un système unique de traitement. Dans une architecture moderne, cette étape est souvent découplée du reste du pipeline via des systèmes de files de messages ou des orchestrateurs de workflows, afin de gérer les flux de données asynchrones et les pics de charge.

Vient ensuite la phase de prétraitement et de nettoyage. Les documents bruts contiennent fréquemment du bruit : caractères spéciaux, erreurs d'encodage, structures incohérentes ou éléments non textuels. Le rôle du **pipeline** est ici de normaliser le contenu en un format homogène. Cela inclut la suppression des artefacts, la correction des encodages, ainsi que l'extraction du texte brut à partir de formats complexes comme les PDF ou les documents scannés.

Une fois les données nettoyées, le système procède à la structuration du contenu. Cette étape consiste à découper les documents en unités logiques cohérentes, souvent appelées chunks. Contrairement à une segmentation naïve basée sur un nombre fixe de caractères, un **pipeline** industriel privilégie des stratégies plus avancées prenant en compte la structure sémantique du texte, comme les sections, les paragraphes ou les titres hiérarchiques.

Après la structuration, les chunks sont enrichis par des métadonnées. Ces informations peuvent inclure la source du document, la date de création, l'auteur, le type de contenu ou encore des tags métier. Ces métadonnées jouent un rôle essentiel dans les étapes ultérieures de **retrieval**, notamment pour le filtrage et la priorisation des résultats.

La phase suivante est celle de la vectorisation, où chaque chunk est transformé en **embedding** via un modèle de représentation sémantique. Cette étape est critique car elle conditionne directement la qualité de la recherche future. Dans un **pipeline** robuste, cette génération d'**embeddings** est souvent parallélisée et optimisée pour gérer de grands volumes de données sans dégradation de performance.

Enfin, les vecteurs générés sont insérés dans une base de données vectorielle. Cette étape inclut non seulement le stockage des **embeddings**, mais également la construction des index **ANN** nécessaires à la recherche rapide. Dans les systèmes industriels, cette insertion doit être optimisée pour supporter des mises à jour continues sans nécessiter de reconstruction complète de l'index.

Un **pipeline** d'ingestion robuste se distingue également par sa capacité à gérer les erreurs et les cas limites. Les documents corrompus, incomplets ou mal formatés ne doivent pas bloquer l'ensemble du processus. Des mécanismes de retry, de validation et de fallback sont donc intégrés pour garantir la résilience du système.

En résumé, le **pipeline** d'ingestion n'est pas une simple suite d'opérations techniques, mais une chaîne de transformation industrielle de la donnée. Il garantit que l'information brute est convertie en une représentation structurée, fiable et exploitable, constituant ainsi la base sur laquelle repose l'ensemble du système **RAG**.

Section 4.2 — Parsing multi-formats (PDF, HTML, DOCX, JSON)

Dans un système **RAG** industriel, les données ne proviennent jamais d'une source unique et homogène. Elles sont distribuées dans une grande variété de formats, chacun ayant ses propres structures, contraintes et niveaux de complexité. Le rôle du parsing multi-formats est de transformer ces sources hétérogènes en un format textuel unifié, exploitable par le reste du **pipeline** d'ingestion.

Cette étape est essentielle car la qualité du parsing conditionne directement la qualité des **embeddings** et, par extension, celle du **retrieval**. Une extraction incorrecte ou incomplète peut introduire du bruit, des pertes d'information ou des incohérences structurelles difficiles à corriger en aval.

PDF — Extraction complexe et structure implicite

Le format PDF est l'un des plus difficiles à traiter dans un contexte **RAG**. Contrairement à un format textuel structuré, un PDF est une représentation visuelle du contenu, sans garantie d'ordre logique explicite. Le texte peut être fragmenté, disposé en colonnes, ou mélangé avec des éléments graphiques.

Le parsing PDF nécessite donc des outils capables de reconstruire la logique de lecture humaine. Cela inclut la détection des blocs de texte, la gestion des colonnes, l'extraction des titres et la suppression des éléments non pertinents comme les en-têtes répétés ou les numéros de page.

Dans les systèmes industriels, l'objectif n'est pas seulement d'extraire du texte, mais de préserver la structure sémantique implicite du document afin de faciliter le **chunking** et la recherche ultérieure.

HTML — Exploitation de la structure hiérarchique

Les documents HTML présentent un avantage majeur : ils contiennent une structure explicite. Les balises telles que `<h1>`, `<h2>`, `<p>` ou `<section>` fournissent des indications précieuses sur la hiérarchie du contenu.

Le parsing HTML dans un **pipeline RAG** consiste donc à exploiter cette structure pour reconstruire un document proprement hiérarchisé. Cela permet de générer des chunks plus cohérents, alignés avec la logique du contenu original.

Cependant, les pages web contiennent souvent du bruit : menus, publicités, scripts ou éléments de navigation. Une étape de nettoyage est donc indispensable pour isoler le contenu principal.

DOCX — Documents bureautiques structurés

Les fichiers DOCX représentent des documents semi-structurés largement utilisés en entreprise. Ils contiennent du texte enrichi, des titres, des listes et parfois des tableaux.

Le parsing DOCX vise à préserver cette richesse structurelle tout en la convertissant en texte exploitable. Les titres peuvent être utilisés pour guider le **chunking**, tandis que les tableaux peuvent être transformés en représentations textuelles normalisées ou enrichies.

Dans un contexte **RAG**, le DOCX est souvent plus simple à traiter que le PDF, car sa structure est explicitement encodée.

JSON — Données structurées natives

Le format JSON occupe une place particulière, car il représente des données déjà structurées. Contrairement aux formats précédents, il ne nécessite pas une extraction de texte, mais une normalisation de structure.

Le parsing JSON consiste à identifier les champs pertinents, à aplatir les structures imbriquées si nécessaire et à transformer les données en unités textuelles cohérentes pour l'**embedding**.

Dans les systèmes **RAG** industriels, le JSON est souvent utilisé pour enrichir les documents avec des métadonnées ou pour intégrer directement des bases de connaissances structurées.

Unification des formats

L'objectif final du parsing multi-formats n'est pas de traiter chaque type de fichier indépendamment, mais de converger vers une représentation unifiée. Tous les documents, qu'ils proviennent de PDF, HTML, DOCX ou JSON, doivent être transformés en une structure commune composée de texte, de segments et de métadonnées.

Cette homogénéisation est essentielle pour garantir la cohérence du **pipeline** d'ingestion et permettre une vectorisation fiable et reproductible.

Ainsi, le parsing multi-formats constitue une étape fondamentale de l'architecture **RAG**, car il transforme la diversité des sources de données en un langage commun exploitable par les modèles d'**embeddings** et les systèmes de **retrieval**.

Section 4.3 — Normalisation et nettoyage

La normalisation et le nettoyage des données constituent une étape critique dans tout **pipeline** d'ingestion **RAG** industriel. Une fois les documents extraits et convertis depuis leurs formats d'origine, le texte obtenu est rarement exploitable tel quel. Il contient généralement du bruit, des incohérences et des variations structurelles qui peuvent dégrader significativement la qualité des **embeddings** et, par conséquent, celle du **retrieval**.

L'objectif de cette étape est de transformer un texte brut en une représentation stable, homogène et exploitable par les modèles de langage et d'**embedding**.

Suppression du bruit et des artefacts

Les documents issus de sources réelles contiennent fréquemment des éléments non pertinents : en-têtes répétés, pieds de page, numéros de pages, caractères spéciaux, artefacts d'encodage ou fragments issus de la conversion PDF. Ces éléments n'ont aucune valeur sémantique et peuvent perturber les modèles d'**embeddings** en introduisant des signaux parasites.

La première phase du nettoyage consiste donc à éliminer ces artefacts afin de conserver uniquement le contenu utile. Cette opération est particulièrement importante dans les systèmes **RAG**, car les **embeddings** sont sensibles à la distribution globale du texte.

Normalisation de l'encodage et du format texte

Les sources de données peuvent utiliser des encodages différents (UTF-8, ISO-8859-1, etc.) ou contenir des caractères mal interprétés. La normalisation de l'encodage permet de garantir une représentation cohérente des caractères, notamment pour les langues non latines ou les contenus multilingues.

Par ailleurs, la mise en forme du texte est uniformisée : suppression des espaces multiples, harmonisation des sauts de ligne, correction des caractères invisibles et standardisation de la ponctuation. Cette homogénéisation améliore la stabilité des **embeddings** et facilite les étapes de **chunking**.

Standardisation linguistique

Dans certains cas, le texte peut contenir des variations linguistiques ou typographiques importantes. La normalisation peut inclure des opérations telles que la mise en minuscule, la normalisation des accents ou la standardisation des abréviations.

Cependant, dans les systèmes **RAG** modernes, cette étape doit être utilisée avec prudence. Une normalisation excessive peut entraîner une perte d'information sémantique, notamment dans les cas où la casse ou la typographie ont une importance contextuelle.

Détection et suppression des contenus non pertinents

Une autre dimension du nettoyage consiste à identifier les segments de texte qui n'apportent aucune valeur pour le **retrieval**. Il peut s'agir de sections de navigation web, de mentions légales répétitives, de publicités ou de blocs techniques.

Dans les systèmes industriels, cette détection peut être réalisée via des heuristiques simples ou des modèles de classification plus avancés. L'objectif est de réduire le bruit global du corpus afin d'améliorer la précision des **embeddings**.

Préservation de la structure utile

Le nettoyage ne doit pas être confondu avec une simplification excessive. Certaines informations structurelles sont essentielles pour la qualité du **RAG**, notamment les titres, les sections et les séparations logiques entre les paragraphes.

Un système bien conçu doit donc conserver la structure sémantique du document tout en supprimant les éléments parasites. Cette préservation est essentielle pour guider efficacement le **chunking** et garantir la cohérence des unités d'information générées.

Impact sur le système RAG

La qualité de la normalisation a un impact direct sur l'ensemble du **pipeline RAG**. Un corpus mal nettoyé entraîne des **embeddings** bruités, des erreurs de **retrieval** et une baisse globale de la pertinence des réponses générées. À l'inverse, une normalisation rigoureuse améliore la stabilité du système et réduit la nécessité de corrections en aval comme le **reranking** ou le filtrage.

En production, cette étape est souvent sous-estimée, alors qu'elle constitue l'un des leviers les plus importants pour améliorer la qualité globale du système sans modifier les modèles sous-jacents.

Ainsi, la normalisation et le nettoyage représentent une étape fondamentale d'ingénierie de la donnée dans les architectures **RAG**, garantissant que les informations envoyées au modèle sont cohérentes, pertinentes et exploitables.

Section 4.4 — Versioning des documents

Dans un système **RAG** en production, les documents ne sont jamais statiques. Ils évoluent : mises à jour de procédures, corrections de contenu, nouvelles versions de rapports, suppression d'informations obsolètes. Sans mécanisme de versioning, un système RAG peut rapidement devenir incohérent, en mélangeant des informations anciennes et récentes au sein d'un même espace de recherche.

Le versioning des documents consiste à suivre et gérer l'évolution des contenus dans le temps, afin de garantir la cohérence entre les données indexées, les **embeddings** générés et les réponses produites par le modèle de langage.

Pourquoi le versioning est indispensable

Dans une architecture **RAG**, les **embeddings** sont directement dérivés du contenu textuel. Toute modification d'un document implique donc une modification potentielle de sa représentation vectorielle. Sans versioning, plusieurs problèmes apparaissent :

- coexistence de plusieurs versions contradictoires d'un même document
- réponses basées sur des informations obsolètes
- impossibilité de reproduire une réponse générée à un instant donné
- difficultés de débogage et d'audit du système

En production, ces problèmes sont critiques, notamment dans les domaines réglementés ou à forte exigence de traçabilité.

Identifiants et gestion des versions

La base du versioning repose sur l'introduction d'un identifiant unique de document, indépendant de son contenu. À cet identifiant sont associées différentes versions, chacune représentant un état du document à un instant donné.

Chaque version contient généralement :

- un identifiant de version
- un horodatage
- un hash du contenu

- les métadonnées associées
- les **embeddings** correspondants

Cette structuration permet de suivre précisément l'évolution du contenu et de relier chaque réponse du système à une version spécifique d'un document.

Stratégies de mise à jour des embeddings

Lorsqu'un document est modifié, il est nécessaire de décider comment mettre à jour la base vectorielle. Trois stratégies principales existent :

- La première est la réindexation complète, où toutes les versions précédentes sont supprimées et remplacées par la nouvelle. Cette approche est simple mais coûteuse en ressources.

La deuxième est la gestion incrémentale, où seules les parties modifiées du document sont recalculées et réindexées. Cette méthode est plus efficace mais nécessite une segmentation fine et une bonne traçabilité des chunks.

La troisième est la coexistence des versions, où plusieurs versions d'un même document sont conservées simultanément. Le système de **retrieval** doit alors être capable de filtrer la version la plus pertinente en fonction du contexte ou de règles métier.

Cohérence entre index et données

Un défi majeur du versioning est de maintenir la cohérence entre les documents stockés et les index vectoriels. Une divergence entre les deux peut entraîner des incohérences dans les résultats de recherche.

Pour éviter cela, les systèmes industriels utilisent souvent des mécanismes de synchronisation, des transactions logiques ou des pipelines d'ingestion atomiques garantissant que chaque mise à jour est entièrement appliquée ou ignorée.

Impact sur le retrieval et la qualité des réponses

Le versioning influence directement la qualité des réponses générées par le système **RAG**. Un système sans versioning peut produire des réponses contradictoires ou obsolètes, ce qui réduit fortement sa fiabilité en production.

À l'inverse, un système bien versionné permet :

- de garantir la fraîcheur des informations
- d'assurer la reproductibilité des réponses
- d'améliorer la traçabilité des sources
- de faciliter les audits et le debugging

Vers un RAG orienté data lifecycle

Le versioning des documents introduit une vision plus large du système **RAG** : il ne s'agit plus uniquement d'un **pipeline** de recherche et de génération, mais d'un système complet de gestion du cycle de vie de la connaissance.

Dans cette perspective, les documents ne sont pas des objets statiques, mais des entités évolutives dont chaque état doit être maîtrisé, indexé et exploité de manière cohérente.

Ainsi, le versioning devient une brique fondamentale de l'industrialisation du **RAG**, garantissant la fiabilité, la stabilité et la traçabilité du système dans le temps.

Chapitre 5 — Chunking avancé et stratégie de découpage

Le **chunking**, ou découpage des documents en segments exploitables, est une étape structurante dans tout système **RAG**. Il détermine directement la granularité de l'information qui sera indexée dans la base vectorielle et, par conséquent, la qualité du **retrieval** et des réponses générées.

Un mauvais choix de stratégie de **chunking** peut dégrader un système pourtant performant sur tous les autres aspects : **embeddings** de qualité, base vectorielle optimisée et **LLM** puissant. À l'inverse, un chunking bien conçu améliore significativement la précision, la cohérence et la pertinence des résultats.

Section 5.1 — Chunking naïf vs chunking sémantique

Dans les architectures **RAG**, il existe deux grandes approches de découpage des documents : le **chunking** naïf et le chunking sémantique. Ces deux méthodes diffèrent fondamentalement dans leur manière de représenter la structure de l'information.

Chunking naïf — découpage mécanique

Le **chunking** naïf consiste à découper un texte selon des règles fixes et arbitraires, généralement basées sur un nombre de caractères, de tokens ou de phrases. Par exemple, un document peut être divisé en segments de 500 ou 1000 tokens avec un chevauchement (overlap) constant entre les chunks.

Cette approche présente l'avantage d'être simple à implémenter, rapide à exécuter et facile à industrialiser. Elle ne nécessite aucune compréhension du contenu et peut être appliquée uniformément à tous les types de documents.

Cependant, ses limites apparaissent rapidement dans un contexte **RAG**. Le découpage peut interrompre des idées en plein milieu, séparer des concepts liés ou mélanger des informations sans cohérence sémantique. Cela conduit à des **embeddings** de faible qualité, car chaque chunk peut

contenir une information incomplète ou mal structurée.

Chunking sémantique — découpage basé sur le sens

Le **chunking** sémantique adopte une approche plus intelligente. Au lieu de découper le texte de manière mécanique, il cherche à respecter la structure logique et conceptuelle du document.

Cette méthode exploite des indices tels que :

- les titres et sous-titres
- les paragraphes cohérents
- les transitions thématiques
- la similarité sémantique entre phrases

L'objectif est de créer des chunks qui correspondent à des unités de sens complètes, capables de représenter une idée cohérente et autonome.

Dans un système **RAG**, cette approche améliore significativement la qualité du **retrieval**, car chaque **embedding** représente une information plus stable et plus contextuelle. Les requêtes utilisateur ont alors plus de chances de correspondre directement à des segments pertinents.

Comparaison des deux approches

Le **chunking** naïf est souvent utilisé dans les prototypes ou les systèmes simples, car il est rapide à mettre en œuvre. Cependant, il introduit un bruit structurel important dans les représentations vectorielles.

Le **chunking** sémantique, bien que plus complexe à implémenter, offre une meilleure qualité globale du système. Il réduit les pertes d'information, améliore la cohérence des **embeddings** et augmente la précision du **retrieval**.

Impact sur le système RAG

Le choix entre **chunking** naïf et chunking sémantique a un impact direct sur plusieurs aspects du système :

- la qualité des **embeddings** générés
- la pertinence des résultats de recherche
- la stabilité des réponses du **LLM**
- le besoin en **reranking** ou en post-traitement

En pratique, un mauvais **chunking** peut être plus pénalisant qu'un modèle d'**embedding** sous-optimal, car il introduit une perte d'information irréversible dès l'étape d'ingestion.

Vers des stratégies hybrides

Dans les systèmes **RAG** industriels modernes, il est fréquent d'adopter des approches hybrides combinant les deux méthodes. Un découpage initial simple peut être enrichi par des heuristiques sémantiques ou des modèles de segmentation avancés. L'objectif est de trouver un compromis entre complexité computationnelle et qualité de représentation.

Ainsi, le **chunking** n'est pas une étape secondaire du **pipeline RAG**, mais un élément fondamental qui conditionne la structure même de la connaissance exploitable par le système.

Section 5.2 — Overlap et perte d'information

Dans les systèmes **RAG**, le découpage des documents en chunks introduit inévitablement une contrainte structurelle : aucune segmentation n'est parfaite. Lorsqu'un texte est fragmenté, il existe toujours un risque de couper une information au mauvais endroit, ce qui peut entraîner une perte de contexte ou une dégradation de la compréhension sémantique. Le mécanisme d'overlap (chevauchement) est précisément conçu pour atténuer ce problème.

Le problème de la coupure d'information

Lorsqu'un document est découpé en segments indépendants, certaines informations peuvent être partiellement contenues dans un chunk et complétées dans le suivant. Sans stratégie adaptée, un modèle d'**embedding** peut alors encoder une information incomplète, ce qui réduit la pertinence du **retrieval**.

Par exemple, une définition, une condition logique ou une explication technique peut être séparée entre deux chunks consécutifs. Individuellement, ces chunks perdent leur sens global, ce qui dégrade la qualité des représentations vectorielles.

Principe de l'overlap

L'overlap consiste à introduire une zone de recouvrement entre deux chunks consécutifs. Concrètement, une partie des derniers tokens d'un chunk est réutilisée au début du chunk suivant. Cette redondance contrôlée permet de préserver la continuité sémantique du texte.

L'objectif n'est pas de dupliquer inutilement les données, mais de garantir que les informations situées aux frontières des segments restent accessibles dans au moins un **embedding** complet.

Effets sur la qualité des embeddings

L'overlap améliore la robustesse des représentations vectorielles en réduisant les pertes d'information liées au découpage. Les modèles d'**embeddings** sont ainsi exposés à des contextes plus complets, ce qui leur permet de mieux capturer les relations entre les concepts.

Cependant, un overlap mal calibré peut introduire de la redondance excessive dans la base vectorielle. Cela peut entraîner :

- une augmentation inutile du volume de données indexées
- une duplication des résultats lors du **retrieval**
- une légère dégradation de la diversité des réponses

Compromis entre précision et coût

Le choix du niveau d'overlap repose sur un compromis entre qualité et efficacité. Un overlap élevé améliore la continuité sémantique mais augmente le coût de stockage et de calcul. À l'inverse, un overlap faible réduit la redondance mais augmente le risque de perte d'information.

Dans un système **RAG** industriel, ce paramètre est généralement ajusté en fonction :

- de la taille des chunks
- du type de documents traités
- du niveau de précision attendu
- des contraintes de performance et de coût

Interaction avec le retrieval

L'overlap influence également le comportement du système de **retrieval**. Lorsqu'une requête correspond à une information située à la frontière entre deux chunks, l'overlap augmente la probabilité qu'au moins un des deux chunks contienne suffisamment de contexte pour être correctement identifié comme pertinent.

Cela améliore le recall du système, c'est-à-dire sa capacité à retrouver toutes les informations pertinentes, au prix potentiel d'une légère augmentation du bruit dans les résultats initiaux.

Vers une stratégie de chunking robuste

Dans les architectures **RAG** avancées, l'overlap n'est pas utilisé de manière isolée mais combiné à des stratégies de **chunking** sémantique. L'objectif est de réduire au maximum la perte d'information tout en maintenant une structure cohérente et exploitable.

Ainsi, l'overlap doit être considéré non pas comme une simple technique d'ajustement, mais comme un mécanisme fondamental de préservation du contexte dans les pipelines d'ingestion.

Section 5.3 — Chunking hiérarchique

Le **chunking** hiérarchique est une approche avancée de découpage des documents qui vise à préserver la structure naturelle de l'information tout en offrant plusieurs niveaux de granularité pour

le **retrieval**. Contrairement aux méthodes simples basées sur une taille fixe de segments, cette approche considère que les documents possèdent une organisation intrinsèque qu'il est essentiel de conserver dans un système **RAG** industriel.

L'idée fondamentale est de représenter un document non pas comme une suite linéaire de chunks indépendants, mais comme une arborescence de segments imbriqués, allant du plus global au plus local. Cette structure permet d'adapter dynamiquement le niveau de détail utilisé lors de la recherche en fonction de la requête utilisateur.

Principe de structuration multi-niveaux

Dans un **chunking** hiérarchique, un document est généralement décomposé selon plusieurs niveaux :

- Niveau global : document complet

Niveau intermédiaire : sections, chapitres, sous-sections

Niveau fin : paragraphes ou chunks sémantiques

Niveau très fin : phrases ou segments courts

Chaque niveau représente une abstraction différente du même contenu. Cette organisation permet au système **RAG** de naviguer dans le document comme dans une structure logique, plutôt que comme dans un espace plat de chunks indépendants.

Avantages dans un système RAG

Le principal avantage du **chunking** hiérarchique est sa capacité à améliorer la précision du **retrieval** tout en conservant du contexte global. Lorsqu'une requête est formulée, le système peut d'abord identifier la section pertinente du document avant de descendre vers des segments plus précis.

Cette approche réduit considérablement le bruit dans les résultats de recherche, car elle évite de comparer une requête directement à des fragments isolés sans contexte global. Elle améliore également la cohérence des réponses générées, car le **LLM** peut accéder à un contexte structuré plutôt qu'à une simple liste de passages indépendants.

Navigation dans l'arborescence documentaire

Le **chunking** hiérarchique transforme le processus de **retrieval** en une forme de navigation. Au lieu de rechercher directement les chunks les plus proches dans l'espace vectoriel, le système peut effectuer une recherche progressive :

- Identification des documents les plus pertinents

Sélection des sections correspondantes

Raffinement vers les sous-sections

Extraction des chunks les plus informatifs

Cette approche permet de réduire l'espace de recherche à chaque étape, ce qui améliore à la fois la performance et la pertinence des résultats.

Impact sur les embeddings

Dans une architecture hiérarchique, les **embeddings** peuvent être générés à différents niveaux de granularité. Les représentations globales d'un document capturent son thème général, tandis que les embeddings de chunks fins capturent des informations spécifiques.

Cette multi-représentation permet d'enrichir le système de **retrieval** en combinant des signaux globaux et locaux. Elle améliore notamment la capacité du système à comprendre les requêtes complexes nécessitant à la fois du contexte large et des détails précis.

Complexité d'implémentation

Le **chunking** hiérarchique est plus complexe à mettre en œuvre que les approches classiques. Il nécessite une analyse structurelle des documents, une gestion des relations entre les niveaux et une stratégie de stockage adaptée dans la base vectorielle.

Cependant, cette complexité est souvent justifiée dans les systèmes **RAG** industriels où la qualité du **retrieval** est critique et où les documents possèdent une structure riche et stable.

Vers une recherche structurée

Le **chunking** hiérarchique marque une transition importante dans la conception des systèmes **RAG** : on passe d'une logique de recherche plate à une logique de recherche structurée sur un espace documentaire organisé.

Cette approche rapproche les systèmes **RAG** des moteurs de recherche avancés et des systèmes de gestion de connaissance, où la structure du document devient un élément central de la performance globale.

Section 5.4 — Optimisation selon cas d'usage

Le **chunking** hiérarchique est une approche avancée de découpage des documents qui vise à préserver la structure naturelle de l'information tout en offrant plusieurs niveaux de granularité pour le **retrieval**. Contrairement aux méthodes simples basées sur une taille fixe de segments, cette approche considère que les documents possèdent une organisation intrinsèque qu'il est essentiel

de conserver dans un système **RAG** industriel.

L'idée fondamentale est de représenter un document non pas comme une suite linéaire de chunks indépendants, mais comme une arborescence de segments imbriqués, allant du plus global au plus local. Cette structure permet d'adapter dynamiquement le niveau de détail utilisé lors de la recherche en fonction de la requête utilisateur.

Principe de structuration multi-niveaux

Dans un **chunking** hiérarchique, un document est généralement décomposé selon plusieurs niveaux :

- Niveau global : document complet

Niveau intermédiaire : sections, chapitres, sous-sections

Niveau fin : paragraphes ou chunks sémantiques

Niveau très fin : phrases ou segments courts

Chaque niveau représente une abstraction différente du même contenu. Cette organisation permet au système **RAG** de naviguer dans le document comme dans une structure logique, plutôt que comme dans un espace plat de chunks indépendants.

Avantages dans un système RAG

Le principal avantage du **chunking** hiérarchique est sa capacité à améliorer la précision du **retrieval** tout en conservant du contexte global. Lorsqu'une requête est formulée, le système peut d'abord identifier la section pertinente du document avant de descendre vers des segments plus précis.

Cette approche réduit considérablement le bruit dans les résultats de recherche, car elle évite de comparer une requête directement à des fragments isolés sans contexte global. Elle améliore également la cohérence des réponses générées, car le **LLM** peut accéder à un contexte structuré plutôt qu'à une simple liste de passages indépendants.

Navigation dans l'arborescence documentaire

Le **chunking** hiérarchique transforme le processus de **retrieval** en une forme de navigation. Au lieu de rechercher directement les chunks les plus proches dans l'espace vectoriel, le système peut effectuer une recherche progressive :

- Identification des documents les plus pertinents

Sélection des sections correspondantes

Raffinement vers les sous-sections

Extraction des chunks les plus informatifs

Cette approche permet de réduire l'espace de recherche à chaque étape, ce qui améliore à la fois la performance et la pertinence des résultats.

Impact sur les embeddings

Dans une architecture hiérarchique, les **embeddings** peuvent être générés à différents niveaux de granularité. Les représentations globales d'un document capturent son thème général, tandis que les embeddings de chunks fins capturent des informations spécifiques.

Cette multi-représentation permet d'enrichir le système de **retrieval** en combinant des signaux globaux et locaux. Elle améliore notamment la capacité du système à comprendre les requêtes complexes nécessitant à la fois du contexte large et des détails précis.

Complexité d'implémentation

Le **chunking** hiérarchique est plus complexe à mettre en œuvre que les approches classiques. Il nécessite une analyse structurelle des documents, une gestion des relations entre les niveaux et une stratégie de stockage adaptée dans la base vectorielle.

Cependant, cette complexité est souvent justifiée dans les systèmes **RAG** industriels où la qualité du **retrieval** est critique et où les documents possèdent une structure riche et stable.

Vers une recherche structurée

Le **chunking** hiérarchique marque une transition importante dans la conception des systèmes **RAG** : on passe d'une logique de recherche plate à une logique de recherche structurée sur un espace documentaire organisé.

Cette approche rapproche les systèmes **RAG** des moteurs de recherche avancés et des systèmes de gestion de connaissance, où la structure du document devient un élément central de la performance globale.

Partie II — Retrieval haute performance

Le **retrieval** constitue le cœur opérationnel d'un système **RAG**. C'est à ce niveau que les informations stockées dans la base vectorielle sont transformées en contexte exploitable par le modèle de langage. La qualité de cette étape détermine directement la pertinence des réponses générées. Dans un environnement de production, le retrieval ne peut pas se limiter à une simple recherche de similarité : il doit être rapide, scalable et robuste face à des volumes massifs de

données.

Chapitre 6 — Recherche vectorielle à grande échelle

La recherche vectorielle à grande échelle consiste à retrouver, dans un espace de haute dimension, les vecteurs les plus proches d'une requête parmi des millions ou des milliards d'éléments. Cette opération, simple en apparence, devient complexe dès lors que les contraintes de latence et de **scalabilité** entrent en jeu.

Dans les systèmes **RAG** industriels, cette recherche repose quasi exclusivement sur des méthodes approximatives, regroupées sous le terme **ANN** (Approximate Nearest Neighbors).

Section 6.1 — ANN search en production

L'**ANN** search (Approximate Nearest Neighbors) est une approche qui permet de résoudre le problème de recherche de voisins proches dans un espace vectoriel de haute dimension en réduisant drastiquement le coût computationnel. Contrairement à une recherche exhaustive, qui compare une requête à tous les vecteurs de la base, l'ANN cherche à identifier rapidement un ensemble restreint de candidats probables.

L'objectif fondamental de l'**ANN** en production n'est pas d'obtenir une exactitude parfaite, mais d'optimiser un compromis entre précision, latence et **scalabilité**. Dans un système **RAG** industriel, ce compromis est essentiel, car une recherche exacte devient rapidement impraticable dès que le volume de données dépasse quelques centaines de milliers de vecteurs.

Principe de l'approximation

L'idée centrale de l'**ANN** est de réduire l'espace de recherche en exploitant des structures de données intelligentes telles que des graphes, des clusters ou des quantifications. Au lieu de parcourir l'intégralité de la base vectorielle, le système explore uniquement une fraction de cet espace, en se concentrant sur les zones les plus prometteuses.

Cette approximation introduit un léger risque de perte de certains voisins optimaux, mais elle permet d'obtenir des performances compatibles avec des usages en temps réel.

Contraintes de production

En environnement industriel, l'**ANN** search doit répondre à plusieurs contraintes simultanées :

- une latence faible, souvent inférieure à quelques dizaines de millisecondes

une capacité à gérer des millions ou milliards de vecteurs

une stabilité des performances malgré l'évolution des données

une intégration fluide avec les pipelines de retrieval et de reranking

Ces contraintes imposent l'utilisation de structures spécialisées comme **HNSW**, **IVF** ou leurs variantes hybrides.

Trade-off précision / performance

L'un des aspects fondamentaux de l'**ANN** est le compromis entre précision et performance. Plus l'algorithme explore en profondeur l'espace vectoriel, plus les résultats sont précis, mais plus la latence augmente. À l'inverse, une recherche plus approximative améliore la vitesse mais peut dégrader la qualité du **retrieval**.

En production, ce paramètre est souvent ajusté dynamiquement en fonction des contraintes du système, comme la charge serveur ou la complexité de la requête.

Rôle dans le pipeline RAG

Dans une architecture **RAG**, l'**ANN** search constitue la première étape du **retrieval**. Il permet de réduire un espace de recherche massif à un ensemble restreint de candidats pertinents, qui seront ensuite éventuellement raffinés par des étapes supplémentaires comme le **reranking**.

Ainsi, l'**ANN** ne fournit pas une réponse finale, mais un filtrage initial indispensable pour rendre le système scalable.

Importance en production

Sans **ANN** search, un système **RAG** ne peut pas fonctionner à grande échelle. La recherche exhaustive devient rapidement prohibitive en termes de coût et de latence. L'**ANN** est donc une brique fondamentale de toute architecture de **retrieval** moderne, permettant de transformer un problème théoriquement coûteux en une opération exploitable en temps réel.

En ce sens, l'**ANN** search représente l'un des piliers techniques qui rendent possible l'industrialisation des systèmes **RAG**.

Section 6.2 — Optimisation latence / recall

Dans un système **RAG** en production, la recherche vectorielle ne se résume pas à “retrouver les bons documents”. Elle consiste à optimiser un équilibre fondamental entre deux objectifs souvent antagonistes : la latence et le recall. Cet arbitrage est au cœur de toutes les décisions d’ingénierie liées au **retrieval**.

La latence correspond au temps nécessaire pour exécuter une requête de recherche. Le recall, quant à lui, mesure la capacité du système à retrouver l’ensemble des éléments pertinents présents dans la base de données. Un système rapide mais incomplet produit des réponses pauvres, tandis qu’un système exhaustif mais lent devient inutilisable en contexte interactif.

Nature du compromis latence / recall

Dans les systèmes **ANN**, améliorer le recall implique généralement d’explorer davantage l’espace vectoriel. Cela signifie augmenter le nombre de candidats examinés ou approfondir la recherche dans les structures d’index. Cependant, chaque opération supplémentaire augmente mécaniquement la latence.

Inversement, réduire la latence revient à limiter l’exploration de l’espace de recherche, ce qui augmente le risque de manquer des documents pertinents. Ce compromis est structurel et ne peut pas être éliminé, seulement optimisé en fonction du cas d’usage.

Paramètres d’optimisation

Les systèmes de recherche vectorielle offrent plusieurs leviers permettant de contrôler cet équilibre :

le nombre de voisins explorés (efSearch dans HNSW)

la taille des clusters inspectés (dans IVF)

le nombre de partitions interrogées

les seuils de filtrage préalable

le nombre de candidats transmis au reranker

Chacun de ces paramètres influence directement la relation entre vitesse et exhaustivité. En production, ils sont généralement calibrés empiriquement à partir de benchmarks spécifiques au domaine d’application.

Stratégies d'optimisation du recall

Pour améliorer le recall sans dégrader excessivement la latence, plusieurs stratégies sont utilisées. La première consiste à augmenter légèrement la taille de la recherche initiale (**Top-K** élargi), puis à affiner les résultats via un **reranking** plus précis. Cette approche permet de compenser les approximations de l'**ANN**.

Une autre stratégie repose sur la diversification des **embeddings** ou des index. En combinant plusieurs représentations vectorielles, le système augmente les chances de capturer différents aspects sémantiques d'une même requête.

Enfin, l'utilisation de pipelines hybrides combinant recherche dense et recherche lexicale permet d'améliorer significativement le recall global sans explosion de la latence.

Optimisation de la latence

La réduction de la latence repose principalement sur l'optimisation de l'exploration de l'espace vectoriel et de l'infrastructure sous-jacente. Cela inclut :

l'utilisation d'index optimisés en mémoire (comme HNSW)

la réduction de la dimension des embeddings

la mise en cache des résultats fréquents

la parallélisation des requêtes

la réduction du nombre de vecteurs candidats

Dans les systèmes industriels, la latence est souvent surveillée en continu, car elle constitue un indicateur critique de la performance globale du système.

Rôle du reranking dans l'équilibre global

Le **reranking** joue un rôle central dans l'optimisation du compromis latence / recall. En permettant de récupérer un ensemble large de candidats rapidement, puis de les réordonner avec un modèle plus précis mais plus coûteux, il permet de déléguer la précision à une étape secondaire.

Cette architecture en deux phases permet de maintenir une latence faible tout en conservant un haut niveau de recall et de précision.

Vision système

L'optimisation latence / recall ne doit pas être vue comme un réglage local, mais comme une propriété globale du système **RAG**. Chaque choix architectural — type d'index, taille des chunks, modèle d'**embedding**, stratégie de **reranking** — influence directement cet équilibre.

Ainsi, la conception d'un système **RAG** performant repose sur une optimisation continue de ce compromis, en fonction des contraintes métier et des exigences de qualité.

Section 6.3 — Top-K retrieval intelligent

Le **Top-K retrieval** constitue l'interface directe entre la recherche vectorielle et le modèle de génération. Une fois les candidats les plus proches identifiés par l'**ANN**, le système doit sélectionner un sous-ensemble de documents qui sera effectivement injecté dans le contexte du **LLM**. Cette étape, souvent sous-estimée, joue un rôle déterminant dans la qualité finale des réponses.

Dans un système naïf, le **Top-K** consiste simplement à prendre les K vecteurs les plus proches de la requête. Dans un système **RAG** industriel, cette approche est généralement insuffisante, car elle ne prend pas en compte la redondance, la diversité ni la pertinence contextuelle globale des documents.

Limites du Top-K naïf

Le **Top-K** brut basé uniquement sur la **similarité cosinus** présente plusieurs limites. Il peut sélectionner plusieurs chunks provenant du même document ou exprimant la même idée sous différentes formes. Cela réduit la diversité du contexte fourni au **LLM** et peut conduire à des réponses répétitives ou biaisées.

De plus, un **Top-K** strictement basé sur le score de similarité peut négliger des documents légèrement moins similaires mais plus informatifs dans le contexte global de la requête.

Vers un Top-K enrichi

Le **Top-K** intelligent introduit des mécanismes supplémentaires pour améliorer la qualité de la sélection. L'objectif n'est plus seulement de maximiser la similarité locale, mais de construire un ensemble de contexte équilibré, diversifié et informatif.

Cela inclut généralement plusieurs étapes de post-traitement après la recherche **ANN**.

Diversification des résultats

Une première stratégie consiste à introduire de la diversité dans les résultats sélectionnés. Au lieu de prendre les K premiers résultats bruts, le système applique des techniques de réordonnancement qui pénalisent les doublons sémantiques ou les chunks trop similaires entre eux.

Cette approche permet de couvrir un spectre plus large d'informations pertinentes, améliorant ainsi la richesse du contexte fourni au **LLM**.

Filtrage par métadonnées

Le **Top-K** intelligent peut également intégrer des filtres basés sur les métadonnées des documents. Cela permet de prioriser certains types de contenus, comme les documents récents, les sources officielles ou les sections spécifiques d'un corpus.

Ce filtrage améliore la pertinence contextuelle en alignant la sélection des chunks avec les contraintes métier du système.

Pondération hybride

Dans les systèmes avancés, le **Top-K** ne repose pas uniquement sur la similarité vectorielle. Il peut combiner plusieurs signaux, tels que :

- score de **similarité cosinus**
- score **BM25** lexical
- score de **reranking**
- importance des métadonnées

Ces signaux sont agrégés pour produire un score global qui reflète mieux la pertinence réelle des documents pour la requête utilisateur.

Interaction avec le reranking

Le **Top-K** intelligent est souvent étroitement couplé avec le **reranking**. Une stratégie courante consiste à sélectionner un Top-K élargi lors de la recherche initiale, puis à appliquer un modèle de reranking pour réduire cet ensemble à un sous-ensemble final plus précis.

Cette architecture en deux étapes permet de concilier recall élevé et qualité de contexte optimale pour le **LLM**.

Impact sur la génération

La qualité du **Top-K** a un impact direct sur la génération de la réponse. Un ensemble mal construit peut introduire du bruit, des contradictions ou des informations redondantes, ce qui dégrade la cohérence du texte généré.

À l'inverse, un **Top-K** bien optimisé améliore la fluidité, la précision et la stabilité des réponses du modèle, même sans modifier le **LLM** sous-jacent.

Conclusion

Le **Top-K retrieval** intelligent transforme une simple opération de sélection en un véritable processus de construction de contexte. Il agit comme un filtre de qualité entre la recherche vectorielle et le modèle de langage, garantissant que seules les informations les plus pertinentes, diversifiées et cohérentes sont transmises à l'étape de génération.

Ainsi, dans une architecture **RAG** industrielle, le **Top-K** n'est pas un paramètre secondaire, mais un composant stratégique du **pipeline de retrieval**.

Section 6.4 — Gestion des embeddings à grande échelle

La gestion des **embeddings** à grande échelle est une composante essentielle des systèmes **RAG** industriels. Lorsque le volume de données passe de quelques milliers à plusieurs millions, voire milliards de chunks, la simple génération et stockage des vecteurs ne suffit plus : il devient nécessaire d'optimiser l'ensemble du cycle de vie des embeddings, depuis leur création jusqu'à leur mise à jour et leur exploitation.

Dans ce contexte, les **embeddings** ne sont pas seulement des représentations statiques du texte, mais des objets dynamiques intégrés dans une infrastructure distribuée, soumise à des contraintes de coût, de performance et de cohérence.

Production et batch processing des embeddings

La génération d'**embeddings** à grande échelle est rarement effectuée en temps réel. Dans les systèmes industriels, elle est généralement traitée en batch asynchrone afin de réduire les coûts et d'optimiser l'utilisation des ressources GPU ou CPU.

Les documents sont regroupés en lots, puis vectorisés de manière parallèle. Cette approche permet de maximiser le throughput tout en minimisant la latence globale du **pipeline** d'ingestion. Elle nécessite toutefois une orchestration fine pour éviter les goulots d'étranglement lors de l'insertion dans la base vectorielle.

Stockage et compression des vecteurs

À grande échelle, le stockage des **embeddings** devient une contrainte majeure. Un vecteur de plusieurs centaines de dimensions, multiplié par des millions de chunks, représente un volume mémoire conséquent.

Pour répondre à cette contrainte, plusieurs techniques de compression sont utilisées :

- réduction de la précision numérique (float32 vers float16 ou int8)
- quantification vectorielle (Product Quantization)
- stockage distribué avec partitionnement horizontal

Ces techniques permettent de réduire significativement l'empreinte mémoire tout en conservant une qualité de **retrieval** acceptable.

Mise à jour et synchronisation des embeddings

Dans un système **RAG** dynamique, les **embeddings** doivent être mis à jour en permanence pour refléter les évolutions des documents. Cela pose un problème de cohérence entre les données sources, les vecteurs et les index **ANN**.

Deux approches principales sont utilisées :

- mise à jour incrémentale, où seuls les nouveaux ou modifiés chunks sont ré-embeddés
- reconstruction partielle ou complète de l'index en fonction de l'ampleur des changements

La gestion de ces mises à jour doit être soigneusement orchestrée pour éviter les incohérences temporaires dans les résultats de recherche.

Partitionnement et distribution

Lorsque le volume d'**embeddings** devient très important, il est nécessaire de distribuer les données sur plusieurs nœuds. Le partitionnement peut être effectué selon différents critères :

- partitionnement aléatoire pour équilibrer la charge
- partitionnement par domaine ou catégorie documentaire
- partitionnement temporel basé sur la fraîcheur des données

Chaque stratégie a un impact direct sur les performances de recherche et la complexité du système.

Cohérence entre embeddings et index

Un défi majeur à grande échelle est de maintenir la cohérence entre les **embeddings** stockés et les index **ANN** utilisés pour la recherche. Toute divergence peut entraîner des résultats incohérents ou obsolètes.

Les systèmes industriels utilisent souvent des mécanismes de versioning des index, de rebuild progressif ou de double écriture pour garantir une transition fluide lors des mises à jour massives.

Optimisation des coûts

La gestion des **embeddings** à grande échelle a un impact économique direct. Les coûts incluent :

- calcul des **embeddings** (CPU/GPU)
- stockage des vecteurs
- maintenance des index
- requêtes en production

Pour optimiser ces coûts, les systèmes **RAG** utilisent des stratégies comme le caching des **embeddings** fréquents, la déduplication des documents ou la réduction de la dimension vectorielle.

Vision industrielle

À grande échelle, les **embeddings** deviennent une infrastructure critique comparable à une base de données transactionnelle. Leur gestion nécessite des outils de monitoring, de versioning et d'optimisation continue.

Ainsi, la maîtrise des **embeddings** à grande échelle ne relève plus uniquement du machine learning, mais d'une véritable ingénierie système, où performance, **scalabilité** et coûts doivent être équilibrés en permanence.

Chapitre 7 — Recherche hybride (vector + lexical)

Les systèmes **RAG** modernes ne reposent presque jamais sur une seule forme de recherche. Même si la recherche vectorielle permet une excellente compréhension sémantique, elle n'est pas suffisante pour couvrir tous les cas d'usage industriels. C'est pourquoi les architectures performantes combinent généralement recherche vectorielle et recherche lexicale, afin de bénéficier des forces des deux approches.

Cette combinaison est appelée recherche hybride. Elle permet de concilier compréhension sémantique, précision terminologique et robustesse face à des requêtes variées.

Section 7.1 — BM25 et recherche lexicale

La recherche lexicale repose sur une logique fondamentalement différente de la recherche vectorielle. Au lieu de représenter les textes dans un espace sémantique, elle se base sur la correspondance exacte ou partielle des mots entre la requête et les documents. L'un des

algorithmes les plus utilisés dans ce domaine est **BM25**.

Principe de BM25

BM25 est une fonction de ranking probabiliste utilisée pour mesurer la pertinence d'un document par rapport à une requête. Elle repose sur la fréquence des termes et leur importance relative dans le corpus.

L'idée centrale est simple : un document est considéré comme pertinent s'il contient les termes de la requête, mais cette pertinence est pondérée par la rareté et la fréquence des mots.

Mathématiquement, **BM25** s'appuie sur une combinaison de facteurs liés à la fréquence des termes dans le document et à leur distribution dans l'ensemble du corpus.

$$\text{score}(D,Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t,D)}{(k_1+1)f(t,D) + k_1} \cdot (1-b + b \cdot \frac{|D|}{\text{avgDL}})$$

- où :
- $f(t,D)$ est la fréquence du terme dans le document

$|D|$ est la longueur du document

avgDL est la longueur moyenne des documents du corpus

k_1 et b sont des paramètres de calibration

Logique de recherche lexicale

Contrairement aux **embeddings**, la recherche lexicale ne comprend pas le sens des mots, mais leur présence explicite. Cela signifie qu'elle est particulièrement efficace pour :

- les noms propres
- les identifiants techniques
- les codes, erreurs, références
- les requêtes exactes

Elle est en revanche moins performante sur les paraphrases ou les reformulations.

Avantages dans un système RAG

BM25 apporte une complémentarité essentielle à la recherche vectorielle. Là où les **embeddings** capturent la sémantique globale, BM25 garantit une précision terminologique.

Cette complémentarité est particulièrement importante dans les environnements industriels où les utilisateurs peuvent formuler des requêtes très techniques ou très précises.

Limites de la recherche lexicale

BM25 présente plusieurs limites structurelles :

- incapacité à comprendre les synonymes
- sensibilité à la formulation exacte
- faible robustesse aux reformulations
- dépendance forte à la qualité du prétraitement du texte

Ces limites justifient son utilisation en combinaison avec la recherche vectorielle plutôt qu'en remplacement.

Rôle dans la recherche hybride

Dans une architecture **RAG** moderne, **BM25** est généralement utilisé en parallèle de la recherche vectorielle. Les deux résultats sont ensuite fusionnés ou pondérés pour produire un ensemble final de candidats.

Cette approche permet d'améliorer à la fois le recall et la précision, en exploitant les forces complémentaires des deux méthodes.

Conclusion

BM25 représente la base historique de la recherche d'information textuelle. Bien qu'il soit limité par rapport aux approches sémantiques modernes, il reste un composant essentiel des systèmes **RAG** industriels. Son intégration dans des architectures hybrides permet de garantir une meilleure robustesse globale du **retrieval**.

Section 7.2 — Fusion dense + sparse

Section 7.2 — Fusion dense + sparse

La fusion dense + sparse est au cœur des systèmes de recherche hybride modernes. Elle consiste à combiner deux familles de signaux de pertinence : d'un côté la recherche dense basée sur les **embeddings** vectoriels, et de l'autre la recherche sparse basée sur des méthodes lexicales comme **BM25**. L'objectif est de tirer parti de la complémentarité de ces deux approches pour améliorer la robustesse et la qualité du **retrieval**.

Dans une architecture **RAG** industrielle, aucune des deux approches n'est suffisante seule. La recherche dense excelle dans la compréhension sémantique et les reformulations, tandis que la

recherche sparse est supérieure pour les correspondances exactes, les termes techniques et les identifiants précis. La fusion permet donc de couvrir un spectre plus large de requêtes utilisateur.

Deux représentations complémentaires

La recherche dense représente les documents et les requêtes dans un espace vectoriel continu où la similarité est mesurée géométriquement. Elle capture les relations sémantiques profondes, mais peut parfois manquer de précision sur les termes exacts.

La recherche sparse, au contraire, repose sur une représentation discrète basée sur les mots présents dans les documents. Elle est très précise lexicalement mais incapable de comprendre les variations linguistiques.

La fusion consiste à exploiter simultanément ces deux représentations pour construire un score global de pertinence.

Stratégies de fusion

Il existe plusieurs méthodes pour combiner les signaux dense et sparse.

La première approche est la fusion pondérée, où les scores des deux systèmes sont normalisés puis combinés selon un poids fixe ou dynamique. Cette méthode est simple à mettre en œuvre mais nécessite un réglage précis des coefficients pour éviter de survaloriser une source au détriment de l'autre.

Une deuxième approche est la fusion par rang (rank fusion), où les résultats des deux systèmes sont classés séparément puis combinés en fonction de leur position dans chaque liste. Cette méthode est plus robuste aux différences d'échelle entre les scores.

Une troisième approche consiste à utiliser des modèles appris pour estimer directement la pertinence finale à partir des signaux dense et sparse. Ces modèles de fusion apprennent à pondérer dynamiquement les contributions en fonction du type de requête.

Normalisation des scores

Un défi important dans la fusion dense + sparse est l'hétérogénéité des scores. Les similarités vectorielles et les scores **BM25** n'ont pas la même distribution ni la même plage de valeurs. Il est donc nécessaire de les normaliser avant combinaison.

Cette normalisation peut être effectuée par des méthodes simples comme le min-max scaling, ou par des techniques plus avancées basées sur des distributions statistiques ou des calibrations empiriques.

Impact sur le retrieval

La fusion améliore significativement la qualité du **retrieval** en augmentant le recall global du système. Des documents qui auraient été manqués par la recherche vectorielle seule peuvent être récupérés grâce au signal lexical, et inversement.

Elle permet également de réduire les erreurs de contexte dans les systèmes **RAG**, en fournissant au modèle de langage un ensemble de documents plus complet et plus diversifié.

Coût et complexité

L'intégration de la fusion dense + sparse augmente légèrement la complexité du système, car elle nécessite deux pipelines de recherche parallèles. Cependant, ce coût est largement compensé par le gain en qualité et en robustesse.

Dans les systèmes industriels, cette approche est devenue un standard, notamment pour les applications où la précision du **retrieval** est critique.

Conclusion

La fusion dense + sparse représente une étape clé dans l'évolution des systèmes **RAG** vers des architectures hybrides robustes. En combinant la compréhension sémantique des **embeddings** et la précision lexicale des méthodes classiques, elle permet de construire des systèmes de recherche plus complets, plus fiables et mieux adaptés aux contraintes réelles de production.

Section 7.3 — Hybrid retrieval architecture

Section 7.3 — Hybrid retrieval architecture

L'architecture de **retrieval** hybride représente l'aboutissement pratique de la recherche combinant signaux denses et signaux sparsés. Au lieu de considérer la fusion dense + sparse comme une simple étape algorithmique, les systèmes **RAG** industriels l'implémentent comme une architecture complète de **pipeline**, intégrée au cœur du système de recherche.

Cette architecture vise un objectif clair : garantir une qualité de **retrieval** élevée, stable et scalable, même en présence de requêtes hétérogènes, de corpus volumineux et de contraintes de latence fortes.

Structure générale du pipeline hybride

Un système de **retrieval** hybride repose généralement sur plusieurs sous-composants exécutés en parallèle ou en cascade :

- un moteur de recherche vectorielle (dense)
- un moteur de recherche lexicale (sparse, **BM25**)
- un module de fusion et de scoring
- éventuellement un **reranker** final

Chaque composant produit un ensemble de candidats, qui sont ensuite combinés pour construire un **Top-K** final optimisé.

Cette séparation permet de découpler les forces de chaque approche tout en conservant une flexibilité maximale dans la stratégie de **retrieval**.

Exécution parallèle des recherches

Dans une architecture industrielle, les recherches dense et sparse sont généralement exécutées en parallèle afin de minimiser la latence globale. Chaque moteur retourne un top-N indépendant, souvent supérieur au K final souhaité.

Cette stratégie permet de maximiser le recall initial sans impacter significativement les performances, puisque les deux recherches exploitent des index différents optimisés pour leurs propres types de requêtes.

Fusion des résultats

Une fois les deux listes de résultats obtenues, le système procède à une étape de fusion. Cette étape consiste à unifier les candidats provenant des deux sources tout en gérant les doublons et les conflits de scoring.

Les techniques utilisées incluent :

- fusion pondérée des scores normalisés
- rank aggregation
- déduplication basée sur les identifiants de chunks
- ajustement par métadonnées

L'objectif est de produire une liste cohérente qui reflète à la fois la similarité sémantique et la pertinence lexicale.

Ajout du reranking

Dans les systèmes les plus avancés, la fusion hybride est suivie d'une étape de **reranking**. Un modèle plus coûteux mais plus précis réévalue les candidats issus du **retrieval** hybride afin de produire un classement final optimisé.

Cette architecture en trois étapes (dense + sparse → fusion → rerank) est aujourd'hui un standard dans les systèmes **RAG** industriels.

Avantages de l'architecture hybride

L'approche hybride offre plusieurs avantages majeurs :

- amélioration significative du recall global
- robustesse face à des types de requêtes variés
- meilleure couverture des cas techniques et sémantiques
- réduction des erreurs de **retrieval** critiques
- adaptation à des corpus hétérogènes

Elle permet également de compenser les faiblesses structurelles de chaque approche prise isolément.

Coût et complexité système

L'architecture hybride introduit une complexité supplémentaire en termes d'infrastructure, car elle nécessite la maintenance de deux systèmes de recherche distincts. Cela implique également une gestion plus fine des ressources et de la latence.

Cependant, dans les environnements de production, ce surcoût est généralement considéré comme acceptable au regard du gain en qualité et en fiabilité.

Conclusion

La hybrid **retrieval** architecture représente une étape clé dans l'industrialisation des systèmes **RAG**. Elle transforme la recherche d'information en un **pipeline** multi-sources, capable de combiner plusieurs paradigmes de recherche pour produire un contexte plus riche, plus précis et plus robuste.

Dans les systèmes modernes, elle constitue la base standard de tout moteur de **retrieval** sérieux, en particulier dans les contextes à forte exigence de qualité et de **scalabilité**.

Section 7.4 — Cas d'usage entreprise

Les architectures de recherche hybride (dense + sparse) ne sont pas seulement une optimisation théorique : elles répondent à des besoins concrets rencontrés dans les environnements industriels. En entreprise, les systèmes **RAG** doivent gérer des utilisateurs variés, des données hétérogènes et des exigences fortes en termes de fiabilité, de traçabilité et de performance.

Le **retrieval** hybride devient alors un standard, car il permet d'adresser simultanément des cas d'usage très différents au sein d'un même système.

Recherche documentaire interne

Un des cas d'usage les plus fréquents est la recherche documentaire interne. Les entreprises disposent souvent de volumes importants de documents : procédures, guides techniques, rapports, documentation produit ou juridique.

Dans ce contexte, la recherche dense est particulièrement efficace pour comprendre les requêtes formulées de manière naturelle par les utilisateurs, tandis que la recherche lexicale est essentielle pour retrouver des termes précis comme des références techniques, des noms de produits ou des clauses contractuelles.

La combinaison des deux permet de garantir un accès fiable à l'information, même lorsque les utilisateurs ne connaissent pas exactement la terminologie utilisée dans les documents.

Support client et assistance technique

Dans les systèmes de support client, les requêtes sont souvent courtes, ambiguës ou mal formulées. Les utilisateurs décrivent des problèmes sans utiliser les termes techniques exacts.

La recherche dense permet de capturer l'intention globale de la requête, tandis que la recherche sparse permet de retrouver des logs d'erreurs, des codes spécifiques ou des messages système exacts.

L'architecture hybride améliore ainsi la capacité du système à identifier rapidement les bonnes solutions, réduisant le temps de résolution et améliorant l'expérience utilisateur.

Recherche juridique et conformité

Les domaines juridiques et réglementaires exigent une grande précision terminologique. Dans ces cas, la recherche lexicale joue un rôle critique pour retrouver des articles de loi, des clauses contractuelles ou des références normatives exactes.

Cependant, les utilisateurs peuvent formuler leurs questions de manière indirecte ou descriptive. La recherche dense permet alors de traduire ces formulations en concepts juridiques pertinents.

La combinaison des deux approches permet de garantir à la fois la précision et la compréhension du contexte, ce qui est essentiel dans des environnements à forte contrainte réglementaire.

Systèmes techniques et documentation produit

Dans les environnements techniques, les utilisateurs recherchent souvent des informations très spécifiques : erreurs systèmes, paramètres de configuration, versions de logiciels ou commandes précises.

La recherche sparse est ici indispensable pour capturer les correspondances exactes, tandis que la recherche dense permet de gérer les variations de formulation et les descriptions plus générales.

L'approche hybride permet ainsi de couvrir à la fois les usages experts et les requêtes plus exploratoires.

Systèmes multi-domaines

Dans les grandes organisations, les systèmes **RAG** doivent souvent couvrir plusieurs domaines simultanément : finance, RH, technique, juridique, produit.

Dans ce contexte, aucune approche unique ne suffit. La recherche hybride permet de s'adapter dynamiquement à la diversité des requêtes et des corpus, en combinant les forces des deux paradigmes.

Cela permet de construire des systèmes unifiés capables de servir différents métiers sans multiplier les moteurs spécialisés.

Impact sur l'architecture globale

L'adoption de la recherche hybride en entreprise influence directement l'ensemble de l'architecture **RAG**. Elle impose la coexistence de plusieurs moteurs de recherche, une logique de fusion avancée et souvent un **reranking** centralisé.

En contrepartie, elle améliore significativement la robustesse du système et réduit les risques de réponses incorrectes ou incomplètes.

Conclusion

Les cas d'usage entreprise montrent que la recherche hybride n'est pas une option avancée, mais une nécessité dans les systèmes **RAG** industriels. Elle permet de concilier précision, robustesse et flexibilité dans des environnements complexes où les requêtes et les données sont extrêmement variées.

Ainsi, elle constitue une brique fondamentale des architectures de **retrieval** modernes en production.

Chapitre 8 — Reranking et amélioration de pertinence

Dans un système **RAG** industriel, le **retrieval** initial (qu'il soit vectoriel, lexical ou hybride) n'est qu'une étape intermédiaire. Il fournit un ensemble de candidats potentiellement pertinents, mais ne garantit pas que ces documents soient réellement les meilleurs pour répondre à la requête utilisateur. C'est précisément le rôle du **reranking** : transformer une liste approximative en un

classement optimisé pour la génération.

Section 8.1 — Pourquoi le retrieval seul est insuffisant

Le **retrieval**, même optimisé avec des techniques **ANN** ou des approches hybrides, reste fondamentalement une étape d'approximation. Il est conçu pour être rapide et scalable, pas pour produire un classement parfait. Cette contrainte introduit plusieurs limites structurelles qui justifient l'ajout d'une couche de **reranking**.

1. Limites des approximations ANN

Les moteurs de recherche vectorielle reposent sur des algorithmes approximatifs comme **HNSW** ou **IVF**. Ces méthodes sacrifient une partie de la précision pour obtenir des performances compatibles avec la production.

En conséquence :

- certains documents pertinents peuvent ne pas apparaître dans le **Top-K** initial
- des documents “proches mais non pertinents” peuvent être surreprésentés
- l'ordre des résultats n'est pas optimal du point de vue sémantique réel

Le **retrieval** est donc efficace pour filtrer, mais imparfait pour ordonner.

2. Ambiguïté sémantique des embeddings

Les **embeddings** compressent l'information textuelle dans un espace vectoriel de dimension fixe. Cette compression implique une perte d'information.

Deux documents peuvent être proches dans l'espace vectoriel tout en étant très différents en termes de pertinence réelle pour une requête donnée. Inversement, des documents réellement importants peuvent être légèrement éloignés dans l'espace vectoriel.

Cette ambiguïté rend le classement initial instable et parfois trompeur.

3. Manque de compréhension fine de la requête

Le **retrieval** repose généralement sur une similarité globale entre requête et documents. Il ne comprend pas :

- les intentions implicites de l'utilisateur
- les contraintes spécifiques de la question
- les nuances de priorité entre différents aspects de la requête

Ainsi, il peut récupérer des documents globalement similaires mais non pertinents dans le contexte exact de la question.

4. Redondance et bruit dans les résultats

Le **Top-K** issu du **retrieval** contient souvent :

- des doublons sémantiques
- des chunks provenant du même document
- des passages redondants exprimant la même idée

Ce bruit réduit la diversité du contexte fourni au **LLM** et peut dégrader la qualité de la génération.

5. Absence de notion de “qualité globale du contexte”

Le **retrieval** évalue chaque chunk individuellement, sans considérer la cohérence globale de l'ensemble des résultats. Or, dans un système **RAG**, ce n'est pas seulement la pertinence individuelle des documents qui compte, mais la qualité du contexte global transmis au modèle de langage.

6. Limites des scores bruts

Les scores de similarité (cosinus, **BM25**, etc.) ne sont pas directement optimisés pour la génération de texte. Ils mesurent une proximité, pas une utilité dans le cadre d'une réponse complète.

Un document peut avoir un score élevé mais être inutile pour répondre à une question précise, car il manque de contexte ou ne couvre qu'un aspect secondaire de la requête.

Conclusion

Le **retrieval** seul est une étape de sélection approximative, conçue pour la vitesse et la **scalabilité**. Il est efficace pour réduire un grand espace de recherche à un ensemble de candidats, mais insuffisant pour garantir une pertinence optimale.

C'est cette limite structurelle qui rend nécessaire l'introduction d'une étape de **reranking**, capable de réévaluer finement les candidats et d'optimiser le contexte final fourni au **LLM**.

Section 8.2 — Cross-encoders

Les cross-encoders constituent l'une des approches les plus efficaces pour améliorer la pertinence du **retrieval** dans les systèmes **RAG**. Contrairement aux modèles d'**embeddings** (bi-encoders), qui

encodent indépendamment la requête et les documents, les cross-encoders évaluent la relation entre les deux de manière conjointe.

Cette différence architecturale a un impact direct sur la qualité du scoring et explique pourquoi les cross-encoders sont presque systématiquement utilisés dans les étapes de **reranking** en production.

Principe fondamental

Un **cross-encoder** prend en entrée une paire (requête, document) et les traite ensemble dans un seul passage du modèle. Au lieu de produire deux vecteurs indépendants comparés ensuite par une métrique de distance, le modèle apprend directement à estimer un score de pertinence.

Cette approche permet une interaction fine entre tous les tokens de la requête et ceux du document, via des mécanismes d'attention croisée. Le modèle peut ainsi comprendre des relations complexes, des dépendances lexicales et des nuances contextuelles impossibles à capturer avec une simple similarité vectorielle.

Différence avec les bi-encoders

Les bi-encoders, utilisés pour la recherche initiale, sont optimisés pour la rapidité. Ils encodent séparément requêtes et documents afin de permettre un calcul de similarité rapide dans un espace vectoriel.

Cependant, cette séparation impose une limitation importante : aucune interaction directe entre requête et document n'a lieu au moment du calcul de similarité.

Les cross-encoders, au contraire, sacrifiant la vitesse au profit de la précision, permettent une compréhension beaucoup plus fine du lien entre les deux éléments.

Avantage en reranking

Dans un **pipeline RAG**, les cross-encoders sont utilisés après la phase de **retrieval** initial. Leur rôle est de réordonner un ensemble restreint de candidats (souvent top 50 à top 200) afin de produire un classement final plus précis.

Ils permettent notamment de :

- mieux distinguer les documents très proches sémantiquement
- capturer les nuances contextuelles de la requête
- réduire les erreurs de classement des **embeddings**
- améliorer la cohérence du contexte final fourni au **LLM**

Coût computationnel

Le principal inconvénient des cross-encoders est leur coût. Contrairement aux bi-encoders, ils ne permettent pas de pré-calculer les représentations des documents. Chaque paire requête-document doit être évaluée individuellement.

Cela implique une complexité $O(N)$ sur le nombre de candidats, ce qui limite leur utilisation à des ensembles réduits après une première étape de **retrieval**.

En production, ils sont donc utilisés comme une couche finale de précision, et non comme moteur principal de recherche.

Impact sur la qualité du RAG

L'intégration d'un **cross-encoder** dans un **pipeline RAG** améliore significativement la qualité des réponses générées. En affinant le classement des documents, il garantit que le **LLM** reçoit un contexte plus pertinent, plus cohérent et mieux aligné avec l'intention réelle de la requête.

Cette amélioration se traduit directement par :

- une réduction des **hallucinations**
- une meilleure précision factuelle
- une meilleure cohérence des réponses longues

Conclusion

Les cross-encoders jouent un rôle critique dans les architectures de **reranking** modernes. Bien qu'ils soient coûteux en calcul, leur capacité à modéliser finement l'interaction entre requête et document en fait un outil indispensable pour maximiser la pertinence du contexte dans les systèmes **RAG** industriels.

Ils représentent ainsi la couche de précision ultime entre le **retrieval** approximatif et la génération du **LLM**.

Section 8.3 — BGE Reranker et modèles modernes

Les rerankers modernes représentent une évolution directe des cross-encoders classiques, optimisés spécifiquement pour les tâches de **retrieval** en production. Parmi eux, les modèles de la famille BGE **reranker** (BAAI General **embedding** Reranker) se sont imposés comme une référence dans les systèmes **RAG** industriels grâce à leur équilibre entre performance, robustesse et coût d'inférence.

Ces modèles ne se limitent pas à améliorer le classement des documents : ils sont conçus dès l'origine pour s'intégrer dans des pipelines de recherche hybrides à grande échelle.

Principe des BGE Rerankers

Les BGE Rerankers reposent sur une architecture de type **cross-encoder**, où la requête et le document sont traités conjointement. Cependant, ils se distinguent des cross-encoders génériques par un entraînement fortement orienté **retrieval**.

Ils sont optimisés sur des datasets de recherche sémantique et de question-réponse, avec des objectifs explicites de classement (ranking loss) plutôt que de simple similarité.

Cette spécialisation leur permet de mieux capturer la notion de pertinence dans un contexte **RAG**, où l'objectif n'est pas seulement de mesurer la proximité sémantique, mais de maximiser l'utilité du document pour la génération finale.

Améliorations par rapport aux cross-encoders classiques

Les modèles BGE **reranker** introduisent plusieurs améliorations importantes :

- meilleure généralisation sur des domaines hétérogènes
- robustesse accrue aux formulations ambiguës
- optimisation spécifique pour le **Top-K reranking**
- meilleure stabilité sur des corpus bruités ou incomplets

Ces améliorations proviennent d'un entraînement sur des données plus proches des conditions réelles d'un système **RAG** industriel.

Position dans le pipeline RAG

Dans une architecture moderne, le BGE **reranker** intervient après la phase de **retrieval** initial :

- **retrieval** vectoriel ou hybride (top 50–200 candidats)
- **reranking** avec BGE **reranker**
- sélection finale du contexte (top 5–20 chunks)

Cette position stratégique permet de limiter le coût computationnel tout en maximisant l'impact sur la qualité finale du contexte envoyé au **LLM**.

Compromis performance / latence

Comme tous les rerankers basés sur cross-encoders, les modèles BGE introduisent un coût supplémentaire important en latence. Chaque paire requête-document doit être évaluée individuellement, ce qui limite leur utilisation à un sous-ensemble réduit de candidats.

En production, ce coût est généralement compensé par :

- une réduction drastique du nombre de documents évalués
- une exécution batch optimisée
- parfois une quantification du modèle (FP16, INT8)

Impact sur la qualité du RAG

L'intégration d'un BGE **reranker** dans un système **RAG** améliore significativement la pertinence du contexte final. Les documents fournis au **LLM** sont mieux alignés avec l'intention réelle de la requête, ce qui se traduit par :

- une meilleure précision factuelle
- une réduction des **hallucinations**
- une amélioration de la cohérence des réponses
- une meilleure hiérarchisation des informations importantes

Modèles modernes et tendances

Au-delà de BGE, de nouveaux modèles de **reranking** émergent avec des objectifs similaires mais des optimisations supplémentaires :

- modèles distillés pour réduire la latence
- rerankers multi-task (**retrieval** + classification)
- modèles intégrant des signaux hybrides (dense + sparse + metadata)
- architectures optimisées pour GPU inference batch

Ces évolutions montrent une tendance claire : le **reranking** devient une brique standardisée et industrialisée des systèmes **RAG** modernes.

Conclusion

Les BGE Rerankers incarnent l'état de l'art actuel du **reranking** en production. Ils combinent la précision des cross-encoders avec des optimisations spécifiques au **retrieval** industriel, ce qui en fait un composant essentiel pour maximiser la qualité des systèmes **RAG**.

Ils constituent la dernière étape critique avant la génération, assurant que le **LLM** reçoit un contexte à la fois pertinent, cohérent et hiérarchisé.

Section 8.4 — Pipeline retrieval → rerank → LLM

Le **pipeline retrieval** → rerank → **LLM** constitue aujourd'hui l'architecture standard des systèmes **RAG** industriels. Il formalise une séparation claire entre trois niveaux de traitement : la recherche approximative à grande échelle, l'affinage de pertinence, et la génération de réponse. Cette

structuration permet d'obtenir un équilibre optimal entre **scalabilité**, précision et qualité de génération.

Vue d'ensemble du pipeline

Dans un système **RAG** moderne, le traitement d'une requête suit généralement trois étapes successives :

- **retrieval** (dense / sparse / hybride)

Sélection d'un ensemble large de candidats pertinents (top 50 à 500 chunks).

reranking (cross-encoder / BGE reranker)

Réévaluation fine des candidats pour produire un classement optimisé (top 5 à 20).

LLM generation

Injection du contexte final dans le modèle de langage pour produire la réponse.

Cette architecture en cascade permet de répartir les responsabilités entre des modules optimisés pour des objectifs différents.

Rôle du retrieval : couverture maximale

La première étape vise à maximiser le recall. Le **retrieval** est conçu pour être rapide et scalable, même au prix d'une précision imparfaite.

Son rôle est de garantir qu'aucune information potentiellement pertinente ne soit oubliée. Il agit comme un filtre large, qui réduit l'espace de recherche global à un sous-ensemble exploitable.

Cependant, ce sous-ensemble contient souvent du bruit, des doublons et des éléments partiellement pertinents.

Rôle du reranking : précision maximale

Le **reranking** intervient comme une couche d'affinage. Contrairement au **retrieval**, il ne cherche pas à être rapide à grande échelle, mais à être précis sur un ensemble restreint.

En réévaluant chaque candidat en fonction de la requête, il permet de corriger les erreurs de classement initial et d'identifier les documents réellement les plus utiles pour la génération.

Cette étape transforme un ensemble approximatif en un contexte structuré et hiérarchisé.

Rôle du LLM : synthèse et génération

Le **LLM** n'a pas pour rôle de rechercher ou de classer l'information, mais de synthétiser un contexte déjà optimisé. Sa performance dépend directement de la qualité du **pipeline** en amont.

Un contexte mal sélectionné entraîne des **hallucinations**, des incohérences ou des réponses incomplètes. À l'inverse, un contexte bien reranké permet au modèle de produire des réponses précises, structurées et fiables.

Séparation des responsabilités

L'un des principes fondamentaux de cette architecture est la séparation des rôles :

- le **retrieval** optimise la couverture
- le **reranking** optimise la pertinence
- le **LLM** optimise la génération

Cette séparation permet de construire des systèmes robustes, où chaque composant est spécialisé et optimisé indépendamment.

Optimisation globale du pipeline

Les performances globales d'un système **RAG** ne dépendent pas uniquement de la qualité individuelle de chaque composant, mais de leur interaction. Plusieurs paramètres doivent être ajustés conjointement :

- taille du **Top-K** initial
- nombre de candidats rerankés
- taille du contexte final injecté au **LLM**
- stratégie de **chunking** en amont
- type de modèle de **reranking** utilisé

L'optimisation de ce **pipeline** est souvent empirique et repose sur des benchmarks spécifiques au domaine d'application.

Contraintes de production

En environnement industriel, ce **pipeline** doit répondre à plusieurs contraintes simultanées :

- latence globale faible (souvent < 1–3 secondes)
- robustesse sur des requêtes variées
- stabilité des résultats
- coût maîtrisé à grande échelle

Cela impose des choix d'architecture précis, notamment sur la taille des modèles de **reranking** et la profondeur du **retrieval** initial.

Conclusion

Le **pipeline retrieval** → rerank → **LLM** constitue la colonne vertébrale des systèmes **RAG** modernes. Il permet de transformer un problème de recherche approximative en un système de génération contrôlée et fiable.

En séparant clairement les rôles entre recherche, optimisation et génération, cette architecture offre un compromis optimal entre performance industrielle et qualité de réponse, ce qui en fait aujourd'hui le standard des déploiements **RAG** en production.

Chapitre 9 — Évaluation du système de retrieval

Évaluer un système de **retrieval** dans un **RAG** n'est pas une étape optionnelle : c'est une condition essentielle pour garantir sa fiabilité en production. Contrairement à la génération, où l'évaluation peut être qualitative et subjective, le retrieval peut être mesuré avec des métriques précises issues de la recherche d'information.

Ces métriques permettent de comparer des stratégies de **chunking**, d'**embeddings**, d'indexation ou de **reranking**, et d'optimiser progressivement l'ensemble du **pipeline**.

Section 9.1 — Recall@K

Le Recall@K est l'une des métriques les plus fondamentales pour évaluer un système de **retrieval**. Il mesure la capacité du système à retrouver les documents pertinents dans les K premiers résultats retournés.

Définition intuitive

Le Recall@K répond à une question simple :

- Parmi tous les documents réellement pertinents pour une requête, combien ont été retrouvés dans les K premiers résultats ?

Autrement dit, il mesure la couverture de la vérité terrain par le système de recherche.

Formulation formelle

Le Recall@K peut être défini comme :

$$\text{Recall@K} = \frac{\text{Nombre de documents pertinents dans le top K}}{\text{Nombre total de documents pertinents}}$$

- où :
- le numérateur correspond aux documents pertinents effectivement retrouvés dans les K premiers résultats
- le dénominateur correspond à l'ensemble des documents pertinents existants pour la requête

Interprétation dans un système RAG

Dans un système **RAG**, le Recall@K mesure la capacité du **retrieval** à ne pas manquer d'informations importantes.

Un Recall@K élevé signifie que :

- le système couvre bien le champ sémantique de la requête
- les chunks pertinents sont correctement indexés et retrouvés
- le **LLM** dispose d'un contexte suffisant pour générer une réponse fiable

À l'inverse, un Recall@K faible indique que des informations importantes sont perdues dès la phase de **retrieval**, ce qui ne peut pas être corrigé par le **reranking** ou le **LLM**.

Importance du choix de K

Le paramètre K est crucial dans l'interprétation de la métrique :

- petit K (ex : 5–10) → mesure stricte, orientée production **LLM**
- K moyen (ex : 20–50) → équilibre recall / bruit
- grand K (ex : 100+) → évaluation du potentiel maximal du **retrieval**

Dans les systèmes **RAG** industriels, on distingue souvent :

- Recall@retrieval (large K)

Recall@rerank (K réduit)

Recall@context (K final injecté dans le **LLM**)

Limites du Recall@K

Bien que fondamental, le Recall@K présente plusieurs limites :

- il ne mesure pas l'ordre des résultats

- il ne tient pas compte de la qualité relative des documents
- il ne reflète pas la redondance ou le bruit dans les résultats
- il dépend fortement de la qualité du ground truth

Ainsi, un système peut avoir un Recall@K élevé tout en fournissant un contexte peu optimal pour le **LLM**.

Rôle dans l'optimisation RAG

Le Recall@K est particulièrement utile pour :

- comparer différents modèles d'**embeddings**
- ajuster les stratégies de **chunking**
- calibrer les paramètres **ANN** (**efSearch**, **nprobe**, etc.)
- évaluer l'impact de la recherche hybride

Il constitue souvent la première métrique utilisée lors de la conception d'un système de **retrieval**.

Conclusion

Le Recall@K est une métrique centrale pour mesurer la couverture d'un système de **retrieval**. Dans un **RAG** industriel, il permet de s'assurer que l'information pertinente est bien récupérée avant toute étape de **reranking** ou de génération.

Il représente donc la base de toute évaluation sérieuse du **pipeline** de recherche.

Section 9.2 — Precision@K

Si le Recall@K mesure la capacité d'un système de **retrieval** à ne pas manquer l'information pertinente, la Precision@K mesure l'autre face du problème : la capacité du système à ne pas introduire de bruit dans les résultats.

Dans un système **RAG**, cette métrique est critique car elle influence directement la qualité du contexte injecté dans le **LLM**. Un contexte riche mais bruité peut être aussi problématique qu'un contexte incomplet.

Définition intuitive

La Precision@K répond à la question suivante :

- Parmi les K résultats retournés, quelle proportion est réellement pertinente ?

Elle mesure donc la qualité moyenne des résultats fournis au modèle de génération.

Formulation formelle

$$\text{Precision@K} = \frac{\text{Nombre de documents pertinents dans le top K}}{K}$$

- où :
- le numérateur représente les documents pertinents retrouvés dans les K premiers résultats
- le dénominateur est le nombre total de résultats retournés (K)

Interprétation dans un système RAG

Dans un **pipeline RAG**, la Precision@K reflète la propreté du contexte fourni au **LLM**.

Une précision élevée signifie que :

- les documents injectés sont majoritairement pertinents
- le **reranking** et le **retrieval** filtrent efficacement le bruit
- le **LLM** reçoit un contexte exploitable sans surcharge inutile

À l'inverse, une faible précision implique :

- présence de documents hors sujet
- dilution de l'information utile
- augmentation du risque d'**hallucinations** ou de réponses imprécises

Trade-off avec le Recall@K

La Precision@K est fortement liée au Recall@K, mais dans une relation de compromis :

- augmenter K améliore souvent le recall
- mais peut réduire la précision
- réduire K améliore la précision
- mais peut diminuer le recall

Ce compromis est central dans la conception des pipelines **RAG**, notamment dans le choix du **Top-K** après **reranking**.

Rôle du reranking

Le **reranking** joue un rôle déterminant dans l'amélioration de la Precision@K. En réordonnant les candidats selon une estimation plus fine de la pertinence, il permet de :

- éliminer les faux positifs du **retrieval** initial

- prioriser les chunks réellement utiles
- améliorer la densité d'information pertinente dans le contexte

Ainsi, le **reranking** agit directement comme un mécanisme d'optimisation de la précision.

Limites de la Precision@K

Comme toute métrique isolée, la Precision@K a des limites :

- elle ne mesure pas la couverture globale de l'information (recall)
- elle ne tient pas compte de l'ordre des résultats
- elle ne distingue pas les degrés de pertinence
- elle peut être artificiellement élevée avec un K très petit

Elle doit donc être interprétée en combinaison avec d'autres métriques.

Utilisation en production

En environnement industriel, la Precision@K est utilisée pour :

- valider la qualité du **reranking**
- ajuster les pipelines hybrides
- calibrer la taille du contexte envoyé au **LLM**
- surveiller la dégradation du **retrieval** dans le temps

Elle est particulièrement importante dans les systèmes où la qualité du contexte est plus critique que la couverture exhaustive.

Conclusion

La Precision@K est une métrique essentielle pour mesurer la qualité du contexte produit par un système de **retrieval**. Dans une architecture **RAG**, elle permet de s'assurer que les informations transmises au **LLM** sont non seulement pertinentes, mais également suffisamment propres pour garantir des réponses fiables.

Elle complète naturellement le Recall@K en offrant une vision centrée sur la qualité plutôt que sur la couverture.

Section 9.3 — MRR et NDCG

Dans l'évaluation d'un système de **retrieval**, les métriques Recall@K et Precision@K donnent une vision globale de la couverture et de la qualité des résultats. Cependant, elles ne prennent pas

pleinement en compte un élément crucial en **RAG** : la position des résultats pertinents dans le ranking. C'est précisément ce que mesurent des métriques comme le MRR et le NDCG.

Ces métriques sont particulièrement importantes en production, car le contexte envoyé au **LLM** est fortement dépendant des premiers résultats du **retrieval**.

Mean Reciprocal Rank (MRR)

Définition intuitive

Le MRR mesure la qualité du premier résultat pertinent retourné par le système.

Il répond à une question simple :

- À quelle position apparaît le premier document pertinent ?

Formulation

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}$$

- où :
- (Q) est l'ensemble des requêtes

(rank_q) est la position du premier document pertinent pour la requête (q)

Interprétation en RAG

Dans un système **RAG**, le MRR est extrêmement important car le **LLM** ne traite qu'un contexte limité (**Top-K** réduit). Cela signifie que :

- si le premier document pertinent apparaît tôt → forte probabilité de bonne réponse
- si le premier document pertinent est mal classé → le **LLM** peut ne jamais le voir

Ainsi, le MRR mesure directement l'efficacité du système à "placer rapidement la bonne information".

Limites du MRR

- ne prend en compte que le premier résultat pertinent
- ignore les autres documents utiles
- sensible aux datasets mal annotés
- peu représentatif des contextes complexes multi-documents

Normalized Discounted Cumulative Gain (NDCG)

Définition intuitive

Le NDCG mesure la qualité globale du ranking en tenant compte :

- de la pertinence des documents
- de leur position dans la liste
- d'une décroissance logarithmique de l'importance selon le rang

Contrairement au MRR, il ne se limite pas au premier résultat pertinent.

Formulation

$$NDCG@K = \frac{DCG@K}{IDCG@K}$$

- avec :

$$DCG@K = \sum_{i=1}^K \frac{rel_i}{\log_2(i+1)}$$

Interprétation en RAG

Le NDCG est particulièrement adapté aux systèmes **RAG** car il :

- prend en compte plusieurs documents pertinents
- valorise les documents les mieux classés
- pénalise fortement les mauvais ordonnancements en haut de liste
- Dans un **pipeline retrieval** → rerank → **LLM**, le NDCG est souvent la métrique la plus représentative de la qualité globale du ranking.

MRR vs NDCG

Importance pour le reranking

Le **reranking** a un impact direct sur ces deux métriques :

- améliore fortement le MRR en remontant le meilleur document
- améliore le NDCG en optimisant tout l'ordre des résultats

Ainsi, ces métriques sont souvent utilisées pour comparer différents modèles de **reranking** (cross-encoders, BGE rerankers, etc.).

Conclusion

Le MRR et le NDCG complètent les métriques classiques du **retrieval** en introduisant la notion essentielle de position dans le ranking. Dans les systèmes **RAG** industriels, ils sont indispensables pour évaluer la qualité réelle du **pipeline**, car ils reflètent directement ce que le **LLM** verra en entrée.

Ils constituent donc un niveau d'évaluation plus fin et plus réaliste que le simple Recall@K ou Precision@K.

Section 9.4 — Benchmarks automatisés

Dans un système **RAG** industriel, l'évaluation ponctuelle des métriques (Recall@K, Precision@K, MRR, NDCG) ne suffit pas. Dès que le système évolue — nouveaux **embeddings**, changement de **chunking**, modification de l'index ou ajout de **reranking** — la qualité peut se dégrader sans alerte visible.

C'est pourquoi les organisations sérieuses mettent en place des benchmarks automatisés, qui permettent de surveiller en continu la performance du **retrieval** et de comparer objectivement différentes versions du système.

Principe général

Un benchmark automatisé **RAG** consiste à exécuter un ensemble de requêtes de référence sur différentes versions du **pipeline**, puis à mesurer automatiquement les métriques de **retrieval** associées.

Le système compare alors :

- version actuelle vs version précédente
- différents modèles d'**embeddings**
- différentes stratégies de **chunking**
- différentes configurations **ANN** et **reranking**

L'objectif est de transformer l'évaluation en un processus reproductible, continu et intégré au cycle de développement.

Construction d'un dataset de benchmark

Un benchmark fiable repose sur un dataset structuré comprenant :

- des requêtes utilisateur représentatives
- des documents ou chunks considérés comme pertinents (ground truth)
- parfois des annotations de niveau de pertinence (faible, moyen, élevé)

La qualité de ce dataset est critique : un benchmark mal construit conduit à des décisions d'optimisation erronées.

Dans les systèmes avancés, ce dataset est souvent enrichi en continu à partir des logs de production.

Pipeline d'évaluation automatisée

Un benchmark automatisé suit généralement une chaîne d'exécution standard :

- ingestion des requêtes de test
- exécution du **retrieval** (dense, sparse ou hybride)
- application éventuelle du **reranking**
- calcul des métriques (Recall@K, MRR, NDCG, etc.)
- agrégation et comparaison avec les versions précédentes

Ce **pipeline** est souvent intégré dans des outils CI/CD ou des jobs planifiés.

Comparaison de versions (A/B testing)

Les benchmarks automatisés permettent de comparer objectivement différentes configurations du système **RAG**.

Par exemple :

- nouveau modèle d'**embedding** vs ancien modèle

HNSW vs IVF

- ajout d'un **reranker** vs **pipeline** sans **reranking**
- modification de la taille des chunks

Les résultats sont analysés non seulement globalement, mais aussi par segment de requêtes (techniques, juridiques, conversationnelles, etc.).

Détection de régressions

L'un des rôles principaux des benchmarks automatisés est la détection de régressions.

Un changement apparemment mineur (ex : nouveau modèle d'**embedding** plus performant en théorie) peut dégrader :

- le Recall@K sur certains types de requêtes
- la stabilité du **reranking**
- la qualité du **Top-K** final

Le benchmark agit donc comme un système de garde-fou avant déploiement en production.

Monitoring continu en production

Au-delà des tests offline, les systèmes avancés intègrent également un monitoring continu basé sur des métriques approximatives calculées sur les logs de production.

Cela permet de détecter :

- dérive des distributions de requêtes
- dégradation progressive du **retrieval**
- impact des nouveaux documents ingérés
- variations de performance selon les périodes

Limites des benchmarks automatisés

Malgré leur importance, les benchmarks présentent plusieurs limites :

- ils ne capturent pas toujours la complexité des requêtes réelles
- le ground truth peut être incomplet ou biaisé
- ils ne mesurent pas directement la qualité de génération du **LLM**
- ils peuvent être sur-optimisés ("overfitting au benchmark")

C'est pourquoi ils doivent être complétés par des tests utilisateurs et des évaluations end-to-end.

Conclusion

Les benchmarks automatisés constituent un pilier fondamental de l'industrialisation des systèmes **RAG**. Ils permettent de transformer une évaluation ponctuelle en un processus continu, reproductible et orienté production.

En combinant métriques classiques et exécution automatisée, ils garantissent la stabilité et la qualité du **retrieval** dans le temps, même lorsque le système évolue rapidement.

Chapitre 10 — Gestion des métadonnées avancées

Dans un système **RAG** industriel, les documents ne sont jamais de simples blocs de texte. Ils sont enrichis de métadonnées qui décrivent leur origine, leur structure, leur temporalité et leur contexte d'utilisation. Ces informations jouent un rôle critique dans le **retrieval**, le **reranking** et même la génération finale.

La gestion des métadonnées devient donc une couche essentielle de l'architecture **RAG**, au même titre que les **embeddings** ou les index vectoriels.

Section 10.1 — Structuration des documents

La structuration des documents consiste à organiser les données textuelles et leurs attributs associés de manière cohérente, exploitable et interrogeable par le système de **retrieval**. Elle transforme un corpus brut en un ensemble d'objets structurés, capables de supporter des filtres, des contraintes et des logiques métier complexes.

Du document brut au document structuré

Dans un système naïf, un document est simplement un texte découpé en chunks et transformé en **embeddings**. Cette approche est insuffisante en production.

Dans une architecture industrielle, chaque document est représenté comme une entité structurée contenant :

- un identifiant unique
- un ensemble de chunks associés
- des métadonnées globales
- des métadonnées par chunk

Cette structuration permet de relier chaque fragment de texte à son contexte d'origine.

Types de métadonnées

Les métadonnées peuvent être classées en plusieurs catégories selon leur rôle dans le système **RAG** :

- Métadonnées descriptives
 - titre du document
 - auteur ou source
 - catégorie ou domaine
 - langue

Métadonnées temporelles

- date de création
- date de modification
- version du document
- fraîcheur des données

Métadonnées structurelles

- section ou chapitre d'origine
- position du chunk dans le document
- hiérarchie documentaire

Métadonnées techniques

- format source (PDF, HTML, DOCX)
- méthode de parsing utilisée
- qualité d'extraction

Rôle des métadonnées dans le retrieval

Les métadonnées ne sont pas uniquement informatives : elles influencent directement le processus de **retrieval**.

Elles permettent notamment :

- de filtrer les documents avant ou après la recherche vectorielle
- de restreindre la recherche à certains domaines ou catégories
- de prioriser les documents récents ou validés
- d'améliorer la précision du **Top-K** final

Dans certains systèmes, les métadonnées sont même intégrées dans le scoring global du **retrieval**.

Métadonnées et chunking

Chaque chunk hérite généralement des métadonnées de son document parent, mais peut également posséder ses propres attributs. Par exemple :

- un chunk peut appartenir à une section spécifique
- un chunk peut être marqué comme "résumé" ou "important"
- un chunk peut être exclu du **retrieval** standard

Cette granularité permet un contrôle fin du comportement du système **RAG**.

Impact sur la scalabilité

À grande échelle, la gestion des métadonnées devient un enjeu de performance. Plus les métadonnées sont riches, plus les opérations de filtrage et de requêtage deviennent coûteuses.

Il est donc nécessaire de :

- structurer les métadonnées de manière indexable
- éviter les schémas trop complexes ou non normalisés
- optimiser les filtres pour les moteurs de recherche vectorielle

Intégration dans les bases vectorielles

Les bases vectorielles modernes (Milvus, **Qdrant**, Weaviate) permettent de stocker des métadonnées associées aux **embeddings**. Ces métadonnées sont utilisées pour :

- filtrer les résultats avant similarité
- post-filtrer les résultats du **Top-K**
- enrichir les résultats pour le **reranking**

Cette intégration fait des métadonnées un élément natif du **pipeline** de **retrieval**.

Conclusion

La structuration des documents est une étape fondamentale de l'industrialisation du **RAG**. Elle transforme des données textuelles brutes en objets riches, exploitables et filtrables, permettant de combiner recherche vectorielle et contraintes métier.

Sans une structuration solide des métadonnées, même les meilleurs modèles d'**embeddings** ou de **reranking** ne peuvent compenser les limites du système.

Section 10.2 — Filtrage multi-critères

Le filtrage multi-critères est une extension naturelle de la gestion des métadonnées dans les systèmes **RAG** industriels. Il consiste à appliquer plusieurs conditions simultanées pour restreindre ou prioriser les documents candidats avant ou après la recherche vectorielle. Cette étape permet d'aligner le **retrieval** avec des contraintes métier précises, tout en améliorant la pertinence globale du contexte fourni au **LLM**.

Principe général

Dans un système **RAG** classique, la recherche vectorielle retourne un ensemble de chunks similaires à une requête. Le filtrage multi-critères introduit une couche supplémentaire qui modifie cet ensemble en fonction de règles explicites basées sur les métadonnées.

Ces filtres peuvent être appliqués :

- avant le **retrieval** (pre-filtering)
- pendant la recherche (filtered **ANN**)
- après le **retrieval** (post-filtering)

L'objectif est de réduire l'espace de recherche à un sous-ensemble cohérent avec le contexte métier.

Types de filtres

Le filtrage multi-critères repose généralement sur plusieurs dimensions de contraintes.

Filtres temporels

Ils permettent de restreindre les résultats selon la date de création ou de mise à jour des documents. Cela est essentiel pour garantir la fraîcheur des informations dans des systèmes dynamiques.

Filtres thématiques

Ils limitent la recherche à des catégories ou domaines spécifiques (juridique, technique, produit, support client). Cela améliore la précision du **retrieval** en réduisant le bruit inter-domaines.

Filtres de source

Ils permettent de privilégier certaines sources de confiance, comme des documents validés, officiels ou internes.

Filtres structurels

Ils ciblent des sections spécifiques de documents, comme des résumés, des conclusions ou des parties techniques.

Interaction avec la recherche vectorielle

Le filtrage multi-critères modifie profondément le comportement de la recherche vectorielle. En réduisant l'espace de recherche, il améliore généralement la précision et la vitesse, mais peut aussi réduire le recall si les filtres sont trop restrictifs.

Dans les systèmes **ANN**, certains filtres sont appliqués directement au niveau de l'index, tandis que d'autres sont appliqués après la récupération des candidats. Cette distinction est importante pour optimiser les performances.

Filtrage et recherche hybride

Dans les architectures hybrides (dense + sparse), le filtrage multi-critères est appliqué de manière cohérente sur les deux pipelines. Cela garantit que les résultats fusionnés respectent les mêmes contraintes métier.

Cependant, certaines implémentations avancées appliquent des filtres différenciés selon le type de recherche, afin d'exploiter les forces spécifiques de chaque approche.

Impact sur le reranking

Le filtrage multi-critères influence directement le **reranking**. En réduisant le bruit en amont, il permet au modèle de reranking de se concentrer sur des candidats plus homogènes et plus pertinents.

Dans certains cas, les filtres eux-mêmes sont intégrés comme features dans le modèle de **reranking**, permettant une prise en compte implicite des contraintes métier dans le scoring final.

Compromis précision / couverture

Comme toute technique d'optimisation du **retrieval**, le filtrage multi-critères implique un compromis :

- des filtres stricts améliorent la précision mais réduisent le recall
- des filtres larges améliorent la couverture mais introduisent du bruit

Le réglage de ces paramètres dépend fortement du cas d'usage et des exigences métier du système **RAG**.

Cas d'usage industriels

Le filtrage multi-critères est particulièrement utilisé dans :

- les systèmes juridiques (filtrage par juridiction ou date)
- les assistants techniques (filtrage par version produit)
- les bases documentaires d'entreprise (filtrage par département)
- les systèmes support client (filtrage par langue ou région)

Dans ces contextes, il est souvent indispensable pour garantir la pertinence des réponses générées.

Conclusion

Le filtrage multi-critères est une composante essentielle des systèmes **RAG** en production. Il permet de combiner recherche sémantique et contraintes métier, assurant ainsi que les résultats du **retrieval** soient non seulement pertinents, mais également contextuellement valides.

En structurant l'accès à l'information selon plusieurs dimensions, il renforce la robustesse et la fiabilité globale du **pipeline RAG**.

Section 10.3 — Multi-tenancy

Section 10.3 — Multi-tenancy

Le multi-tenancy (ou architecture multi-locataire) désigne la capacité d'un système **RAG** à servir plusieurs utilisateurs, équipes ou organisations à partir d'une infrastructure partagée, tout en garantissant une isolation stricte des données et des performances.

Dans un contexte industriel, cette problématique devient centrale dès que le système dépasse un usage individuel ou un simple prototype. Elle touche directement à la sécurité, à la **scalabilité** et à la gouvernance des données.

Principe du multi-tenancy

Un système **multi-tenant** repose sur une idée simple : une seule infrastructure logique (index vectoriel, base de données, **pipeline de retrieval**), mais plusieurs espaces de données isolés.

Chaque tenant peut représenter :

- une entreprise différente
- un département interne
- un client SaaS
- un projet ou espace documentaire

L'objectif est de mutualiser les ressources tout en empêchant tout accès croisé non autorisé entre les données des différents tenants.

Isolation des données

L'isolation est le point critique du multi-tenancy. Elle peut être mise en œuvre à plusieurs niveaux :

Isolation logique

Tous les documents sont stockés dans la même base, mais chaque chunk est associé à un identifiant de tenant. Les filtres garantissent que seules les données autorisées sont accessibles.

Isolation physique

Chaque tenant dispose de sa propre base vectorielle ou de ses propres index. Cette approche offre une meilleure isolation mais augmente les coûts d'infrastructure.

Isolation hybride

Combinaison des deux approches : mutualisation partielle avec séparation des index ou des partitions.

Impact sur le retrieval

Le multi-tenancy influence directement le **pipeline** de **retrieval**. Chaque requête doit être automatiquement contextualisée avec le tenant correspondant afin d'appliquer des filtres stricts dès la phase de recherche.

Cela implique :

- filtrage systématique par tenant_id
- adaptation des index **ANN** pour supporter les partitions
- gestion de la cohérence des résultats dans des environnements multi-utilisateurs

Sans ces mécanismes, le système peut exposer des informations sensibles entre tenants, ce qui constitue une faille critique.

Scalabilité et performance

Le multi-tenancy introduit des contraintes supplémentaires sur la **scalabilité** du système. Plus le nombre de tenants augmente, plus la gestion des index et des partitions devient complexe.

Les principaux défis sont :

- équilibrage de charge entre tenants
- gestion des “hot tenants” avec forte activité
- optimisation de la mémoire et du stockage
- maintien de performances constantes malgré la croissance du nombre d'utilisateurs

Les bases vectorielles modernes comme Milvus, **Qdrant** ou Weaviate proposent des mécanismes natifs de partitionnement pour répondre à ces contraintes.

Sécurité et gouvernance

Dans un environnement **multi-tenant**, la sécurité devient un enjeu majeur. Il est nécessaire de garantir que :

- chaque requête est correctement authentifiée
- les filtres d'accès sont appliqués de manière systématique
- les logs ne contiennent pas de fuites inter-tenants
- les **embeddings** ne peuvent pas être exploités pour reconstruire des données sensibles

La gouvernance des données devient ainsi une couche essentielle du système **RAG**.

Rôle dans les architectures SaaS RAG

Le multi-tenancy est particulièrement important dans les produits **RAG** de type SaaS. Il permet de proposer un service unique à plusieurs clients tout en maintenant une isolation logique et contractuelle.

Cela permet :

- une réduction des coûts d'infrastructure
- une montée en charge plus efficace
- une maintenance centralisée du système
- une personnalisation par tenant

Conclusion

Le multi-tenancy est une composante fondamentale des systèmes **RAG** industriels modernes. Il transforme une architecture de recherche documentaire en une plateforme scalable, sécurisée et mutualisée, capable de servir plusieurs utilisateurs ou organisations sans compromis sur l'isolation des données.

Sa mise en œuvre correcte est indispensable pour tout déploiement **RAG** à grande échelle en environnement entreprise ou SaaS.

Section 10.4 — Data governance

Section 10.4 — Data governance

La data governance dans un système **RAG** désigne l'ensemble des règles, mécanismes et processus qui encadrent la gestion, la qualité, la sécurité et l'utilisation des données tout au long du **pipeline** : ingestion, **chunking**, **embedding**, indexation, **retrieval** et génération.

Dans un contexte industriel, elle ne se limite pas à une bonne pratique organisationnelle. Elle devient une contrainte structurelle du système, au même titre que la latence ou la **scalabilité**.

Pourquoi la data governance est critique en RAG

Un système **RAG** manipule en continu des données sensibles et dynamiques. Contrairement à un modèle statique, il dépend directement de la qualité et de la fraîcheur de ses sources.

Sans gouvernance solide, plusieurs risques apparaissent :

- propagation de données obsolètes dans les réponses
- incohérences entre versions de documents
- fuite d'informations sensibles entre utilisateurs ou tenants
- impossibilité d'auditer une réponse générée par le **LLM**

La data governance devient donc un mécanisme de contrôle global du **pipeline**.

Traçabilité des données

La traçabilité consiste à pouvoir remonter, pour chaque réponse générée, aux éléments précis qui l'ont produite.

Dans un système **RAG** bien conçu, cela implique :

- identification unique des documents et chunks
- suivi des versions de documents utilisés
- enregistrement des **embeddings** associés
- conservation des résultats de **retrieval** et **reranking**

Cette traçabilité est essentielle pour les audits, le debugging et la conformité réglementaire.

Qualité et validation des données

La gouvernance impose également des règles de qualité sur les données ingérées dans le système.

Cela peut inclure :

- validation des formats de documents
- détection des contenus dupliqués ou corrompus
- scoring de qualité des sources
- filtrage des données non fiables ou non validées

Un système **RAG** performant ne dépend pas uniquement de bons modèles, mais aussi de données correctement filtrées en amont.

Gestion des versions et cohérence

La data governance est fortement liée au versioning des documents et des **embeddings**. Elle garantit que :

- les réponses sont basées sur des versions cohérentes de documents

- les index vectoriels sont synchronisés avec les sources
- les mises à jour ne créent pas d'incohérences temporaires

Sans cette cohérence, le système peut produire des réponses contradictoires selon le moment ou la requête.

Contrôle d'accès et conformité

Dans un contexte entreprise, la data governance inclut également la gestion des droits d'accès.

Cela implique :

- contrôle des permissions par utilisateur ou groupe
- filtrage des données sensibles avant **retrieval**
- conformité avec les réglementations (**RGPD**, politiques internes)
- audit des accès aux documents

Ces règles doivent être appliquées de manière systématique dans tout le **pipeline RAG**, et pas uniquement au niveau de l'interface utilisateur.

Observabilité et monitoring

La gouvernance des données s'appuie sur des mécanismes d'observabilité permettant de surveiller le système en continu :

- logs de requêtes et de **retrieval**
- métriques de qualité des résultats
- suivi des distributions de données
- détection de dérive (data drift)

Ces éléments permettent d'anticiper les dégradations de performance et d'assurer un contrôle continu du système.

Gouvernance et lifecycle des données

Dans un **RAG** industriel, les données suivent un cycle de vie complet :

- ingestion
- transformation
- indexation
- utilisation
- mise à jour ou suppression

La data governance définit les règles qui régissent chaque étape de ce cycle. Elle assure que les données restent cohérentes, exploitables et conformes dans le temps.

Conclusion

La data governance est une couche invisible mais essentielle des systèmes **RAG** en production. Elle garantit que les données utilisées par le **LLM** sont fiables, traçables et conformes aux exigences métier et réglementaires.

Sans elle, même un système techniquement performant peut devenir instable, incohérent ou non conforme. Avec elle, le **RAG** devient une véritable infrastructure de connaissance maîtrisée et exploitable à grande échelle.

Partie III — Architecture backend production

Chapitre 11 — Architecture logicielle d'un RAG scalable

Section 11.1 — Microservices vs monolithe

Le choix entre une architecture monolithique et une architecture en microservices constitue une décision structurante dans la conception d'un système **RAG** en production. Cette décision influence directement la **scalabilité**, la maintenabilité, la latence ainsi que la complexité opérationnelle du système.

Dans un contexte **RAG**, ce choix ne doit pas être considéré comme une préférence technique abstraite. Il détermine concrètement la manière dont les différentes briques du système, telles que l'ingestion, le **retrieval**, le **reranking** et la génération, vont interagir et évoluer dans le temps.

Architecture monolithique

Une architecture monolithique consiste à regrouper l'ensemble des composants du système **RAG** dans une seule application déployée. Dans cette approche, toutes les étapes du **pipeline**, depuis l'ingestion des documents jusqu'à la génération de la réponse, sont exécutées au sein d'un même service.

Dans un monolithe **RAG**, l'ingestion des documents, le **chunking**, la génération des **embeddings**, l'indexation vectorielle, le **retrieval**, le **reranking** et l'appel au **LLM** sont généralement intégrés dans une seule base de code et un seul processus de déploiement.

Avantages du monolithe

Une architecture monolithique présente plusieurs avantages, en particulier dans les phases initiales de développement d'un système **RAG**.

Elle permet d'abord une grande simplicité de développement, car l'ensemble du système est centralisé dans un seul projet. Cela facilite la compréhension globale du code et réduit la complexité des interactions entre composants.

Elle offre également une latence plus faible, car les communications entre les différentes étapes du **pipeline** s'effectuent en mémoire ou de manière directe, sans passage par le réseau.

Enfin, elle facilite le debugging, car l'ensemble du flux de données est localisé dans une seule application, ce qui permet de tracer plus facilement les erreurs et les comportements du système.

Limites du monolithe

Cependant, cette approche atteint rapidement ses limites dans un contexte industriel.

Le principal problème est le couplage fort entre les différents composants du système. Toute modification d'un élément, comme le **reranking** ou l'indexation vectorielle, peut impacter l'ensemble du système.

La **scalabilité** devient également un problème, car il est impossible de scaler indépendamment les composants les plus coûteux, comme le **reranking** ou la génération d'**embeddings**.

De plus, le monolithe devient difficile à maintenir à mesure que la complexité du système augmente, notamment lorsque plusieurs équipes interviennent sur différentes parties du code.

Architecture en microservices

Une architecture en microservices consiste à découper le système **RAG** en plusieurs services indépendants. Chaque service est responsable d'une fonctionnalité spécifique du **pipeline**.

Dans une architecture **RAG** typique, on peut trouver un service d'ingestion, un service de **retrieval**, un service de **reranking**, un service d'**embeddings**, un service de base vectorielle et un service de génération **LLM**.

Ces services communiquent entre eux via des appels réseau, généralement en HTTP ou en gRPC, ou via des systèmes de files de messages.

Avantages des microservices

Les microservices apportent une grande flexibilité architecturale. Ils permettent de scaler indépendamment chaque composant du système en fonction de sa charge spécifique.

Par exemple, le service de **reranking** peut être déployé sur des GPU et scalé séparément du service d'API, qui peut rester léger et stateless.

Cette approche améliore également la résilience du système, car la défaillance d'un service n'entraîne pas nécessairement l'arrêt complet de la plateforme.

Enfin, elle permet une meilleure spécialisation des composants, ce qui facilite l'optimisation individuelle de chaque brique du **pipeline RAG**.

Inconvénients des microservices

Cependant, les microservices introduisent une complexité supplémentaire importante.

La communication entre services passe par le réseau, ce qui augmente la latence globale du système. Cette latence peut devenir critique dans des pipelines **RAG** où plusieurs étapes doivent être exécutées en séquence.

L'orchestration des services devient également plus complexe, car il faut gérer la coordination entre ingestion, **retrieval**, **reranking** et génération.

Enfin, la maintenance de l'infrastructure est plus lourde, car elle nécessite souvent des outils comme Kubernetes, des systèmes de monitoring avancés et une gestion rigoureuse des versions.

Architecture hybride

Dans les systèmes **RAG** industriels, une approche hybride est souvent utilisée. Dans cette approche, certaines parties du système sont regroupées dans un monolithe logique, tandis que les composants les plus coûteux ou les plus scalables sont isolés en microservices.

Par exemple, l'API et l'orchestration du **retrieval** peuvent rester dans un monolithe, tandis que le **reranking** ou la génération d'**embeddings** sont externalisés dans des services spécialisés.

Cette approche permet de combiner la simplicité du monolithe avec la **scalabilité** des microservices.

Conclusion

Le choix entre monolithe et microservices dépend fortement du stade de maturité du projet **RAG**. Le monolithe est souvent privilégié pour les prototypes et les premières versions du produit, tandis que les microservices deviennent nécessaires pour les systèmes à grande échelle.

Dans la pratique, les architectures **RAG** modernes adoptent fréquemment une approche hybride afin de trouver un équilibre entre simplicité, performance et **scalabilité**.

Section 11.2 — Event-driven ingestion

L'ingestion event-driven (ou ingestion orientée événements) est une approche architecturale dans laquelle l'ajout, la modification ou la suppression de documents déclenche automatiquement une chaîne de traitements asynchrones. Dans un système **RAG** industriel, cette approche remplace progressivement les pipelines d'ingestion synchrones, car elle permet de mieux gérer la **scalabilité**, la fraîcheur des données et la résilience du système.

Principe général

Dans une architecture event-driven, chaque action sur les données est considérée comme un événement. Lorsqu'un document est ajouté, modifié ou supprimé, le système génère un événement correspondant qui est publié dans une file de messages ou un broker.

Ces événements sont ensuite consommés par différents services spécialisés, qui exécutent chacun une étape du **pipeline** d'ingestion. Le traitement n'est plus centralisé, mais distribué et découplé.

Chaîne d'ingestion classique

Dans un système **RAG** event-driven, le flux d'ingestion suit généralement une séquence de traitements asynchrones :

Un document est d'abord reçu par un service d'ingestion, qui publie un événement indiquant qu'un nouveau contenu doit être traité. Ce message est ensuite consommé par un service de parsing, qui extrait le texte brut et normalise le contenu. Une fois cette étape terminée, un nouvel événement est généré pour déclencher le **chunking** et la génération des **embeddings**. Enfin, un dernier service indexe les vecteurs dans la base de données vectorielle.

Chaque étape produit donc un événement en sortie, ce qui permet de chaîner l'ensemble du **pipeline** sans couplage direct entre les services.

Avantages de l'approche event-driven

L'architecture event-driven apporte plusieurs avantages majeurs dans un système **RAG** à grande échelle.

Elle permet d'abord une meilleure **scalabilité**, car chaque étape du **pipeline** peut être exécutée indépendamment et parallélisée. Les services peuvent être dupliqués horizontalement en fonction de la charge de chaque étape.

Elle améliore également la résilience du système, car une panne sur un service ne bloque pas l'ensemble du **pipeline**. Les événements peuvent être mis en file d'attente et traités ultérieurement lorsque le service est de nouveau disponible.

Enfin, elle facilite l'ajout de nouvelles fonctionnalités, car il suffit d'ajouter un nouveau consommateur d'événements sans modifier le reste du **pipeline**.

Gestion de la cohérence des données

L'un des défis majeurs de l'ingestion event-driven est la gestion de la cohérence des données. Comme les traitements sont asynchrones, il peut exister un décalage temporel entre l'ajout d'un document et son indexation complète dans la base vectorielle.

Pour résoudre ce problème, les systèmes **RAG** industriels utilisent souvent des mécanismes de versioning et d'idempotence. Chaque événement est associé à un identifiant unique, ce qui permet d'éviter les duplications ou les traitements incohérents.

Impact sur la latence et la fraîcheur

L'ingestion event-driven améliore la **scalabilité** globale du système, mais introduit une latence d'ingestion variable. Un document n'est pas immédiatement disponible dans le **retrieval**, car il doit traverser l'ensemble du **pipeline** asynchrone.

Cependant, cette latence est généralement acceptable dans les systèmes **RAG**, car la fraîcheur des données n'est pas toujours critique à la milliseconde près. Dans les cas où la fraîcheur est essentielle, des mécanismes hybrides peuvent être mis en place pour traiter certaines requêtes en mode synchrone.

Intégration avec les systèmes de message

L'ingestion event-driven repose généralement sur des systèmes de messagerie ou de streaming de données. Ces systèmes permettent de gérer le flux d'événements et d'assurer leur livraison fiable entre les différents composants.

Ils jouent un rôle central dans la coordination du **pipeline RAG**, car ils garantissent que chaque étape reçoit les données nécessaires dans le bon ordre et sans perte d'information.

Conclusion

L'ingestion event-driven constitue une approche fondamentale pour les systèmes **RAG** industriels à grande échelle. Elle permet de découpler les différentes étapes du **pipeline**, d'améliorer la **scalabilité** et de renforcer la résilience globale du système.

Bien qu'elle introduise une complexité supplémentaire en termes de cohérence et de latence, elle devient indispensable dès que le volume de documents et la fréquence de mise à jour augmentent.

Section 11.3 — Queue systems (Redis, Kafka)

Les systèmes de files de messages constituent une brique essentielle des architectures **RAG** modernes, en particulier lorsqu'elles reposent sur une ingestion event-driven et une exécution distribuée des traitements. Des solutions comme **Redis** (via Redis Streams ou listes) et **Kafka** sont couramment utilisées pour orchestrer les flux de données entre les différents services du **pipeline**.

Rôle des queues dans une architecture RAG

Dans un système **RAG** industriel, les queues servent à découpler les producteurs d'événements des consommateurs. Elles permettent de transformer un flux synchrone de traitement en une chaîne asynchrone, robuste et scalable.

Concrètement, les queues assurent la transmission des événements liés à :

- l'ingestion de documents
- le parsing et la normalisation
- le **chunking**
- la génération d'**embeddings**

- l'indexation vectorielle

Chaque étape consomme des messages, effectue un traitement, puis publie à son tour un nouvel événement.

Redis comme système de queue

Redis est souvent utilisé comme solution de queue simple et rapide dans les architectures **RAG** de petite à moyenne taille. Grâce à ses structures comme les listes ou Redis Streams, il permet de mettre en place rapidement un système de traitement asynchrone.

Redis est particulièrement adapté lorsque :

- le volume de données reste modéré
- la latence doit être très faible
- la simplicité d'infrastructure est prioritaire
- le système ne nécessite pas de garantie de streaming avancé

Cependant, **Redis** montre ses limites dans les environnements à très grande échelle, notamment en termes de persistance avancée et de gestion de flux massifs.

Kafka comme système de streaming distribué

Kafka est une solution beaucoup plus robuste et adaptée aux architectures **RAG** à grande échelle. Contrairement à **Redis**, Kafka est conçu pour gérer des flux de données continus avec une forte garantie de persistance et de **scalabilité**.

Dans un système **RAG**, Kafka permet de :

- gérer des volumes massifs d'événements d'ingestion
- garantir la durabilité des messages
- permettre la relecture des événements (replay)
- scaler horizontalement les consommateurs

Kafka est particulièrement adapté aux architectures event-driven complexes où plusieurs services consomment simultanément les mêmes événements.

Comparaison Redis vs Kafka

Redis et Kafka répondent à des besoins différents dans une architecture **RAG**.

Redis privilégie la simplicité et la faible latence. Il est souvent utilisé pour des pipelines légers ou des prototypes avancés.

Kafka privilégie la robustesse et la **scalabilité**. Il est utilisé dans les systèmes industriels où le volume de données et la criticité du **pipeline** nécessitent une infrastructure plus lourde.

Dans certains systèmes **RAG**, les deux sont même combinés : **Redis** pour les tâches rapides et Kafka pour les flux principaux d'ingestion.

Impact sur le pipeline RAG

L'introduction d'un système de queue transforme profondément l'architecture **RAG**. Elle permet de découpler complètement les étapes du **pipeline** et d'introduire une exécution parallèle massive.

Cela améliore :

- la **scalabilité** de l'ingestion
- la résilience du système
- la capacité à absorber des pics de charge
- la flexibilité des traitements asynchrones

En revanche, cela introduit également une complexité supplémentaire liée à la gestion des états, des erreurs et des délais de propagation.

Gestion des erreurs et fiabilité

Dans un système basé sur des queues, la gestion des erreurs devient un aspect central de l'architecture. Les messages peuvent être réessayés, redirigés vers des dead-letter queues ou rejoués en cas de défaillance.

Cette approche garantit que les documents ne sont pas perdus et que le **pipeline** peut être restauré après incident sans perte de cohérence globale.

Conclusion

Les systèmes de queues comme **Redis** et Kafka sont indispensables dans les architectures **RAG** modernes. Ils permettent de transformer un **pipeline** rigide en un système distribué, asynchrone et scalable.

Le choix entre **Redis** et Kafka dépend principalement du volume de données, des contraintes de latence et du niveau de robustesse attendu. Dans les systèmes **RAG** industriels, Kafka tend à s'imposer pour les architectures à grande échelle, tandis que Redis reste pertinent pour des implémentations plus légères ou hybrides.

Section 11.4 — Orchestration des pipelines

L'orchestration des pipelines dans un système **RAG** consiste à coordonner l'ensemble des étapes qui transforment une requête utilisateur ou un document brut en une réponse générée par un modèle de langage. Cette orchestration est un élément central de l'architecture, car elle garantit que chaque composant du système s'exécute dans le bon ordre, avec les bonnes dépendances et dans des conditions de performance acceptables.

Rôle de l'orchestrateur

L'orchestrateur agit comme un chef d'orchestre entre les différentes briques du système **RAG**. Il ne réalise pas directement les traitements, mais il contrôle leur enchaînement et leur exécution.

Dans un **pipeline RAG**, il est responsable de coordonner des étapes telles que l'ingestion, le **chunking**, la génération d'**embeddings**, la recherche vectorielle, le **reranking** et l'appel au modèle de langage. Il s'assure que chaque étape reçoit les données nécessaires et que les résultats sont correctement transmis à l'étape suivante.

Orchestration synchrone

Dans une approche synchrone, l'orchestrateur exécute les différentes étapes du **pipeline** de manière séquentielle. Chaque étape attend la fin de la précédente avant de commencer son traitement.

Cette approche est simple à comprendre et à implémenter, mais elle peut devenir limitante en termes de latence et de **scalabilité**. Elle est souvent utilisée dans les systèmes **RAG** de petite taille ou dans les phases initiales de développement.

Orchestration asynchrone

Dans une approche asynchrone, les différentes étapes du **pipeline** sont découplées et exécutées indépendamment. L'orchestrateur publie des événements ou des tâches dans des files de messages, et chaque service consomme ces événements à son propre rythme.

Cette approche permet une meilleure **scalabilité** et une meilleure tolérance aux pannes, mais elle introduit une complexité supplémentaire liée à la gestion de l'état et de la cohérence des données.

Orchestration basée sur des workflows

Dans les systèmes **RAG** industriels, l'orchestration est souvent implémentée sous forme de workflows structurés. Chaque workflow représente une séquence d'étapes définies à l'avance, avec des règles de transition entre chaque composant.

Ces workflows permettent de modéliser explicitement le **pipeline RAG**, en définissant par exemple un flux ingestion → parsing → **chunking** → **embedding** → indexation ou un flux requête →

retrieval → **reranking** → génération.

Gestion des dépendances

L'un des rôles critiques de l'orchestration est la gestion des dépendances entre les différentes étapes du **pipeline**. Certaines étapes ne peuvent pas s'exécuter tant que les précédentes ne sont pas terminées.

Par exemple, le **reranking** ne peut pas être exécuté tant que la phase de **retrieval** n'a pas produit un ensemble de candidats. De même, la génération du **LLM** dépend directement du résultat du reranking et de la construction du contexte final.

Gestion des erreurs et reprise

Dans un système **RAG** en production, l'orchestrateur doit également gérer les erreurs et les cas de défaillance. Lorsqu'une étape échoue, il doit être capable de relancer le traitement, de le mettre en attente ou de déclencher une stratégie de fallback.

Cette capacité est essentielle pour garantir la robustesse du système, en particulier dans des environnements distribués où les pannes partielles sont fréquentes.

Scalabilité de l'orchestration

L'orchestrateur doit être conçu pour fonctionner à grande échelle. Il doit pouvoir gérer simultanément un grand nombre de pipelines d'ingestion et de requêtes utilisateurs sans devenir un goulot d'étranglement.

Pour cela, il est souvent conçu comme un service stateless, capable d'être répliqué horizontalement, ou remplacé par des systèmes spécialisés de workflow management.

Conclusion

L'orchestration des pipelines est un élément fondamental des systèmes **RAG** industriels. Elle garantit la coordination correcte des différentes étapes du traitement, tout en permettant de gérer la complexité croissante des architectures distribuées.

Une bonne orchestration permet de transformer un ensemble de composants indépendants en un système cohérent, robuste et scalable, capable de fonctionner efficacement en production.

Chapitre 12 — API RAG avec FastAPI

Dans une architecture **RAG** en production, l'API constitue la couche d'entrée principale du système. Elle est responsable de l'exposition des fonctionnalités de recherche, d'ingestion et de génération de réponses aux utilisateurs ou aux systèmes externes. Cette couche doit être conçue avec une forte attention portée à la performance, à la sécurité et à la stabilité.

Section 12.1 — Design d'API production

Le design d'une API **RAG** en production ne consiste pas uniquement à exposer des endpoints HTTP. Il s'agit de concevoir une interface cohérente, scalable et robuste qui sert de point d'entrée unique vers l'ensemble du **pipeline** RAG.

Principes fondamentaux de conception

Une API **RAG** production-ready repose sur plusieurs principes essentiels. Elle doit être stateless afin de pouvoir être scalée horizontalement sans dépendance à une instance spécifique. Elle doit également être idempotente pour garantir que les requêtes répétées ne provoquent pas d'effets secondaires non contrôlés.

L'API doit aussi être conçue autour de contrats clairs, avec des schémas de données strictement définis. Cela permet de garantir la compatibilité entre les différents services du système **RAG** et de limiter les erreurs liées aux incohérences de format.

Séparation des endpoints

Dans un système **RAG** industriel, l'API est généralement structurée autour de plusieurs types d'endpoints. Un premier groupe est dédié aux requêtes utilisateur, comme la génération de réponses à partir d'une question. Un deuxième groupe est consacré à l'ingestion de documents et à la gestion du corpus. Un troisième groupe peut être dédié à l'administration, au monitoring ou à la gestion des utilisateurs.

Cette séparation permet de clarifier les responsabilités et de faciliter la maintenance du système.

Design du endpoint de requête RAG

Le endpoint principal d'un système **RAG** est généralement celui qui traite les requêtes utilisateur. Ce endpoint reçoit une question, déclenche le **pipeline** de **retrieval**, puis appelle le **reranking** et le modèle de langage pour générer une réponse.

Dans un design production, ce endpoint doit être optimisé pour minimiser la latence. Il doit également gérer les timeouts, les erreurs de **retrieval** et les cas où le **LLM** ne répond pas correctement.

Gestion des entrées et validations

Une API **RAG** doit valider strictement les entrées utilisateurs. Cela inclut la vérification du format des requêtes, la limitation de la taille des inputs et la protection contre les injections ou les abus.

Les schémas de validation permettent de garantir que seules des données propres et exploitables sont transmises au **pipeline** de **retrieval**.

Gestion des réponses

Les réponses générées par une API **RAG** ne se limitent pas au texte produit par le **LLM**. Elles incluent souvent des métadonnées supplémentaires, comme les sources utilisées, les scores de pertinence ou les chunks récupérés.

Cette transparence est essentielle pour les cas d'usage professionnels, car elle permet d'expliquer et de justifier les réponses générées.

Performance et latence

Le design de l'API doit prendre en compte les contraintes de latence du **pipeline RAG**. Chaque appel API déclenche potentiellement plusieurs étapes coûteuses, comme le **retrieval** vectoriel, le **reranking** et l'appel au **LLM**.

Pour réduire la latence globale, l'API doit être conçue pour supporter la parallélisation des appels, le caching des résultats fréquents et la limitation du volume de données traitées.

Sécurité et contrôle d'accès

Une API **RAG** en production doit intégrer des mécanismes de sécurité robustes. Cela inclut l'authentification des utilisateurs, la gestion des permissions et le filtrage des données sensibles.

Ces mécanismes doivent être appliqués dès la couche API afin d'éviter que des données non autorisées n'atteignent les étapes de **retrieval** ou de génération.

Extensibilité de l'API

Un bon design d'API **RAG** doit également anticiper l'évolution du système. De nouveaux endpoints peuvent être ajoutés pour supporter de nouvelles fonctionnalités comme le **reranking** avancé, les agents ou les workflows multi-étapes.

Cette extensibilité est essentielle pour permettre au système de s'adapter aux besoins futurs sans refonte complète de l'architecture.

Conclusion

Le design d'une API **RAG** en production est une étape critique qui conditionne la performance, la sécurité et la maintenabilité du système. Une API bien conçue permet de transformer un **pipeline** complexe en une interface simple, stable et exploitable par des applications externes.

Elle constitue la porte d'entrée essentielle entre les utilisateurs et l'ensemble de l'architecture **RAG**.

Section 12.2 — Endpoint /query

L'endpoint /query constitue le cœur fonctionnel d'une API **RAG** en production. C'est par cet endpoint que transitent la majorité des interactions utilisateur. Il est responsable de transformer une requête en langage naturel en une réponse générée par le système RAG, en orchestrant l'ensemble du **pipeline retrieval**, **reranking** et génération.

Rôle de l'endpoint /query

L'endpoint /query sert d'interface principale entre l'utilisateur et le système **RAG**. Il reçoit une question, déclenche le **pipeline** de recherche d'information, puis retourne une réponse enrichie par les documents récupérés.

Son rôle ne se limite pas à transmettre une requête au **LLM**. Il agit comme un orchestrateur léger qui coordonne les différentes briques du système tout en respectant les contraintes de latence, de sécurité et de qualité.

Structure d'une requête

Une requête envoyée à /query contient généralement plusieurs éléments. Elle inclut la question principale de l'utilisateur, mais peut également contenir des métadonnées comme l'identifiant utilisateur, le contexte de session ou les paramètres de recherche.

Ces informations permettent d'adapter le comportement du **pipeline RAG** en fonction du contexte d'utilisation et des contraintes métier.

Pipeline exécuté par /query

Lorsqu'une requête est reçue, l'endpoint déclenche un **pipeline** structuré en plusieurs étapes. La première étape consiste à transformer la question en **embedding** afin de permettre la recherche vectorielle. Ensuite, le système exécute une recherche hybride combinant signaux denses et lexicaux pour récupérer un ensemble de candidats.

Une fois les candidats récupérés, une étape de **reranking** est effectuée afin de réordonner les documents selon leur pertinence réelle. Le système sélectionne ensuite les meilleurs chunks pour construire un contexte optimisé.

Enfin, ce contexte est envoyé au modèle de langage, qui génère une réponse finale.

Gestion de la latence

L'endpoint /query est particulièrement sensible à la latence, car il est directement exposé à l'utilisateur final. Chaque étape du **pipeline** contribue au temps de réponse global, ce qui impose des optimisations strictes.

Le système doit minimiser le nombre de documents récupérés, optimiser les appels au **reranker** et réduire la taille du contexte envoyé au **LLM**. Des mécanismes de cache peuvent également être utilisés pour accélérer les requêtes fréquentes.

Gestion des erreurs

Un système **RAG** en production doit anticiper les défaillances possibles à chaque étape du **pipeline**. L'endpoint /query doit donc être capable de gérer les erreurs de **retrieval**, les timeouts du **reranking** ou les indisponibilités du modèle de langage.

Dans ces cas, des stratégies de fallback peuvent être mises en place, comme la réduction du **Top-K**, la désactivation du **reranking** ou l'utilisation d'un modèle **LLM** secondaire.

Structuration de la réponse

La réponse retournée par /query ne se limite pas à un texte brut. Elle inclut généralement plusieurs éléments structurés, comme la réponse générée, les sources utilisées, les scores de pertinence et parfois les chunks exacts ayant servi à la génération.

Cette structuration permet d'améliorer la transparence du système et de faciliter le debugging ainsi que l'audit des réponses.

Sécurité et contrôle d'accès

L'endpoint /query doit intégrer des mécanismes de sécurité robustes. Cela inclut l'authentification des utilisateurs, la validation des permissions d'accès aux documents et la protection contre les requêtes abusives.

Ces contrôles doivent être appliqués avant même l'exécution du **pipeline RAG** afin d'éviter tout accès non autorisé aux données sensibles.

Optimisation en production

En environnement industriel, l'endpoint `/query` est souvent optimisé via plusieurs techniques. Le batching de requêtes peut être utilisé pour améliorer le débit global. Le caching des résultats de **retrieval** ou de génération permet également de réduire les coûts et la latence.

Enfin, la surveillance des performances de cet endpoint est essentielle pour détecter rapidement toute dégradation du système.

Conclusion

L'endpoint `/query` est le point d'entrée critique d'un système **RAG** en production. Il orchestre l'ensemble du **pipeline** de **retrieval** et de génération tout en respectant des contraintes fortes de latence, de sécurité et de robustesse.

Sa conception influence directement la qualité perçue du système par les utilisateurs finaux.

Section 12.3 — Endpoint `/ingest`

L'endpoint `/ingest` est responsable de l'intégration des données dans un système **RAG** en production. Il constitue la porte d'entrée du flux documentaire et déclenche l'ensemble du **pipeline** d'ingestion, depuis la réception d'un document brut jusqu'à son indexation dans la base vectorielle.

Contrairement à l'endpoint `/query`, qui est centré sur la génération de réponses en temps réel, `/ingest` est orienté vers des traitements souvent plus lourds, asynchrones et orientés batch.

Rôle de l'endpoint `/ingest`

L'endpoint `/ingest` a pour objectif principal de transformer des documents non structurés en données exploitables par le système **RAG**. Il reçoit des fichiers ou des contenus textuels, puis déclenche les différentes étapes du **pipeline** d'ingestion.

Ces étapes incluent généralement le parsing du document, la normalisation du texte, le **chunking**, la génération d'**embeddings** et enfin l'indexation dans la base vectorielle.

Types de données ingérées

L'endpoint `/ingest` doit être capable de traiter plusieurs types de documents. Il peut s'agir de fichiers PDF, de documents Word, de pages HTML ou de fichiers texte bruts. Dans certains systèmes plus avancés, il peut également gérer des formats structurés comme JSON ou des données issues d'API externes.

Chaque type de document nécessite un traitement spécifique afin d'extraire correctement le contenu exploitable.

Pipeline d'ingestion déclenché

Lorsqu'un document est soumis à `/ingest`, le système déclenche une chaîne de traitement structurée. Le document est d'abord stocké ou référencé dans un système de stockage. Ensuite, il est analysé pour extraire le texte brut et supprimer les éléments non pertinents comme les balises, les métadonnées inutiles ou les artefacts de formatage.

Une fois le texte extrait, il est découpé en chunks selon une stratégie définie, puis chaque chunk est transformé en **embedding**. Ces **embeddings** sont ensuite indexés dans la base vectorielle avec leurs métadonnées associées.

Traitement synchrone vs asynchrone

Dans un système **RAG** en production, l'ingestion peut être traitée de manière synchrone ou asynchrone. Le traitement synchrone consiste à exécuter toutes les étapes immédiatement lors de l'appel à `/ingest`, ce qui permet une indexation rapide mais augmente fortement la latence de l'endpoint.

À l'inverse, le traitement asynchrone repose sur un système de queue ou d'événements, où l'endpoint `/ingest` se contente de valider et d'enregistrer le document avant de déléguer le traitement à des workers spécialisés. Cette approche est plus scalable et plus robuste.

Gestion des erreurs d'ingestion

L'endpoint `/ingest` doit être conçu pour gérer les erreurs de manière robuste. Les erreurs peuvent survenir lors du parsing du document, de la génération des **embeddings** ou de l'indexation.

Dans ces cas, le système doit être capable de réessayer automatiquement le traitement ou de placer le document dans une file d'erreurs afin d'éviter toute perte de données.

Métadonnées et enrichissement

Lors de l'ingestion, le système associe des métadonnées aux documents et aux chunks générés. Ces métadonnées incluent généralement des informations sur la source du document, sa date de création, sa catégorie et sa structure interne.

Ces informations sont essentielles pour permettre un filtrage efficace lors des phases de **retrieval** et pour améliorer la pertinence globale du système **RAG**.

Scalabilité de l'ingestion

L'endpoint `/ingest` doit être capable de gérer des volumes importants de documents sans devenir un goulot d'étranglement. Pour cela, il est souvent découplé du **pipeline** principal et repose sur des systèmes distribués capables de traiter les documents en parallèle.

Cette **scalabilité** est essentielle dans les systèmes **RAG** industriels où des milliers voire des millions de documents peuvent être ajoutés régulièrement.

Sécurité et validation

Comme pour les autres endpoints, `/ingest` doit intégrer des mécanismes de sécurité et de validation stricts. Cela inclut la vérification du type de fichier, la taille des documents et la conformité des données.

Ces contrôles permettent d'éviter l'injection de contenus malveillants ou corrompus dans le système **RAG**.

Conclusion

L'endpoint `/ingest` est un composant fondamental de l'architecture **RAG**. Il permet d'alimenter le système en données structurées et exploitables, tout en déclenchant les pipelines d'ingestion nécessaires à la construction de l'index vectoriel.

Sa conception influence directement la qualité des données disponibles pour le **retrieval** et, par conséquent, la performance globale du système **RAG**.

Section 12.4 — Gestion des erreurs et validation

La gestion des erreurs et la validation des données constituent une composante critique d'une API **RAG** en production. Dans un système aussi dépendant de chaînes de traitement complexes — ingestion, **embedding**, **retrieval**, **reranking** et génération — la moindre erreur non contrôlée peut se propager et dégrader fortement la qualité globale des réponses.

Cette couche ne sert donc pas uniquement à "éviter les bugs", mais à garantir la stabilité, la sécurité et la cohérence du système dans son ensemble.

Importance de la validation en entrée

La validation des entrées constitue la première barrière de protection d'un système **RAG**. Chaque requête utilisateur ou document ingéré doit être vérifié avant d'être injecté dans le **pipeline**.

Dans le cas de l'endpoint `/query`, cela implique de contrôler la structure de la question, la taille maximale autorisée et la conformité des paramètres optionnels. Dans le cas de `/ingest`, cela inclut la vérification du type de fichier, de son intégrité et de sa taille.

Une validation insuffisante peut entraîner des comportements imprévisibles, comme des **embeddings** incohérents ou des erreurs de parsing en cascade.

Typologie des erreurs dans un système RAG

Les erreurs dans une API **RAG** peuvent être classées en plusieurs catégories. Les erreurs de validation concernent les données mal formées ou non conformes aux schémas attendus. Les erreurs de traitement surviennent lors des étapes internes du **pipeline**, comme le parsing de documents ou la génération d'**embeddings**. Les erreurs de dépendances proviennent des services externes, tels que la base vectorielle ou le modèle de langage.

Enfin, les erreurs de performance apparaissent lorsque les délais de réponse dépassent les seuils acceptables en production.

Stratégies de gestion des erreurs

Une API **RAG** en production doit adopter des stratégies robustes pour gérer ces erreurs. Lorsqu'une erreur est détectée, le système peut choisir de réessayer automatiquement l'opération, de basculer vers un service de secours ou de dégrader la qualité de la réponse pour garantir une continuité de service.

Par exemple, si le **reranker** est indisponible, le système peut continuer avec les résultats bruts du **retrieval**. Si le **LLM** principal échoue, un modèle secondaire plus léger peut être utilisé.

Validation des schémas de données

La validation structurée des données est essentielle pour garantir la cohérence du système. Les APIs **RAG** utilisent généralement des schémas stricts pour définir les formats des requêtes et des réponses.

Ces schémas permettent de s'assurer que les données transmises entre les différents composants du **pipeline** respectent des règles précises, ce qui réduit considérablement le risque d'erreurs en cascade.

Gestion des timeouts et des latences

Dans un système **RAG**, les délais de traitement peuvent varier fortement selon la complexité de la requête. Il est donc nécessaire de définir des timeouts adaptés pour chaque étape du **pipeline**.

Lorsque ces timeouts sont dépassés, le système doit être capable d'interrompre proprement l'exécution et de retourner une réponse dégradée ou une erreur contrôlée. Cela permet d'éviter les blocages et de maintenir la stabilité globale de l'API.

Logging et observabilité des erreurs

La gestion des erreurs ne peut être efficace sans une observabilité complète du système. Chaque erreur doit être loggée avec un maximum de contexte, incluant la requête initiale, les étapes du **pipeline** concernées et les dépendances impliquées.

Ces logs permettent d'identifier rapidement les points de défaillance et d'améliorer continuellement la robustesse du système **RAG**.

Impact sur l'expérience utilisateur

La manière dont les erreurs sont gérées a un impact direct sur l'expérience utilisateur. Une API **RAG** bien conçue ne renvoie pas simplement des messages d'erreur techniques, mais des réponses contrôlées et compréhensibles.

Dans certains cas, le système peut même fournir une réponse partielle ou approximative plutôt qu'un échec complet, afin de maintenir une continuité de service.

Conclusion

La gestion des erreurs et la validation des données constituent une couche essentielle dans une API **RAG** en production. Elles permettent de garantir la robustesse du système, d'éviter les dégradations en cascade et d'assurer une expérience utilisateur stable même en cas de défaillance partielle.

Sans cette couche de contrôle, même une architecture techniquement performante peut devenir instable et imprévisible en environnement réel.

Chapitre 13 — Cache et optimisation

Dans un système **RAG** en production, les performances ne dépendent pas uniquement de la qualité des modèles ou de la structure du **pipeline**, mais aussi de la capacité à réduire les coûts de calcul et la latence. Les mécanismes de cache jouent ici un rôle fondamental, car ils permettent d'éviter de recalculer des opérations coûteuses à chaque requête.

Section 13.1 — Cache embeddings

Le cache d'**embeddings** consiste à stocker les représentations vectorielles déjà calculées afin d'éviter de les recalculer inutilement. Dans un système **RAG**, la génération d'embeddings est une opération coûteuse, surtout lorsque le volume de documents ou de requêtes est élevé. Le cache permet donc d'optimiser à la fois la latence et le coût computationnel.

Principe général

Le principe du cache d'**embeddings** repose sur une idée simple : si une entrée textuelle a déjà été transformée en vecteur, il est inutile de recalculer cet **embedding** une seconde fois. Le système conserve donc une correspondance entre le texte d'entrée et son embedding associé.

Cette correspondance peut être stockée dans une base de données rapide en mémoire, comme **Redis**, ou dans une structure persistante optimisée pour les lectures fréquentes.

Cache côté ingestion

Lors de l'ingestion des documents, le cache d'**embeddings** est particulièrement utile lorsque des contenus similaires ou identiques sont traités plusieurs fois. Dans ce cas, le système peut réutiliser les embeddings existants au lieu de relancer le modèle.

Cela permet de réduire significativement le temps d'indexation, en particulier dans les systèmes où les documents sont fréquemment mis à jour ou ré-ingérés.

Cache côté requête

Le cache peut également être utilisé lors des requêtes utilisateur. Si une question a déjà été posée, le système peut réutiliser l'**embedding** de la requête pour accélérer la phase de **retrieval**.

Dans certains systèmes avancés, le cache peut même stocker des résultats complets de **retrieval**, ce qui permet de répondre immédiatement à des requêtes fréquentes sans recalcul du **pipeline** complet.

Stratégies d'invalidation

Un aspect critique du cache d'**embeddings** est la gestion de l'invalidation. Lorsque les documents changent ou lorsque les modèles d'**embedding** sont mis à jour, les valeurs mises en cache peuvent devenir obsolètes.

Le système doit donc être capable de détecter ces changements et de régénérer les **embeddings** lorsque cela est nécessaire. Cela peut être réalisé via des versions de modèles, des timestamps ou des identifiants de contenu.

Compromis entre performance et fraîcheur

L'utilisation du cache introduit un compromis entre performance et fraîcheur des données. Plus le cache est agressif, plus les performances sont bonnes, mais plus le risque d'utiliser des **embeddings** obsolètes augmente.

Dans les systèmes **RAG** industriels, ce compromis est généralement géré via des politiques de cache configurables en fonction des besoins métier.

Impact sur la scalabilité

Le cache d'**embeddings** améliore fortement la **scalabilité** d'un système **RAG**. En réduisant le nombre de calculs redondants, il permet de supporter un plus grand volume de requêtes et de documents avec les mêmes ressources matérielles.

Il contribue également à lisser les pics de charge, en évitant les recalculs massifs lors des périodes d'activité élevée.

Conclusion

Le cache d'**embeddings** est un composant essentiel de l'optimisation d'un système **RAG** en production. Il permet de réduire les coûts, d'améliorer la latence et de rendre le système plus scalable.

Cependant, son efficacité dépend fortement de la stratégie d'invalidation et de la capacité du système à maintenir un équilibre entre performance et fraîcheur des données.

Section 13.2 — Cache retrieval

Le cache de **retrieval** est un mécanisme d'optimisation qui consiste à stocker les résultats de recherche d'un système **RAG** afin d'éviter de recalculer une requête identique ou fortement similaire. Dans un système en production, cette optimisation est essentielle, car la phase de retrieval combine des opérations coûteuses telles que la recherche vectorielle, la recherche lexicale et parfois le **reranking**.

Principe général

Le cache de **retrieval** repose sur l'idée que certaines requêtes utilisateur sont répétées ou très proches dans leur formulation. Lorsqu'une requête a déjà été traitée, le système peut réutiliser directement les résultats de recherche précédemment calculés.

Cela permet de court-circuiter les étapes du **pipeline** de **retrieval** et de fournir une réponse beaucoup plus rapide, sans relancer la recherche dans la base vectorielle.

Cache basé sur les requêtes exactes

La forme la plus simple de cache de **retrieval** consiste à associer une requête exacte à un ensemble de résultats. Dans ce cas, le système vérifie si la requête a déjà été traitée avant d'exécuter le **pipeline** complet.

Cette approche est très efficace pour les systèmes où les utilisateurs posent fréquemment les mêmes questions, mais elle est limitée dès que les formulations varient légèrement.

Cache basé sur la similarité sémantique

Dans des systèmes plus avancés, le cache de **retrieval** ne repose pas uniquement sur des correspondances exactes, mais sur la similarité sémantique entre requêtes. Les **embeddings** des questions sont utilisés pour identifier des requêtes proches dans l'espace vectoriel.

Lorsqu'une nouvelle requête est suffisamment similaire à une requête déjà traitée, le système peut réutiliser les résultats de **retrieval** associés.

Cette approche est plus flexible, mais elle introduit une complexité supplémentaire dans la gestion du seuil de similarité.

Cache et pipeline hybride

Dans un système **RAG** hybride combinant recherche vectorielle et recherche lexicale, le cache de **retrieval** doit prendre en compte l'ensemble du **pipeline**. Les résultats stockés incluent généralement les documents candidats, les scores de similarité et parfois les résultats de **reranking**.

Cela permet de réutiliser une partie importante du **pipeline** sans recalculer chaque étape.

Gestion de la cohérence des résultats

Un défi majeur du cache de **retrieval** est la cohérence des résultats dans le temps. Les documents indexés peuvent évoluer, de nouveaux contenus peuvent être ajoutés et les modèles d'**embedding** peuvent être mis à jour.

Pour éviter d'utiliser des résultats obsolètes, le cache doit être invalidé en fonction de la version de l'index, de la version des **embeddings** ou des mises à jour du corpus.

Impact sur la latence

Le cache de **retrieval** a un impact direct sur la latence du système **RAG**. Lorsqu'une requête est servie depuis le cache, le temps de réponse est réduit de manière significative, car le système évite les étapes coûteuses de recherche et de **reranking**.

Cela améliore fortement l'expérience utilisateur, en particulier pour les requêtes fréquentes ou répétitives.

Compromis et limites

L'utilisation du cache de **retrieval** implique un compromis entre performance et fraîcheur des résultats. Un cache trop agressif peut renvoyer des résultats obsolètes ou non optimisés par rapport à l'état actuel de l'index.

Il est donc nécessaire de définir des politiques de cache adaptées au contexte métier et au niveau de criticité des données.

Conclusion

Le cache de **retrieval** est un levier d'optimisation majeur dans les systèmes **RAG** en production. Il permet de réduire la latence, d'améliorer la **scalabilité** et de diminuer la charge sur les bases vectorielles et les systèmes de **reranking**.

Lorsqu'il est correctement conçu et correctement invalidé, il constitue un élément clé de performance dans les architectures **RAG** industrielles.

Section 13.3 — Réduction des coûts LLM

La réduction des coûts liés aux modèles de langage constitue un enjeu central dans tout système **RAG** en production. Les **LLM** représentent souvent la partie la plus coûteuse du **pipeline**, à la fois en termes de latence et de consommation de ressources. Optimiser leur utilisation ne consiste donc pas seulement à réduire les dépenses, mais aussi à améliorer la **scalabilité** globale du système.

Nature des coûts LLM

Les coûts associés aux **LLM** proviennent principalement de trois facteurs. Le premier est le nombre d'appels au modèle, qui augmente avec le volume de requêtes utilisateur. Le second est la taille du contexte envoyé au modèle, car plus le prompt est long, plus le coût de calcul est élevé. Le troisième est la complexité du modèle lui-même, certains modèles étant beaucoup plus coûteux que d'autres pour une qualité de sortie comparable.

Réduction du nombre d'appels

Une première stratégie consiste à réduire le nombre d'appels au **LLM**. Cela peut être réalisé en introduisant des mécanismes de cache de réponses, en regroupant plusieurs requêtes similaires ou en évitant les appels inutiles lorsque le **retrieval** ne retourne pas suffisamment de contexte pertinent.

Dans certains systèmes, une étape de filtrage préalable permet également de déterminer si une requête nécessite réellement une génération par **LLM** ou si une réponse simple peut être fournie à partir des résultats du **retrieval**.

Optimisation du contexte

Une autre stratégie importante consiste à réduire la taille du contexte envoyé au modèle. Le coût d'un **LLM** est directement proportionnel au nombre de tokens en entrée, ce qui rend essentiel un contrôle strict de la quantité d'information injectée.

Cela passe par une sélection plus agressive des chunks, une limitation du **Top-K** après **reranking** et une suppression des informations redondantes. L'objectif est de fournir uniquement les éléments strictement nécessaires à la génération de la réponse.

Utilisation de modèles hiérarchiques

Les systèmes **RAG** industriels utilisent souvent une approche hiérarchique des modèles. Un modèle léger est utilisé pour les requêtes simples ou pour filtrer les cas où un **LLM** plus puissant n'est pas nécessaire. Le modèle principal n'est sollicité que pour les cas complexes nécessitant une compréhension approfondie.

Cette stratégie permet de réserver les ressources les plus coûteuses aux cas où elles apportent une réelle valeur ajoutée.

Compression et summarization du contexte

Une autre technique consiste à compresser le contexte avant de l'envoyer au **LLM**. Cela peut être réalisé par des modèles de résumé ou par des techniques de sélection intelligente des passages les plus pertinents.

Cette compression permet de réduire la taille du prompt tout en conservant l'essentiel de l'information nécessaire à la génération de la réponse.

Batch et mutualisation des requêtes

Dans certains environnements à forte charge, les requêtes **LLM** peuvent être regroupées en batchs afin d'optimiser l'utilisation des ressources. Cette mutualisation permet de réduire le coût moyen par requête et d'améliorer le débit global du système.

Cependant, cette approche est plus difficile à mettre en œuvre dans des systèmes interactifs nécessitant des réponses en temps réel.

Choix des modèles

Le choix du modèle de langage a également un impact direct sur les coûts. Les systèmes **RAG** en production utilisent souvent une combinaison de modèles, allant de modèles légers pour les tâches simples à des modèles plus puissants pour les cas complexes.

Ce choix permet d'adapter le coût de calcul au niveau de difficulté de la requête.

Conclusion

La réduction des coûts **LLM** est un élément essentiel de l'optimisation des systèmes **RAG** en production. Elle repose sur une combinaison de stratégies incluant la réduction du nombre d'appels, l'optimisation du contexte, l'utilisation de modèles hiérarchiques et la compression de l'information.

Une bonne optimisation permet de rendre un système **RAG** non seulement plus économique, mais aussi plus rapide et plus scalable, sans dégrader significativement la qualité des réponses.

Section 13.4 — Latence optimisée

La latence est l'un des critères les plus critiques dans un système **RAG** en production. Elle correspond au temps total nécessaire pour transformer une requête utilisateur en réponse générée par le modèle de langage. Dans un contexte industriel, une latence élevée dégrade directement l'expérience utilisateur et limite la capacité du système à être utilisé à grande échelle.

Décomposition de la latence

La latence d'un système **RAG** n'est pas un bloc unique, mais la somme de plusieurs étapes successives. Elle inclut le temps de transformation de la requête en **embedding**, le temps de recherche dans la base vectorielle, le temps de **reranking** des documents, ainsi que le temps de génération de la réponse par le **LLM**.

Chaque étape contribue différemment au temps total, et certaines deviennent des goulots d'étranglement plus importants que d'autres selon la charge et la complexité du système.

Optimisation du retrieval

Une première source importante de latence se situe au niveau du **retrieval**. Pour optimiser cette étape, il est nécessaire de réduire le volume de recherche en limitant le **Top-K** initial et en optimisant les structures d'index comme **HNSW** ou **IVF**.

L'utilisation de filtres efficaces sur les métadonnées permet également de réduire l'espace de recherche et donc le temps d'exécution.

Parallélisation des étapes

Une stratégie clé pour réduire la latence globale consiste à paralléliser certaines étapes du **pipeline**. Par exemple, la recherche dense et la recherche lexicale peuvent être exécutées simultanément dans une architecture hybride.

De même, certaines opérations comme la préparation du contexte peuvent être réalisées en parallèle du **reranking** afin de réduire le temps global avant l'appel au **LLM**.

Optimisation du reranking

Le **reranking** est souvent l'une des étapes les plus coûteuses du **pipeline RAG**. Pour réduire son impact sur la latence, il est courant de limiter le nombre de candidats envoyés au modèle de reranking.

L'utilisation de modèles plus légers ou distillés permet également de réduire significativement le temps de calcul, tout en conservant une qualité acceptable.

Réduction du temps LLM

Le temps de génération du **LLM** représente souvent la plus grande partie de la latence totale. Il peut être optimisé en réduisant la taille du prompt, en limitant le nombre de tokens générés ou en utilisant des modèles plus rapides.

Dans certains systèmes, des techniques de streaming sont utilisées pour envoyer progressivement la réponse à l'utilisateur, améliorant ainsi la perception de latence.

Cache et pré-calcul

Les mécanismes de cache jouent un rôle important dans la réduction de la latence. Le cache de **retrieval** permet d'éviter la recherche vectorielle pour les requêtes fréquentes, tandis que le cache de réponses permet d'éviter complètement l'appel au **LLM**.

Des pré-calculs peuvent également être utilisés pour anticiper certaines requêtes ou pré-indexer des résultats fréquemment demandés.

Optimisation de l'infrastructure

La latence dépend également fortement de l'infrastructure sous-jacente. L'utilisation de bases vectorielles optimisées, de GPU pour les modèles de **reranking** et de serveurs proches des utilisateurs permet de réduire les temps de réponse réseau et computationnels.

L'architecture distribuée doit être conçue pour minimiser les communications inter-services, qui peuvent ajouter une latence non négligeable.

Conclusion

L'optimisation de la latence dans un système **RAG** est un problème multi-factoriel qui implique à la fois le design algorithmique et l'architecture système. Elle repose sur la réduction des coûts de chaque étape du **pipeline**, la parallélisation des traitements et l'utilisation intelligente du cache.

Une latence bien maîtrisée est essentielle pour garantir une expérience utilisateur fluide et permettre le déploiement du système à grande échelle en production.

Chapitre 14 — Conteneurisation et déploiement

Le passage d'un système **RAG** à un environnement de production nécessite une standardisation forte de l'exécution des services. La conteneurisation, et en particulier **Docker**, joue ici un rôle central, car elle permet de garantir que chaque composant du système s'exécute de manière reproductible, isolée et portable, indépendamment de l'environnement d'hébergement.

Section 14.1 — Dockerisation complète

La dockerisation complète d'un système **RAG** consiste à encapsuler l'ensemble des services du **pipeline** dans des conteneurs **Docker**. Cela inclut l'API, les services d'ingestion, le **retrieval**, le **reranking**, ainsi que les dépendances comme la base vectorielle et les systèmes de cache.

Principe de la conteneurisation

Un conteneur **Docker** est une unité d'exécution qui encapsule une application avec toutes ses dépendances. Dans un système **RAG**, chaque brique du **pipeline** peut être isolée dans un conteneur distinct afin de garantir une exécution cohérente entre les environnements de développement, de test et de production.

Cette isolation permet d'éviter les problèmes liés aux différences de configuration système, de versions de bibliothèques ou de dépendances externes.

Découpage des conteneurs

Dans une architecture **RAG** industrialisée, la dockerisation ne se limite pas à une seule image monolithique. Le système est généralement découpé en plusieurs conteneurs spécialisés.

L'API **FastAPI** est souvent exécutée dans un conteneur dédié, responsable de l'exposition des endpoints `/query` et `/ingest`. Le service de **retrieval** peut être isolé dans un autre conteneur, tout comme le service de **reranking**, qui peut nécessiter des ressources GPU spécifiques.

Les bases de données vectorielles comme Milvus ou **Qdrant** sont également déployées dans des conteneurs séparés afin de garantir leur indépendance et leur **scalabilité**.

Orchestration des conteneurs

Une fois les conteneurs définis, ils doivent être orchestrés pour fonctionner ensemble comme un système cohérent. **Docker Compose** est souvent utilisé pour les environnements de développement ou de test, tandis que Kubernetes est privilégié pour les environnements de production à grande échelle.

L'orchestration permet de gérer les dépendances entre services, les réseaux internes et les politiques de redémarrage automatique en cas de défaillance.

Gestion des dépendances

Chaque conteneur doit être construit avec un contrôle strict des dépendances. Cela inclut la définition explicite des versions des bibliothèques Python, des frameworks utilisés et des drivers nécessaires pour l'exécution des modèles.

Cette rigueur permet d'assurer la reproductibilité du système **RAG** sur différents environnements et de limiter les erreurs liées aux incompatibilités logicielles.

Optimisation des images Docker

Dans un système **RAG** en production, la taille et la performance des images **Docker** sont des facteurs importants. Des images trop volumineuses peuvent ralentir le déploiement et augmenter les coûts d'infrastructure.

Il est donc courant d'utiliser des images minimalistes, basées sur des distributions légères, et de supprimer les dépendances inutiles. Le multi-stage build est également utilisé pour séparer la phase de construction de la phase d'exécution.

Gestion des ressources

La dockerisation permet également de contrôler précisément les ressources allouées à chaque composant du système. Il est possible de définir des limites de CPU, de mémoire et d'accès GPU pour chaque conteneur.

Cette granularité est essentielle pour éviter qu'un service comme le **reranking** ou l'ingestion ne monopolise toutes les ressources disponibles.

Déploiement reproductible

L'un des principaux avantages de la dockerisation est la reproductibilité du déploiement. Un même système **RAG** peut être déployé de manière identique sur une machine locale, un serveur VPS ou un cluster cloud sans modification du code.

Cela simplifie fortement le cycle de développement et réduit les risques liés aux différences d'environnement.

Conclusion

La dockerisation complète est une étape indispensable dans la mise en production d'un système **RAG**. Elle permet de standardiser l'exécution des services, de faciliter le déploiement et d'assurer la reproductibilité du système dans tous les environnements.

Elle constitue également la base sur laquelle reposent les architectures plus avancées d'orchestration et de **scalabilité** en production.

Section 14.2 — Docker Compose multi-services

Section 14.2 — Docker Compose multi-services

Docker Compose est un outil essentiel dans la phase de déploiement d'un système **RAG**, car il permet de définir et d'exécuter plusieurs conteneurs comme un système unique. Dans une architecture RAG industrielle, où plusieurs services doivent fonctionner ensemble (API, **retrieval**, **reranking**, base vectorielle, cache), Docker Compose sert de couche d'orchestration simple et reproductible.

Rôle de Docker Compose

Docker Compose permet de décrire l'ensemble des composants d'un système **RAG** dans un seul fichier de configuration. Ce fichier définit les services, leurs dépendances, leurs ports exposés et leurs réseaux internes.

Dans un système **RAG**, cela signifie que l'API **FastAPI**, le service de **retrieval**, la base vectorielle et les services de support comme **Redis** peuvent être démarrés et arrêtés ensemble de manière cohérente.

Définition des services

Chaque composant du système **RAG** est défini comme un service indépendant dans **Docker** Compose. L'API est généralement exposée comme point d'entrée principal, tandis que les autres services restent accessibles uniquement via le réseau interne du cluster Docker.

Le service de base vectorielle est configuré avec un stockage persistant afin de garantir la conservation des données entre les redémarrages. Le service de cache est configuré pour accélérer les requêtes répétées, tandis que les services de traitement comme le **reranking** peuvent être isolés pour des raisons de performance.

Gestion des dépendances entre services

Dans un système **RAG**, certains services dépendent fortement d'autres composants. Par exemple, le service de **retrieval** dépend de la base vectorielle, et le service de génération dépend du **reranking** et du **retrieval**.

Docker Compose permet de définir ces dépendances afin de garantir que les services démarrent dans le bon ordre. Cependant, il ne garantit pas que les services soient pleinement prêts, ce qui nécessite souvent des mécanismes supplémentaires comme des health checks.

Réseaux internes et communication

Docker Compose crée automatiquement un réseau interne permettant aux services de communiquer entre eux sans exposition externe. Cela est essentiel pour sécuriser les échanges entre les composants du système **RAG**.

L'API peut ainsi appeler directement le service de **retrieval** ou le **reranking** via leurs noms de service, sans avoir besoin d'adresses IP fixes.

Persistance des données

Dans un système **RAG**, la persistance des données est critique, notamment pour la base vectorielle et les systèmes de cache. **Docker** Compose permet de définir des volumes persistants qui assurent la conservation des index et des **embeddings** même en cas de redémarrage des conteneurs.

Cette persistance est indispensable pour éviter de devoir reconstruire l'index à chaque déploiement.

Environnements de développement et production

Docker Compose est souvent utilisé dans les environnements de développement et de test, car il permet de simuler une architecture complète de manière simple et rapide.

En production, il est souvent complété ou remplacé par des orchestrateurs plus avancés comme Kubernetes, qui offrent une meilleure **scalabilité** et une gestion plus fine des ressources.

Limitations de Docker Compose

Malgré sa simplicité, **Docker** Compose présente des limitations importantes pour les systèmes **RAG** à grande échelle. Il ne gère pas nativement la **scalabilité** automatique, ni la tolérance aux pannes avancée.

Il est également limité en termes de gestion dynamique des charges et de déploiement distribué sur plusieurs machines.

Conclusion

Docker Compose constitue une étape essentielle dans la construction et le déploiement d'un système **RAG** multi-services. Il permet de structurer l'ensemble des composants dans un environnement cohérent et reproductible, facilitant ainsi le développement et les tests.

Cependant, son utilisation reste principalement adaptée aux environnements de développement ou aux systèmes de taille modérée, tandis que les architectures industrielles à grande échelle nécessitent des solutions d'orchestration plus avancées.

Section 14.3 — Déploiement VPS / cloud

Le déploiement d'un système **RAG** sur un VPS ou dans le cloud constitue l'étape qui transforme une architecture locale conteneurisée en un service accessible en production. Cette étape implique des choix importants en matière d'infrastructure, de sécurité, de **scalabilité** et de gestion des coûts.

Principe du déploiement

Le déploiement sur VPS ou cloud consiste à exécuter l'ensemble des services du système **RAG** sur une infrastructure distante. Cette infrastructure peut être un serveur virtuel unique (VPS) ou un ensemble de ressources cloud distribuées.

Dans les deux cas, l'objectif est de rendre le système accessible via Internet tout en garantissant sa stabilité et sa performance.

Déploiement sur VPS

Le VPS est souvent la solution la plus simple pour déployer un système **RAG** en production initiale. Il consiste à louer une machine virtuelle sur laquelle l'ensemble des conteneurs **Docker** est exécuté.

Cette approche permet de contrôler entièrement l'environnement d'exécution et de simplifier la configuration du système. Elle est particulièrement adaptée aux projets de taille moyenne ou aux premières versions de produits **RAG**.

Cependant, le VPS présente des limitations importantes en termes de **scalabilité** et de résilience, car toutes les charges reposent sur une seule machine.

Déploiement cloud

Le déploiement cloud repose sur des infrastructures plus complexes et distribuées. Les services peuvent être répartis sur plusieurs machines virtuelles, conteneurs managés ou services serverless.

Cette approche permet une **scalabilité** beaucoup plus importante, ainsi qu'une meilleure tolérance aux pannes. Elle est adaptée aux systèmes **RAG** à forte charge ou à usage multi-utilisateurs.

Le cloud permet également d'intégrer facilement des services complémentaires comme le stockage distribué, les bases de données managées ou les GPU à la demande.

Gestion des ressources

Dans un environnement VPS ou cloud, la gestion des ressources devient un élément critique. Le système **RAG** doit être dimensionné en fonction de la charge attendue, en tenant compte de la consommation CPU, mémoire et GPU des différents services.

Les composants les plus coûteux, comme le **reranking** ou les **embeddings**, nécessitent souvent des ressources spécialisées pour garantir des performances acceptables.

Sécurité du déploiement

La sécurité est un aspect fondamental du déploiement en production. Le système doit être protégé contre les accès non autorisés, les injections de requêtes malveillantes et les fuites de données.

Cela inclut la configuration de pare-feu, la gestion des certificats SSL, l'authentification des utilisateurs et la sécurisation des communications entre services internes.

Scalabilité et élasticité

Le cloud offre des capacités d'élasticité qui permettent d'adapter automatiquement les ressources en fonction de la charge. Cela est particulièrement utile pour les systèmes **RAG** dont l'utilisation peut varier fortement dans le temps.

La **scalabilité** peut être horizontale, en ajoutant de nouvelles instances de services, ou verticale, en augmentant la puissance des machines existantes.

Monitoring et maintenance

Un système **RAG** déployé en production doit être surveillé en permanence. Le monitoring permet de suivre les performances, la latence, l'utilisation des ressources et les erreurs système.

Cette observabilité est essentielle pour détecter rapidement les problèmes et maintenir un niveau de service stable.

Conclusion

Le déploiement sur VPS ou cloud constitue une étape clé dans l'industrialisation d'un système **RAG**. Il transforme une architecture locale en un service accessible et scalable, capable de répondre à des usages réels en production.

Le choix entre VPS et cloud dépend principalement des besoins en **scalabilité**, en budget et en complexité opérationnelle, mais dans les systèmes **RAG** à grande échelle, le cloud devient généralement incontournable.

Section 14.4 — Scalabilité horizontale

La **scalabilité** horizontale est un principe fondamental des systèmes **RAG** en production. Elle consiste à augmenter la capacité du système non pas en renforçant une seule machine, mais en ajoutant plusieurs instances de services identiques qui travaillent en parallèle. Cette approche est essentielle pour absorber des charges importantes et garantir une disponibilité continue du service.

Principe de la scalabilité horizontale

La **scalabilité** horizontale repose sur la duplication des composants du système **RAG**. Au lieu de faire évoluer une seule instance vers une machine plus puissante, le système est réparti sur plusieurs instances identiques qui partagent la charge de travail.

Dans une architecture **RAG**, cela signifie que plusieurs instances de l'API, du service de **retrieval** ou du **reranking** peuvent fonctionner simultanément pour traiter des requêtes en parallèle.

Stateless et distribution des charges

Pour que la **scalabilité** horizontale soit efficace, les services doivent être conçus de manière stateless. Cela signifie qu'ils ne doivent pas conserver d'état critique en mémoire locale, mais s'appuyer sur des systèmes externes comme des bases de données ou des caches partagés.

Cette conception permet de répartir librement les requêtes entre différentes instances sans perte de cohérence.

Load balancing

Le load balancing joue un rôle central dans la **scalabilité** horizontale. Il permet de répartir les requêtes entrantes entre les différentes instances disponibles d'un service.

Dans un système **RAG**, le load balancer distribue les requêtes vers les instances de l'API ou du **retrieval** en fonction de la charge, de la disponibilité et parfois de la localisation géographique des utilisateurs.

Scalabilité des composants du pipeline

Tous les composants d'un système **RAG** ne scalent pas de la même manière. L'API et le **retrieval** sont généralement facilement scalables horizontalement, car ils peuvent être répliqués sans état.

En revanche, certains composants comme la base vectorielle ou le **reranking** nécessitent des stratégies spécifiques de partitionnement ou de distribution des données pour fonctionner efficacement à grande échelle.

Partitionnement des données

Pour supporter la **scalabilité** horizontale, les données doivent souvent être partitionnées. Dans une base vectorielle, cela signifie répartir les **embeddings** sur plusieurs shards ou segments.

Chaque instance peut alors traiter une partie des données, ce qui permet d'augmenter la capacité globale du système sans dégrader les performances.

Impact sur la latence et le débit

La **scalabilité** horizontale permet d'augmenter significativement le débit global du système **RAG**. Plusieurs requêtes peuvent être traitées en parallèle, ce qui réduit les files d'attente et améliore la réactivité globale.

Cependant, une mauvaise configuration peut introduire une latence supplémentaire liée à la coordination entre services et à la recherche distribuée.

Tolérance aux pannes

Un des avantages majeurs de la **scalabilité** horizontale est l'amélioration de la tolérance aux pannes. Si une instance tombe en panne, les autres instances peuvent continuer à traiter les requêtes sans interruption de service.

Cette redondance est essentielle pour garantir la haute disponibilité d'un système **RAG** en production.

Limites de la scalabilité horizontale

Malgré ses avantages, la **scalabilité** horizontale introduit une complexité supplémentaire. La gestion de la cohérence des données, la synchronisation des index et la distribution des charges deviennent plus difficiles à maîtriser.

Certains composants, comme les bases vectorielles ou les modèles de **reranking** lourds, nécessitent des architectures spécifiques pour éviter les goulots d'étranglement.

Conclusion

La **scalabilité** horizontale est un pilier des architectures **RAG** industrielles. Elle permet de transformer un système limité par une seule machine en une plateforme distribuée capable de gérer un grand nombre de requêtes simultanées.

Bien qu'elle introduise une complexité supplémentaire, elle est indispensable pour atteindre un niveau de performance et de disponibilité compatible avec les exigences de production.

Chapitre 15 — Observabilité et monitoring

Dans un système **RAG** en production, l'observabilité n'est pas un ajout optionnel, mais une composante structurelle de l'architecture. Un système aussi distribué, combinant ingestion, **retrieval**, **reranking** et génération, doit être capable d'expliquer en permanence son comportement interne afin d'être exploitable, débogable et fiable.

Section 15.1 — Logs structurés

Les logs structurés constituent la base de toute stratégie d'observabilité dans un système **RAG**. Contrairement aux logs textuels non structurés, ils organisent l'information sous forme de champs explicites, ce qui permet leur exploitation automatique par des outils de monitoring, d'analyse et de détection d'anomalies.

Principe des logs structurés

Un log structuré consiste à enregistrer chaque événement du système sous la forme d'un ensemble de paires clé-valeur. Chaque log représente une action précise du **pipeline RAG**, comme une requête utilisateur, une recherche vectorielle, une opération de **reranking** ou un appel au **LLM**.

Cette structuration permet de transformer les logs en données exploitables, qui peuvent être filtrées, agrégées et analysées de manière automatisée.

Granularité des logs

Dans un système **RAG**, la granularité des logs est essentielle. Un système trop verbeux devient difficile à exploiter, tandis qu'un système trop minimaliste ne permet pas de diagnostiquer les problèmes.

Il est donc nécessaire de logger les étapes clés du **pipeline**, notamment l'ingestion des documents, la génération des **embeddings**, les résultats du **retrieval**, les scores de **reranking** et les appels au modèle de langage.

Corrélation des événements

Les logs structurés permettent de corréler les événements entre les différentes étapes du **pipeline RAG**. Chaque requête utilisateur peut être associée à un identifiant unique qui suit le flux complet du système.

Cette corrélation est essentielle pour reconstruire le chemin exact d'une requête, depuis son entrée dans l'API jusqu'à la génération de la réponse finale.

Logs et debugging

Les logs structurés jouent un rôle central dans le debugging des systèmes **RAG**. Lorsqu'une réponse est incorrecte ou incohérente, les logs permettent d'identifier précisément l'étape du **pipeline** où le problème est survenu.

Ils permettent également de comparer les résultats attendus et les résultats réels à chaque étape, facilitant ainsi l'analyse des erreurs.

Performance et analyse

Les logs structurés ne servent pas uniquement au debugging, mais également à l'analyse des performances du système. Ils permettent de mesurer la latence de chaque étape du **pipeline**, d'identifier les goulots d'étranglement et d'optimiser les composants les plus coûteux.

Cette analyse fine est essentielle pour améliorer la performance globale d'un système **RAG** en production.

Stockage et exploitation des logs

Dans un système industriel, les logs structurés sont généralement envoyés vers des systèmes centralisés de collecte et d'analyse. Ces systèmes permettent de stocker les logs à grande échelle et de les interroger efficacement.

Ils sont souvent couplés à des outils de visualisation qui permettent de créer des dashboards de monitoring en temps réel.

Sécurité et conformité

Les logs structurés doivent également être conçus avec des contraintes de sécurité et de conformité. Il est important de ne pas exposer d'informations sensibles dans les logs, notamment des données personnelles ou des contenus confidentiels issus du **retrieval**.

Des mécanismes de filtrage ou d'anonymisation peuvent être nécessaires pour respecter les exigences réglementaires.

Conclusion

Les logs structurés constituent un pilier fondamental de l'observabilité dans un système **RAG**. Ils permettent de transformer le comportement interne du système en données exploitables, facilitant ainsi le debugging, l'optimisation et la supervision en production.

Sans une stratégie de logging structurée, il devient extrêmement difficile de maintenir et d'améliorer un système **RAG** à grande échelle.

Section 15.2 — Metrics système

Les métriques système constituent le deuxième pilier de l'observabilité dans une architecture **RAG** en production, complémentaire aux logs structurés. Là où les logs décrivent des événements unitaires et détaillés, les métriques permettent de mesurer en continu l'état global du système sous forme agrégée et quantifiable.

Rôle des métriques dans un système RAG

Dans un système **RAG**, les métriques servent à surveiller la santé, la performance et la stabilité de l'ensemble du **pipeline**. Elles permettent de détecter rapidement les dégradations, d'anticiper les surcharges et d'identifier les composants les plus coûteux en ressources.

Ces indicateurs sont essentiels pour maintenir un niveau de service constant, en particulier dans des environnements à forte charge ou à usage critique.

Types de métriques principales

Les métriques d'un système **RAG** peuvent être regroupées en plusieurs catégories. Les métriques de performance mesurent la latence des différentes étapes du **pipeline**, comme le **retrieval**, le **reranking** et la génération. Les métriques de débit mesurent le nombre de requêtes traitées par seconde ou par minute.

Les métriques de ressources concernent l'utilisation CPU, mémoire et GPU des différents services. Enfin, les métriques de qualité permettent d'évaluer indirectement la pertinence des réponses générées.

Latence par composant

Une métrique essentielle dans un système **RAG** est la latence segmentée par composant. Au lieu de mesurer uniquement le temps total de réponse, il est important de mesurer séparément la latence de l'**embedding**, du **retrieval**, du **reranking** et du **LLM**.

Cette granularité permet d'identifier précisément les goulots d'étranglement du **pipeline** et d'optimiser les composants les plus coûteux.

Débit et charge système

Le débit d'un système **RAG** correspond au nombre de requêtes traitées par unité de temps. Cette métrique est essentielle pour comprendre la capacité réelle du système à supporter une charge utilisateur.

Elle doit être analysée en parallèle avec la latence, car un système peut maintenir un débit élevé tout en dégradant fortement les temps de réponse.

Utilisation des ressources

Les métriques d'utilisation des ressources permettent de surveiller la consommation CPU, mémoire et GPU de chaque composant du système **RAG**. Ces indicateurs sont particulièrement importants pour les services de **reranking** et de génération, qui sont souvent les plus gourmands en calcul.

Une surveillance fine de ces ressources permet d'optimiser le dimensionnement de l'infrastructure et d'éviter les saturations.

Métriques de qualité

Au-delà des performances techniques, un système **RAG** doit également être évalué sur la qualité de ses réponses. Des métriques indirectes peuvent être utilisées, comme le taux de clic sur les sources, les retours utilisateurs ou des scores d'évaluation automatisés.

Ces indicateurs permettent de mesurer l'efficacité réelle du système du point de vue utilisateur.

Alerting et seuils

Les métriques système sont utilisées pour définir des seuils d'alerte. Lorsqu'une métrique dépasse une valeur critique, un système d'alerte peut être déclenché pour signaler une anomalie.

Par exemple, une augmentation brutale de la latence du **retrieval** ou une chute du débit peut indiquer une défaillance du système.

Agrégation et visualisation

Les métriques sont généralement agrégées sur différentes fenêtres de temps afin de lisser les variations et d'identifier les tendances. Elles sont ensuite visualisées dans des dashboards permettant de surveiller en temps réel l'état du système **RAG**.

Cette visualisation est essentielle pour les équipes d'exploitation et de maintenance.

Corrélation avec les logs

Les métriques et les logs structurés sont complémentaires. Les métriques permettent d'identifier qu'un problème existe, tandis que les logs permettent de comprendre pourquoi il existe.

Cette corrélation est indispensable pour diagnostiquer efficacement les incidents dans un système **RAG** complexe.

Conclusion

Les métriques système constituent un élément fondamental de l'observabilité dans une architecture **RAG** en production. Elles permettent de surveiller en continu la performance, la charge et la santé du système, tout en fournissant une base solide pour l'optimisation et l'alerte.

Sans métriques fiables et bien définies, il est impossible de garantir la stabilité et la performance d'un système **RAG** à grande échelle.

Section 15.3 — Monitoring RAG (latence, recall)

Le monitoring d'un système **RAG** ne peut pas se limiter aux métriques classiques d'infrastructure. Il doit également intégrer des indicateurs spécifiques au fonctionnement du **pipeline de retrieval** et à la qualité des réponses générées. Parmi ces indicateurs, la latence et le recall occupent une place centrale, car ils représentent respectivement la performance système et la capacité du système à retrouver l'information pertinente.

Monitoring de la latence RAG

La latence dans un système **RAG** correspond au temps total nécessaire pour traiter une requête utilisateur, depuis la réception de la question jusqu'à la génération de la réponse finale par le **LLM**. Cette latence doit être mesurée de manière globale, mais également décomposée par étape du **pipeline**.

Un monitoring efficace distingue la latence de l'**embedding**, la latence du **retrieval**, la latence du **reranking** et la latence du modèle de langage. Cette décomposition permet d'identifier précisément les goulots d'étranglement et d'optimiser les composants les plus critiques.

En production, il est également important de suivre des percentiles de latence, comme le P50, P95 et P99, afin de comprendre non seulement la performance moyenne, mais aussi les cas extrêmes qui impactent l'expérience utilisateur.

Importance du recall dans le retrieval

Le recall est une métrique fondamentale pour évaluer la qualité du **retrieval** dans un système **RAG**. Il mesure la capacité du système à récupérer les documents pertinents parmi l'ensemble des documents disponibles.

Un recall élevé signifie que les informations nécessaires à la réponse sont bien présentes dans les résultats récupérés, ce qui augmente les chances de générer une réponse correcte. À l'inverse, un faible recall indique que des informations importantes sont manquées dès la phase de **retrieval**, ce qui ne peut pas être compensé par le **LLM**.

Mesure du recall en production

Le recall ne peut pas toujours être mesuré directement en production, car il nécessite de connaître les documents pertinents pour chaque requête. Dans les systèmes industriels, il est souvent estimé à partir de jeux de données de test ou de benchmarks internes.

Ces évaluations permettent de mesurer la capacité du système à retrouver les bons chunks dans le **Top-K** des résultats de recherche. Elles sont essentielles pour comparer différentes configurations de **retrieval** ou différents modèles d'**embeddings**.

Corrélation entre latence et recall

Dans un système **RAG**, il existe souvent un compromis entre latence et recall. Augmenter le nombre de documents récupérés améliore généralement le recall, mais augmente également la latence globale du système, notamment à cause du **reranking** et de la taille du contexte envoyé au **LLM**.

Le monitoring doit donc permettre de suivre ces deux dimensions simultanément afin de trouver un équilibre optimal entre performance et qualité.

Dégradation contrôlée du système

Le monitoring de la latence et du recall permet également de mettre en place des stratégies de dégradation contrôlée. Lorsque la latence dépasse un certain seuil, le système peut réduire le **Top-K** du **retrieval** ou désactiver certaines étapes comme le **reranking**.

Ces mécanismes permettent de maintenir un service opérationnel même en cas de surcharge, au prix d'une légère baisse de qualité.

Visualisation des performances RAG

Les systèmes de monitoring avancés intègrent des dashboards dédiés aux métriques **RAG**. Ces dashboards affichent la distribution de la latence, les scores de recall, ainsi que leur évolution dans le temps.

Cette visualisation permet de détecter rapidement les régressions après un changement de modèle, une mise à jour d'index ou une modification du **pipeline**.

Conclusion

Le monitoring de la latence et du recall est essentiel pour garantir la performance et la qualité d'un système **RAG** en production. Il permet de s'assurer que le système reste à la fois rapide et capable de récupérer les informations pertinentes nécessaires à la génération des réponses.

Sans ces indicateurs spécifiques, il est impossible d'évaluer correctement l'efficacité réelle d'un système **RAG**, même si les métriques d'infrastructure restent stables.

Section 15.4 — Alerting

L'alerting constitue la couche active de l'observabilité dans un système **RAG** en production. Contrairement au monitoring, qui consiste à observer et mesurer l'état du système, l'alerting a pour objectif de détecter automatiquement les anomalies et de déclencher des notifications ou des actions correctives lorsque certaines conditions critiques sont atteintes.

Rôle de l'alerting dans un système RAG

Dans une architecture **RAG**, l'alerting permet de garantir que les dégradations de performance ou les défaillances fonctionnelles sont détectées rapidement. Il joue un rôle essentiel dans la prévention des interruptions de service et dans la réduction du temps de résolution des incidents.

Un système **RAG** en production peut comporter de nombreux points de défaillance, notamment au niveau du **retrieval**, des **embeddings**, du **reranking** ou du **LLM**. L'alerting permet de surveiller ces composants de manière proactive.

Définition des seuils d'alerte

Les alertes reposent sur des seuils définis à partir des métriques système. Ces seuils peuvent être fixes ou dynamiques. Un seuil fixe déclenche une alerte lorsque une métrique dépasse une valeur prédéfinie, par exemple une latence supérieure à un certain nombre de millisecondes.

Les seuils dynamiques s'adaptent aux variations normales du système et permettent de détecter des anomalies par rapport au comportement habituel.

Types d'alertes

Dans un système **RAG**, les alertes peuvent être classées en plusieurs catégories. Les alertes de performance concernent la latence excessive ou la baisse de débit. Les alertes de disponibilité signalent les pannes de services comme la base vectorielle ou le **LLM**.

Les alertes de qualité sont également importantes, car elles permettent de détecter une dégradation du recall ou une augmentation des réponses incohérentes.

Alerting basé sur les métriques RAG

L'alerting dans un système **RAG** ne se limite pas aux métriques système classiques. Il repose également sur des indicateurs spécifiques comme la latence du **retrieval**, le taux de succès du **reranking** ou la stabilité des **embeddings**.

Par exemple, une baisse soudaine du recall peut indiquer un problème d'indexation, tandis qu'une augmentation de la latence du **reranker** peut signaler une surcharge GPU.

Réduction du bruit d'alerte

Un problème majeur dans les systèmes d'alerting est la génération de faux positifs ou d'alertes trop fréquentes. Dans un système **RAG**, cela peut entraîner une fatigue des équipes et une perte de réactivité.

Pour éviter cela, des techniques de filtrage sont utilisées, comme l'agrégation des alertes sur des fenêtres de temps ou l'utilisation de seuils adaptatifs basés sur les tendances historiques.

Intégration avec les systèmes de notification

Les alertes doivent être intégrées à des systèmes de notification capables de prévenir les équipes en temps réel. Cela peut inclure des emails, des messages Slack ou des systèmes de pager pour les incidents critiques.

Cette intégration permet de réduire le temps de réaction en cas de problème et d'assurer une continuité de service.

Automatisation des réponses

Dans certains systèmes **RAG** avancés, les alertes peuvent déclencher automatiquement des actions correctives. Par exemple, une surcharge du service de **retrieval** peut entraîner une mise à l'échelle automatique des instances, ou une dégradation contrôlée du **pipeline** peut être activée.

Cette automatisation permet de limiter l'impact des incidents sans intervention humaine immédiate.

Corrélation avec le monitoring et les logs

L'alerting est étroitement lié aux systèmes de monitoring et de logs. Lorsqu'une alerte est déclenchée, les métriques permettent de comprendre la nature du problème, tandis que les logs permettent d'en analyser la cause précise.

Cette combinaison est essentielle pour diagnostiquer efficacement les incidents dans un système **RAG** complexe.

Conclusion

L'alerting est une composante essentielle de l'observabilité dans un système **RAG** en production. Il permet de transformer les données de monitoring en actions concrètes, garantissant ainsi une réactivité rapide face aux anomalies et une meilleure stabilité globale du système.

Sans un système d'alerting bien conçu, même un système **RAG** performant peut devenir difficile à maintenir en conditions réelles.

Partie IV — Sécurité, robustesse et multi-utilisateurs

Les systèmes **RAG** en production ne se limitent pas à des problématiques de performance ou de qualité de réponse. Lorsqu'ils sont déployés dans des contextes réels, ils manipulent des données potentiellement sensibles et sont exposés à des utilisateurs multiples. La sécurité devient alors une contrainte structurelle de l'architecture, au même titre que la **scalabilité** ou la latence.

Chapitre 16 — Authentification et sécurité

L'authentification et la sécurité constituent la première ligne de défense d'un système **RAG** en production. Elles garantissent que seules les entités autorisées peuvent accéder aux données, interroger le système ou ingérer de nouveaux documents. Sans cette couche, un système RAG devient vulnérable aux fuites d'information, aux abus d'usage et aux attaques par injection.

Rôle de l'authentification

L'authentification permet d'identifier de manière fiable les utilisateurs ou les services qui interagissent avec l'API **RAG**. Elle est essentielle pour associer chaque requête à une identité et appliquer des règles d'accès adaptées.

Dans un système **RAG** multi-utilisateurs, cette identification permet également de segmenter les données, d'isoler les contextes et d'éviter les contaminations entre différents espaces documentaires.

Gestion des tokens et sessions

Les systèmes **RAG** modernes utilisent généralement des mécanismes basés sur des tokens pour gérer l'authentification. Ces tokens sont transmis à chaque requête et validés par l'API avant l'exécution du **pipeline**.

Ils peuvent être associés à des sessions utilisateur, ce qui permet de conserver un contexte conversationnel ou un historique de requêtes sans exposer directement les données sensibles.

Contrôle d'accès aux données

Au-delà de l'authentification, un système **RAG** doit implémenter un contrôle d'accès précis aux données. Tous les utilisateurs n'ont pas nécessairement accès aux mêmes documents ou aux mêmes index vectoriels.

Ce contrôle peut être basé sur des rôles, des permissions ou des espaces isolés, garantissant ainsi que chaque requête ne récupère que des informations autorisées.

Sécurisation du pipeline RAG

La sécurité d'un système **RAG** ne se limite pas à l'API. Elle doit être appliquée à chaque étape du **pipeline**, notamment lors du **retrieval** et de la génération.

Les mécanismes de filtrage doivent garantir que les documents sensibles ne sont jamais exposés à des utilisateurs non autorisés, même s'ils sont présents dans la base vectorielle.

Protection contre les injections

Les systèmes **RAG** sont particulièrement exposés aux attaques par injection, notamment via les prompts envoyés au **LLM**. Un utilisateur malveillant peut tenter de manipuler le modèle pour contourner les règles ou extraire des informations sensibles.

Pour limiter ces risques, il est nécessaire de nettoyer et de filtrer les entrées utilisateur, ainsi que de structurer strictement les prompts envoyés au modèle de langage.

Isolation multi-utilisateurs

Dans un système **RAG multi-tenant**, l'isolation des utilisateurs est une exigence fondamentale. Chaque utilisateur ou organisation doit disposer de son propre espace logique de données.

Cette isolation permet d'éviter les fuites d'informations entre clients et garantit que les résultats de **retrieval** sont strictement pertinents pour le contexte utilisateur.

Chiffrement et protection des données

La sécurité d'un système **RAG** inclut également le chiffrement des données, aussi bien en transit qu'au repos. Les communications entre les services doivent être sécurisées, et les bases de données doivent protéger les **embeddings** et les documents sensibles.

Cette protection est indispensable dans les environnements où les données traitées sont confidentielles ou réglementées.

Journalisation sécurisée

Les logs et les métriques doivent être conçus avec des contraintes de sécurité strictes. Il est essentiel d'éviter la fuite d'informations sensibles dans les logs, notamment les contenus de documents ou les requêtes utilisateur complètes.

Des mécanismes d'anonymisation ou de masquage peuvent être nécessaires pour respecter les exigences de conformité.

Conclusion

L'authentification et la sécurité constituent un pilier essentiel des systèmes **RAG** en production. Elles garantissent que le système reste fiable, isolé et protégé contre les usages malveillants ou non autorisés.

Sans cette couche de sécurité, même une architecture **RAG** techniquement performante ne peut être considérée comme exploitable en environnement réel.

Section 16.1 — JWT et OAuth2

Dans un système **RAG** en production, l'authentification doit être à la fois robuste, scalable et compatible avec une architecture distribuée. Les mécanismes basés sur **JWT** et **OAuth2** sont aujourd'hui les standards les plus utilisés pour sécuriser les APIs exposées à des utilisateurs multiples et à des services internes.

Rôle de JWT dans une API RAG

Le JSON Web Token (**JWT**) est un format de jeton permettant de transmettre de manière sécurisée des informations d'authentification entre un client et un serveur. Dans un système **RAG**, le JWT est généralement utilisé pour identifier un utilisateur à chaque requête envoyée à l'API.

Le token est signé cryptographiquement, ce qui permet au serveur de vérifier son authenticité sans avoir besoin de conserver un état de session centralisé. Cette caractéristique est particulièrement importante dans les architectures **RAG** scalables, où plusieurs instances de l'API peuvent être déployées en parallèle.

Structure d'un JWT

Un **JWT** est composé de trois parties distinctes. L'en-tête contient les informations sur l'algorithme de signature utilisé. La charge utile contient les informations d'identité et les claims associés à

l'utilisateur. La signature garantit l'intégrité du token et empêche toute modification non autorisée.

Dans un système **RAG**, la charge utile peut inclure des informations comme l'identifiant utilisateur, les rôles, ou encore les permissions d'accès aux documents.

Validation des tokens

Lorsqu'une requête est envoyée à une API **RAG**, le token **JWT** est vérifié avant toute exécution du **pipeline**. Cette validation consiste à contrôler la signature, la date d'expiration et les permissions associées au token.

Si le token est invalide ou expiré, la requête est rejetée avant d'accéder aux étapes de **retrieval** ou de génération, ce qui protège l'ensemble du système contre les accès non autorisés.

Rôle d'OAuth2

OAuth2 est un protocole d'autorisation qui permet de déléguer l'accès à des ressources protégées sans exposer directement les identifiants utilisateur. Dans un système **RAG**, OAuth2 est souvent utilisé pour gérer l'authentification auprès de fournisseurs externes ou pour intégrer des systèmes d'identité existants.

Il permet également de standardiser la gestion des permissions dans des environnements multi-applications.

Flux d'authentification OAuth2

Dans un flux **OAuth2** classique, un utilisateur s'authentifie auprès d'un fournisseur d'identité, qui délivre un access token. Ce token est ensuite utilisé pour accéder à l'API **RAG**.

Ce mécanisme permet de centraliser la gestion de l'identité et de simplifier l'intégration avec des systèmes d'entreprise existants.

Intégration JWT et OAuth2 dans un système RAG

Dans une architecture **RAG** industrielle, **JWT** et **OAuth2** sont souvent utilisés ensemble. OAuth2 sert à obtenir un token d'accès, tandis que JWT est utilisé pour transmettre et valider les informations d'authentification au sein du système.

Cette combinaison permet de garantir à la fois la flexibilité d'intégration et la performance de validation côté API.

Sécurité et durée de vie des tokens

La gestion de la durée de vie des tokens est un élément critique de la sécurité. Des tokens trop longs augmentent les risques de compromission, tandis que des tokens trop courts peuvent dégrader l'expérience utilisateur.

Dans un système **RAG**, il est courant d'utiliser des access tokens à durée courte combinés à des refresh tokens pour maintenir une session sécurisée.

Impact sur le pipeline RAG

L'authentification basée sur **JWT** et **OAuth2** influence directement le **pipeline RAG**. Elle permet d'associer chaque requête à un contexte utilisateur spécifique, ce qui est essentiel pour appliquer des filtres de **retrieval**, isoler les données et personnaliser les réponses générées.

Sans cette couche, il serait impossible de garantir une séparation correcte des données dans un environnement multi-utilisateurs.

Conclusion

JWT et **OAuth2** constituent des fondations essentielles pour la sécurité des systèmes **RAG** en production. Ils permettent de gérer l'identité, les permissions et l'accès aux données de manière scalable et standardisée.

Leur utilisation combinée garantit à la fois la robustesse de l'authentification et l'intégration fluide dans des architectures distribuées complexes.

Section 16.2 — Protection API

Section 16.2 — Protection API

La protection d'une API **RAG** en production ne se limite pas à l'authentification des utilisateurs. Elle englobe un ensemble de mécanismes destinés à préserver l'intégrité du système, à éviter les abus et à garantir la continuité de service face à des usages normaux comme malveillants. Dans une architecture RAG, l'API constitue le point d'entrée critique vers des ressources sensibles comme les documents, les **embeddings** et les résultats de **retrieval**.

Rôle de la protection API

La protection API a pour objectif de contrôler la manière dont les requêtes accèdent au système **RAG**. Elle agit comme une couche de défense située entre les utilisateurs et le **pipeline** interne.

Cette couche permet de limiter les accès non autorisés, de prévenir les attaques automatisées et de garantir que les ressources système ne sont pas saturées par des requêtes abusives ou mal formées.

Rate limiting et contrôle de débit

Le rate limiting est un mécanisme essentiel de protection. Il consiste à limiter le nombre de requêtes qu'un utilisateur ou un service peut effectuer sur une période donnée.

Dans un système **RAG**, ce mécanisme est particulièrement important car chaque requête peut déclencher des opérations coûteuses comme le **retrieval** vectoriel ou l'appel au **LLM**. Sans limitation, un usage intensif peut rapidement saturer l'infrastructure.

Protection contre les attaques par surcharge

Les systèmes **RAG** peuvent être exposés à des attaques par déni de service, intentionnelles ou non. Pour se protéger, l'API doit intégrer des mécanismes de contrôle de charge, comme la limitation du nombre de requêtes simultanées et la mise en file d'attente des requêtes excédentaires.

Ces mécanismes permettent de maintenir le système opérationnel même en cas de pic de trafic.

Validation stricte des requêtes

Une API **RAG** doit valider strictement toutes les requêtes entrantes avant de les traiter. Cela inclut la vérification des formats, la taille des entrées et la conformité des paramètres.

Cette validation permet d'éviter les erreurs de parsing, les injections malveillantes et les comportements inattendus dans le **pipeline** de **retrieval** et de génération.

Filtrage des contenus malveillants

La protection API inclut également le filtrage des contenus susceptibles de nuire au système. Cela peut inclure des tentatives d'injection dans les prompts, des requêtes visant à extraire des informations sensibles ou des contenus non conformes aux règles d'utilisation.

Ce filtrage est essentiel pour protéger à la fois le système et les données qu'il traite.

Isolation des utilisateurs et des ressources

Dans un système **RAG** multi-utilisateurs, la protection API doit garantir une isolation stricte entre les différents utilisateurs. Chaque utilisateur doit être limité à ses propres données et ne doit pas

pouvoir accéder aux ressources d'autres tenants.

Cette isolation est essentielle pour éviter les fuites d'informations et garantir la confidentialité des données.

Sécurisation des endpoints sensibles

Certains endpoints d'une API **RAG** sont particulièrement critiques, notamment `/ingest` et `/query`. Ces endpoints doivent être protégés de manière renforcée, car ils donnent accès respectivement à l'ajout de données dans le système et à la récupération d'informations issues du corpus.

Des politiques de sécurité spécifiques peuvent être appliquées à ces endpoints, comme des contrôles d'accès plus stricts ou des quotas plus faibles.

Journalisation des accès

La protection API inclut également la journalisation des accès afin de tracer toutes les interactions avec le système. Ces logs permettent d'identifier les comportements anormaux et de retracer les actions en cas d'incident de sécurité.

Cette traçabilité est indispensable pour assurer la transparence et la conformité du système.

Conclusion

La protection API est une couche essentielle dans l'architecture d'un système **RAG** en production. Elle garantit que le système reste stable, sécurisé et résilient face aux usages normaux comme aux comportements malveillants.

Sans cette couche de protection, même une API correctement authentifiée reste vulnérable aux abus et aux surcharges.

Section 16.3 — Isolation des données

L'isolation des données est un principe fondamental des systèmes **RAG** multi-utilisateurs en production. Elle garantit que chaque utilisateur ou organisation n'accède qu'à ses propres informations, même lorsque toutes les données sont stockées dans une infrastructure commune. Dans une architecture RAG, où les documents sont transformés en **embeddings** et indexés dans une base vectorielle partagée, cette isolation devient un enjeu critique de sécurité et de conformité.

Rôle de l'isolation des données

L'isolation des données a pour objectif d'empêcher toute fuite d'information entre différents utilisateurs ou clients d'un même système **RAG**. Elle assure que les résultats de **retrieval**, les **embeddings** et les documents sont filtrés en fonction du contexte d'accès de chaque requête.

Sans isolation stricte, un utilisateur pourrait potentiellement récupérer des informations issues d'un autre espace documentaire, ce qui constitue une faille majeure de sécurité.

Isolation au niveau des métadonnées

Une première approche d'isolation consiste à utiliser des métadonnées associées à chaque document et chaque chunk. Ces métadonnées contiennent des informations comme l'identifiant du tenant, le rôle utilisateur ou le niveau de classification du document.

Lors du **retrieval**, ces métadonnées sont utilisées comme filtres pour restreindre la recherche aux seules données autorisées pour l'utilisateur courant.

Isolation au niveau des index vectoriels

Dans certains systèmes **RAG** plus avancés, l'isolation est directement intégrée dans la structure de la base vectorielle. Chaque tenant peut disposer de son propre index ou de partitions logiques séparées au sein d'un même index.

Cette approche permet de réduire fortement les risques de fuite de données, tout en améliorant les performances du **retrieval** grâce à une réduction de l'espace de recherche.

Isolation logique vs isolation physique

L'isolation des données peut être mise en œuvre de manière logique ou physique. L'isolation logique repose sur des filtres et des règles appliquées au moment du **retrieval**, tandis que l'isolation physique repose sur la séparation réelle des données dans des bases ou des clusters distincts.

L'isolation physique offre un niveau de sécurité plus élevé, mais elle est plus coûteuse et plus complexe à maintenir. L'isolation logique est plus flexible, mais nécessite une vigilance accrue pour éviter les erreurs de filtrage.

Gestion du multi-tenancy

Dans un système **RAG multi-tenant**, l'isolation des données est directement liée à la gestion des tenants. Chaque tenant représente une entité indépendante avec ses propres documents, utilisateurs et politiques d'accès.

Le système doit garantir que toutes les opérations d'ingestion, de **retrieval** et de génération respectent strictement cette séparation.

Impact sur le pipeline RAG

L'isolation des données influence l'ensemble du **pipeline RAG**. Lors de l'ingestion, les documents doivent être correctement étiquetés avec les métadonnées d'isolation. Lors du **retrieval**, ces métadonnées doivent être utilisées comme filtres obligatoires. Lors de la génération, le contexte transmis au **LLM** doit être strictement limité aux données autorisées.

Cette cohérence est essentielle pour garantir une isolation efficace de bout en bout.

Risques de fuite de données

Les systèmes **RAG** mal conçus peuvent présenter des risques de fuite de données, notamment lorsque les filtres de métadonnées sont mal appliqués ou contournés. Les erreurs dans la configuration du **retrieval** peuvent entraîner la récupération de documents appartenant à un autre tenant.

Ces risques rendent indispensable une validation stricte des mécanismes d'isolation et des tests de sécurité réguliers.

Conclusion

L'isolation des données est un pilier essentiel des systèmes **RAG** en production, en particulier dans les environnements multi-utilisateurs. Elle garantit la confidentialité, la sécurité et la conformité des données tout en permettant un partage efficace de l'infrastructure.

Une isolation bien conçue permet de concilier performance et sécurité, tout en évitant les risques critiques de fuite d'informations.

Chapitre 17 — Multi-tenant RAG

Les systèmes **RAG** modernes ne sont pas uniquement conçus pour un usage mono-utilisateur ou interne. Dans un contexte industriel, ils doivent souvent servir plusieurs organisations, équipes ou clients au sein d'une même infrastructure. Cette capacité, appelée multi-tenancy, impose des contraintes fortes sur l'architecture, la sécurité et la gestion des données.

Section 17.1 — Architecture multi-utilisateurs

L'architecture multi-utilisateurs d'un système **RAG** consiste à concevoir une plateforme capable de servir simultanément plusieurs utilisateurs, équipes ou organisations, tout en garantissant une isolation stricte de leurs données et de leurs requêtes. Cette architecture est un passage obligé dès lors qu'un système RAG est déployé comme produit ou service partagé.

Principe général de l'architecture multi-utilisateurs

Dans une architecture multi-utilisateurs, un même système **RAG** est utilisé par plusieurs entités indépendantes. Chaque requête envoyée à l'API doit donc être contextualisée afin de déterminer à quel utilisateur ou organisation elle appartient.

Ce contexte est ensuite utilisé tout au long du **pipeline RAG** pour filtrer les données, adapter les résultats de **retrieval** et garantir que la génération de réponse reste cohérente avec l'environnement de l'utilisateur.

Mutualisation et séparation logique

L'un des principes fondamentaux de cette architecture repose sur la mutualisation des ressources. Les composants lourds du système, comme la base vectorielle, les services d'**embedding** ou les modèles de langage, sont partagés entre tous les utilisateurs afin d'optimiser les coûts et la performance globale.

Cependant, cette mutualisation est toujours accompagnée d'une séparation logique stricte des données. Même si les ressources sont partagées, chaque utilisateur ne doit pouvoir accéder qu'à son propre espace documentaire.

Gestion du contexte utilisateur

Chaque requête dans un système multi-utilisateurs est enrichie d'un contexte utilisateur. Ce contexte contient des informations essentielles comme l'identifiant du tenant, les rôles associés, les permissions d'accès et parfois des paramètres de personnalisation.

Ce contexte est transmis à travers l'ensemble du **pipeline RAG** et influence directement le comportement du **retrieval**, du **reranking** et de la génération.

Isolation au niveau applicatif

L'isolation dans une architecture multi-utilisateurs est principalement réalisée au niveau applicatif. Cela signifie que les règles d'accès sont appliquées dans l'API et dans les services de traitement plutôt que dans une séparation physique complète des infrastructures.

Cette approche permet de conserver une grande flexibilité tout en réduisant les coûts d'infrastructure.

scalabilité du système multi-utilisateurs

L'architecture multi-utilisateurs doit être conçue pour supporter une montée en charge importante. Plusieurs utilisateurs peuvent interroger le système simultanément, ce qui nécessite une gestion efficace des ressources et une capacité de traitement parallèle.

Cette **scalabilité** repose généralement sur des services stateless, une répartition des requêtes via load balancing et une architecture distribuée des composants du **pipeline RAG**.

Implications sur le **pipeline RAG**

Le caractère multi-utilisateurs du système influence directement le **pipeline RAG**. Chaque étape, de l'ingestion à la génération, doit prendre en compte le contexte utilisateur afin de garantir la cohérence des résultats.

Le **retrieval** doit être filtré en fonction du tenant, le **reranking** doit respecter les mêmes contraintes, et le **LLM** doit générer des réponses basées uniquement sur les données autorisées.

Complexité de conception

La mise en place d'une architecture multi-utilisateurs introduit une complexité supplémentaire importante. Il devient nécessaire de gérer simultanément la performance, la sécurité, la cohérence des données et la **scalabilité**.

Cette complexité est principalement liée à la nécessité de maintenir une séparation stricte des données tout en partageant une infrastructure commune.

Conclusion

L'architecture multi-utilisateurs constitue une base essentielle des systèmes **RAG** industriels. Elle permet de transformer un système initialement conçu pour un usage isolé en une plateforme scalable capable de servir plusieurs organisations de manière simultanée.

Sa conception repose sur un équilibre entre mutualisation des ressources et isolation logique des données, garantissant ainsi à la fois efficacité économique et sécurité fonctionnelle.

Section 17.2 — Séparation des index

La séparation des index est un mécanisme central dans les systèmes **RAG multi-tenant** en production. Elle garantit que les données vectorielles appartenant à différents utilisateurs ou

organisations ne se mélangent pas lors des opérations de **retrieval**. Sans cette séparation, un système RAG exposé à plusieurs clients devient rapidement vulnérable à des fuites de données et à des résultats incohérents.

Objectif de la séparation des index

L'objectif principal de la séparation des index est de garantir une isolation forte des espaces de recherche. Chaque requête utilisateur doit interroger uniquement les **embeddings** qui correspondent à son contexte d'accès.

Cela permet de maintenir la confidentialité des données tout en conservant les performances du **retrieval** vectoriel.

Approche par index dédiés

Une première stratégie consiste à attribuer un index vectoriel distinct à chaque tenant. Dans ce modèle, chaque organisation dispose de son propre espace de stockage et de recherche indépendant.

Cette approche simplifie fortement la logique de filtrage, car l'isolation est directement garantie par la structure de la base de données. Elle est particulièrement adaptée aux systèmes où le nombre de tenants est limité ou où les exigences de sécurité sont élevées.

Approche par index partagé avec filtres

Une alternative plus scalable consiste à utiliser un index unique partagé entre tous les tenants, en ajoutant des métadonnées pour chaque vecteur. Ces métadonnées incluent notamment un identifiant de tenant qui est utilisé comme filtre lors du **retrieval**.

Dans ce modèle, l'isolation est assurée par la couche applicative plutôt que par la structure de l'index lui-même. Cette approche est plus efficace en termes de ressources, mais elle exige une rigueur absolue dans l'application des filtres.

Partitionnement et sharding

Entre les deux approches extrêmes, certains systèmes utilisent un partitionnement des index. Les données sont réparties en shards, chacun contenant un sous-ensemble des vecteurs.

Ce partitionnement peut être basé sur des critères comme le tenant, la catégorie de documents ou la fréquence d'accès. Il permet d'optimiser à la fois les performances et l'isolation logique.

Impact sur la performance du retrieval

La stratégie de séparation des index a un impact direct sur la performance du système **RAG**. Des index dédiés réduisent la taille de l'espace de recherche, ce qui améliore la latence du **retrieval**.

À l'inverse, les index partagés nécessitent des filtres supplémentaires qui peuvent augmenter légèrement le coût de recherche, mais permettent une meilleure utilisation des ressources globales.

Gestion de la cohérence des données

La séparation des index implique également une gestion rigoureuse de la cohérence des données. Lors de l'ingestion, chaque document doit être correctement attribué à un index ou à un tenant spécifique.

Toute erreur dans cette attribution peut entraîner des fuites de données ou des incohérences dans les résultats de recherche.

Choix d'architecture

Le choix entre index dédiés, index partagés ou partitionnés dépend principalement des contraintes du système. Les systèmes à forte exigence de sécurité privilégient souvent des index séparés, tandis que les systèmes à grande échelle favorisent des architectures partagées avec filtrage.

Ce choix doit être aligné avec les objectifs de performance, de coût et de conformité.

Conclusion

La séparation des index est un élément fondamental de l'architecture **RAG multi-tenant**. Elle permet de garantir l'isolation des données tout en maintenant des performances élevées dans les opérations de **retrieval**.

Une conception rigoureuse de cette séparation est indispensable pour assurer la sécurité et la **scalabilité** d'un système **RAG** en production.

Section 17.3 — Gestion des permissions

La gestion des permissions est un élément central des systèmes **RAG multi-tenant** en production. Elle définit précisément quelles actions un utilisateur peut effectuer et quelles données il est autorisé à consulter. Dans une architecture RAG, cette notion est critique car elle s'applique à toutes les étapes du **pipeline**, depuis l'ingestion des documents jusqu'à la génération des réponses par le **LLM**.

Rôle des permissions dans un système RAG

Les permissions servent à contrôler l'accès aux ressources du système **RAG**. Elles garantissent qu'un utilisateur ne peut interroger que les documents auxquels il a droit et que les réponses générées ne contiennent pas d'informations sensibles issues d'autres espaces utilisateurs.

Sans un système de permissions robuste, la mutualisation des données dans un environnement **multi-tenant** devient un risque majeur de fuite d'information.

Modèle basé sur les rôles (RBAC)

Le contrôle d'accès basé sur les rôles est l'un des modèles les plus utilisés dans les systèmes **RAG**. Chaque utilisateur se voit attribuer un ou plusieurs rôles, et chaque rôle possède un ensemble de permissions définissant les actions autorisées.

Par exemple, un administrateur peut avoir accès à l'ensemble des documents et des opérations, tandis qu'un utilisateur standard est limité à la consultation de son propre espace documentaire.

Permissions au niveau des données

Les permissions peuvent également être appliquées directement au niveau des documents ou des chunks. Chaque élément indexé dans la base vectorielle peut être associé à des règles d'accès spécifiques.

Lors du **retrieval**, ces règles sont utilisées pour filtrer les résultats afin de ne retourner que les documents autorisés pour l'utilisateur courant.

Application dans le pipeline RAG

Dans un système **RAG**, les permissions doivent être appliquées de manière cohérente à toutes les étapes du **pipeline**. Lors de l'ingestion, elles déterminent dans quel espace de stockage les documents sont indexés. Lors du **retrieval**, elles filtrent les résultats de recherche. Lors de la génération, elles garantissent que le contexte transmis au **LLM** respecte strictement les règles d'accès.

Cette cohérence est indispensable pour éviter toute fuite de données entre utilisateurs.

Gestion dynamique des permissions

Dans certains systèmes avancés, les permissions ne sont pas statiques mais peuvent évoluer dynamiquement. Un utilisateur peut obtenir ou perdre des droits en fonction de son rôle, de son organisation ou de politiques internes.

Le système **RAG** doit être capable de prendre en compte ces changements en temps réel, notamment lors des opérations de **retrieval**.

Audit et traçabilité des accès

La gestion des permissions doit être accompagnée d'un système d'audit complet. Chaque accès à un document ou à un résultat de **retrieval** doit être tracé afin de pouvoir reconstituer l'historique des actions.

Cette traçabilité est essentielle pour des raisons de sécurité, de conformité et de diagnostic en cas d'incident.

Sécurité et prévention des fuites

Une mauvaise implémentation des permissions peut entraîner des fuites de données critiques dans un système **RAG**. Ces fuites peuvent survenir si les filtres sont mal appliqués, si les métadonnées sont incorrectes ou si des chemins d'accès échappent au contrôle du système.

Il est donc indispensable de tester rigoureusement les mécanismes de permission et de les valider à chaque évolution du système.

Conclusion

La gestion des permissions est un pilier essentiel des systèmes **RAG multi-tenant**. Elle garantit que chaque utilisateur n'accède qu'aux données qui lui sont autorisées, tout en permettant une mutualisation efficace des ressources.

Une conception rigoureuse des permissions est indispensable pour assurer la sécurité, la conformité et la fiabilité d'un système **RAG** en production.

Chapitre 18 — Résilience et tolérance aux pannes

Un système **RAG** en production ne peut pas être considéré comme fiable uniquement parce qu'il fonctionne dans des conditions idéales. Dans un environnement réel, les services peuvent ralentir, les bases de données peuvent devenir indisponibles, les modèles peuvent répondre avec retard, et les réseaux peuvent introduire des latences imprévisibles. La résilience désigne la capacité du système à continuer de fonctionner malgré ces perturbations.

La tolérance aux pannes est donc une propriété essentielle des architectures **RAG** industrielles. Elle permet de garantir que le service reste disponible, même lorsque certains composants internes échouent ou dégradent temporairement leurs performances. Cette approche repose sur des mécanismes de récupération automatique, de redondance et de dégradation contrôlée du système.

Section 18.1 — Retry logic

La retry logic, ou logique de relance, est un mécanisme fondamental de résilience dans un système **RAG**. Elle consiste à réexécuter automatiquement une opération lorsqu'une première tentative échoue, afin de compenser les erreurs temporaires liées au réseau, à la charge système ou à des indisponibilités ponctuelles d'un service.

Dans un **pipeline RAG**, les appels au **retrieval**, au **reranking** ou au modèle de langage peuvent tous être sujets à des erreurs transitoires. La retry logic permet de masquer une partie de ces instabilités en réessayant l'opération après un court délai.

Gestion des échecs temporaires

La majorité des erreurs rencontrées dans un système distribué ne sont pas permanentes, mais temporaires. Une surcharge de la base vectorielle, une latence excessive du **LLM** ou une interruption réseau peuvent provoquer des échecs ponctuels.

La retry logic exploite ce caractère temporaire en réessayant l'opération après un délai calculé. Ce délai peut être fixe ou augmenter progressivement afin d'éviter de surcharger encore davantage le système.

Stratégies de retry

Différentes stratégies de retry peuvent être mises en place dans un système **RAG**. La stratégie la plus simple consiste à effectuer un nombre fixe de tentatives. Des approches plus avancées utilisent un backoff exponentiel, où le délai entre chaque tentative augmente progressivement.

Cette approche permet de réduire la pression sur les services défaillants tout en augmentant les chances de succès global de l'opération.

Impact sur le pipeline RAG

La retry logic doit être intégrée de manière cohérente dans l'ensemble du **pipeline RAG**. Elle ne doit pas être appliquée uniquement au niveau de l'API, mais également aux appels internes vers le **retrieval**, les bases vectorielles et les modèles de **reranking**.

Une mauvaise implémentation peut entraîner des effets de cascade, où plusieurs retries simultanés aggravent la surcharge du système.

Limites de la retry logic

Bien que la retry logic améliore la robustesse, elle ne résout pas les problèmes structurels de capacité ou de défaillance prolongée. Si un service est durablement indisponible, les retries deviennent inefficaces et peuvent même détériorer la performance globale du système.

Elle doit donc être combinée avec d'autres mécanismes de résilience comme le failover et la dégradation contrôlée.

Section 18.2 — Failover retrieval

Le failover **retrieval** est un mécanisme de résilience qui permet à un système **RAG** de continuer à fonctionner même lorsqu'un composant critique du retrieval devient indisponible ou dégradé. Dans une architecture RAG en production, la couche de retrieval est particulièrement sensible, car elle conditionne directement la qualité des réponses générées par le **LLM**.

Principe du failover

Le principe du failover repose sur la redondance des mécanismes de recherche. Lorsqu'un service de **retrieval** principal échoue ou dépasse un seuil de latence, le système bascule automatiquement vers une solution alternative afin de garantir la continuité du service.

Cette bascule peut être transparente pour l'utilisateur, ce qui permet de maintenir une expérience stable même en cas de panne partielle du système.

Stratégies de failover

Plusieurs stratégies de failover peuvent être mises en place dans un système **RAG**. La première consiste à utiliser une base vectorielle secondaire qui prend le relais en cas de défaillance de la base principale. Cette redondance permet de garantir une continuité de service minimale.

Une autre stratégie consiste à basculer vers une recherche hybride ou lexicale lorsque la recherche vectorielle devient indisponible, ce qui permet de conserver une capacité de réponse même en mode dégradé.

Détection des défaillances

Le failover repose sur un mécanisme de détection des anomalies. Le système doit être capable d'identifier rapidement une dégradation du **retrieval**, que ce soit en termes de latence, de taux d'erreur ou d'absence de résultats pertinents.

Cette détection peut s'appuyer sur des seuils de performance ou sur des systèmes de monitoring en temps réel.

Basculement automatique

Une fois une défaillance détectée, le système doit déclencher un basculement automatique vers une solution de secours. Ce processus doit être rapide et transparent afin de minimiser l'impact sur l'utilisateur final.

Le basculement peut impliquer un changement de source de données, une réduction du volume de recherche ou une modification de la stratégie de ranking.

Cohérence des résultats

L'un des défis du failover **retrieval** est de maintenir une cohérence acceptable des résultats. Les systèmes de secours peuvent ne pas offrir la même qualité que le système principal, ce qui peut entraîner une dégradation des réponses générées.

Il est donc important de concevoir des stratégies de failover qui privilégient la continuité du service tout en maintenant un niveau minimal de pertinence.

Impact sur l'expérience utilisateur

Le failover **retrieval** est généralement invisible pour l'utilisateur, mais il peut influencer indirectement la qualité des réponses. Une bonne implémentation permet de garantir une expérience fluide, même en cas de panne partielle de l'infrastructure.

À l'inverse, un failover mal conçu peut entraîner des incohérences, des réponses incomplètes ou une augmentation de la latence.

Conclusion

Le failover **retrieval** est un mécanisme essentiel pour garantir la disponibilité d'un système **RAG** en production. Il permet de maintenir un niveau de service acceptable même en cas de défaillance d'un composant critique du **pipeline** de recherche.

Combiné à d'autres mécanismes de résilience, il contribue à rendre le système robuste, tolérant aux pannes et adapté à des environnements industriels exigeants.

Section 18.3 — Dégradation contrôlée

La dégradation contrôlée est un mécanisme de résilience qui permet à un système **RAG** de continuer à répondre aux utilisateurs lorsque ses ressources ou certains composants deviennent insuffisants. Plutôt que de s'effondrer complètement sous la charge ou en cas de panne partielle, le système réduit volontairement la qualité de certaines étapes du **pipeline** afin de préserver la

disponibilité globale du service.

Principe de la dégradation contrôlée

Le principe repose sur une idée simple : il est préférable de fournir une réponse partiellement dégradée que de ne pas fournir de réponse du tout. Dans un système **RAG**, cela signifie adapter dynamiquement le comportement du **pipeline** en fonction de la charge, de la latence ou de la disponibilité des services.

Cette adaptation peut affecter le **retrieval**, le **reranking** ou même la génération, en réduisant leur complexité ou leur profondeur de traitement.

Réduction du scope de retrieval

Une première forme de dégradation contrôlée consiste à réduire le volume de recherche dans la base vectorielle. Le système peut diminuer le paramètre **Top-K**, limiter les shards interrogés ou restreindre les filtres de recherche.

Cette réduction permet d'accélérer le **retrieval** et de diminuer la charge sur la base de données, au prix d'une perte potentielle de recall.

Désactivation du reranking

Lorsque les ressources sont fortement contraintes, le **reranking** peut être désactivé temporairement. Dans ce mode dégradé, le système utilise directement les résultats bruts du **retrieval** pour alimenter le **LLM**.

Cette approche réduit considérablement la latence, mais peut impacter la pertinence des réponses générées.

Simplification du modèle de génération

Dans certains cas, la dégradation contrôlée peut également s'appliquer au modèle de langage. Le système peut basculer vers un modèle plus léger ou réduire la taille du contexte envoyé au **LLM**.

Cela permet de maintenir une capacité de réponse même lorsque les ressources GPU sont saturées ou indisponibles.

Priorisation des fonctionnalités critiques

La dégradation contrôlée implique également une hiérarchisation des fonctionnalités du **pipeline RAG**. Les étapes essentielles comme le **retrieval** de base et la génération de réponse sont

maintenues en priorité, tandis que les optimisations avancées comme le **reranking** ou les enrichissements contextuels sont désactivées en premier.

Cette hiérarchisation garantit que le système reste fonctionnel même dans des conditions extrêmes.

Déclenchement automatique

La dégradation contrôlée est généralement déclenchée automatiquement en fonction de métriques système. Lorsque la latence dépasse un seuil critique ou que les ressources sont saturées, le système active progressivement des modes dégradés.

Ce mécanisme permet une adaptation continue du système à la charge sans intervention humaine.

Impact sur la qualité des réponses

La dégradation contrôlée entraîne nécessairement une baisse de la qualité des réponses générées. Cependant, cette baisse est volontaire et encadrée afin de préserver la continuité du service.

L'objectif est de trouver un équilibre entre performance et disponibilité, en évitant les interruptions complètes du système.

Conclusion

La dégradation contrôlée est un mécanisme essentiel dans les systèmes **RAG** en production. Elle permet de maintenir un service fonctionnel même en situation de contrainte forte, en adaptant dynamiquement la complexité du **pipeline**.

Combinée au retry logic et au failover **retrieval**, elle constitue un pilier fondamental de la résilience des architectures **RAG** industrielles.

Chapitre 19 — Qualité des réponses LLM

Dans un système **RAG** en production, la qualité finale perçue par l'utilisateur ne dépend pas uniquement de la qualité du **retrieval** ou de la base vectorielle. Elle dépend principalement de la capacité du modèle de langage à exploiter correctement les informations récupérées pour produire une réponse fidèle, utile et cohérente.

Le **LLM** agit comme une couche de synthèse entre les documents récupérés et l'utilisateur final. C'est à ce niveau que se jouent les problèmes les plus critiques : **hallucinations**, perte de contexte, mauvaise interprétation des sources ou génération de réponses non fondées. Garantir la qualité des réponses LLM est donc une étape centrale de toute architecture **RAG** industrielle.

Section 19.1 — Prompt engineering structuré

Le prompt engineering structuré consiste à concevoir des instructions claires, organisées et contraintes pour guider le comportement du modèle de langage dans un système **RAG**. Contrairement à un prompt libre ou conversationnel, un prompt structuré impose une forme de rigueur dans l'utilisation du contexte récupéré et dans la génération de la réponse.

Rôle du prompt dans un système RAG

Dans un **pipeline RAG**, le prompt est l'interface entre le **retrieval** et le **LLM**. Il contient généralement la question de l'utilisateur, les documents ou chunks récupérés, ainsi que des instructions explicites sur la manière de produire la réponse.

Un prompt mal conçu peut conduire le modèle à ignorer les sources, à mélanger les informations ou à halluciner des contenus inexistantes. À l'inverse, un prompt bien structuré améliore fortement la fidélité et la stabilité des réponses.

Structuration du contexte

Une approche essentielle du prompt engineering structuré consiste à organiser le contexte fourni au **LLM**. Les documents récupérés doivent être clairement séparés, étiquetés et hiérarchisés afin de faciliter leur interprétation.

Chaque chunk peut être identifié avec une source, un identifiant ou un score de pertinence, ce qui permet au modèle de distinguer les informations les plus fiables.

Instructions explicites au modèle

Le prompt doit contenir des instructions précises sur le comportement attendu du modèle. Cela inclut des règles comme l'obligation de se baser uniquement sur le contexte fourni, l'interdiction d'inventer des informations ou la nécessité de privilégier les sources les plus pertinentes.

Ces instructions réduisent fortement le risque d'hallucination et améliorent la cohérence globale des réponses.

Séparation entre question et contexte

Une bonne pratique du prompt engineering structuré consiste à séparer clairement la question de l'utilisateur et les documents récupérés. Cette séparation permet d'éviter que le modèle mélange la requête avec les données sources.

Le contexte est généralement présenté sous une forme délimitée, ce qui aide le modèle à identifier les informations exploitables pour la réponse.

Contrôle de la sortie

Le prompt peut également imposer une structure de sortie, par exemple en demandant une réponse avec des sections, des listes ou des citations obligatoires. Ce contrôle permet d'uniformiser les réponses et de faciliter leur exploitation en aval.

Dans les systèmes **RAG** industriels, cette standardisation est souvent essentielle pour garantir une expérience utilisateur cohérente.

Limites du prompt engineering

Bien que le prompt engineering structuré améliore fortement la qualité des réponses, il ne suffit pas à lui seul à éliminer les **hallucinations** ou les erreurs de raisonnement. Il doit être combiné avec des techniques de **retrieval** robustes, du **reranking** et des mécanismes de grounding strict.

Le prompt est une couche de contrôle, mais il ne remplace pas la qualité des données fournies au modèle.

Conclusion

Le prompt engineering structuré est un élément fondamental de la qualité des réponses dans un système **RAG**. Il permet de guider le **LLM**, de structurer l'information et de réduire les comportements imprévisibles.

Une conception rigoureuse des prompts est indispensable pour transformer un système **RAG** technique en un produit fiable et exploitable en production.

Section 19.2 — Réduction des hallucinations

La réduction des **hallucinations** est l'un des défis les plus critiques dans les systèmes **RAG** en production. Une hallucination se produit lorsque le modèle de langage génère une information qui n'est pas supportée par les données fournies dans le contexte. Dans une architecture RAG, ce phénomène est particulièrement problématique car le système est justement conçu pour ancrer les réponses dans des sources externes fiables.

Origine des hallucinations

Les **hallucinations** proviennent principalement de la nature probabiliste des modèles de langage. Même lorsqu'un contexte pertinent est fourni, le **LLM** peut combler des manques d'information en générant du contenu plausible mais incorrect.

Dans un système **RAG**, cela peut être aggravé par un **retrieval** incomplet, un mauvais **chunking** ou un contexte mal structuré. Si les informations nécessaires ne sont pas présentes dans le contexte, le modèle a tendance à "deviner" la réponse.

Amélioration du grounding

La première stratégie pour réduire les **hallucinations** consiste à améliorer le grounding, c'est-à-dire la capacité du modèle à s'appuyer strictement sur les documents fournis. Cela passe par un prompt engineering rigoureux, qui interdit explicitement l'invention d'informations non présentes dans le contexte.

Le modèle doit être encouragé à répondre uniquement à partir des sources disponibles et à indiquer lorsqu'une information est absente.

Qualité du retrieval

La réduction des **hallucinations** dépend fortement de la qualité du **retrieval**. Un système qui ne récupère pas les bons documents ne peut pas espérer produire des réponses fiables, même avec un prompt bien conçu.

L'amélioration du recall, l'utilisation de **reranking** et la recherche hybride sont donc des leviers essentiels pour réduire les **hallucinations** à la source.

Structuration du contexte

Un contexte mal structuré augmente fortement le risque d'hallucination. Lorsque les documents sont mal séparés ou trop longs, le modèle peut mélanger les informations ou perdre la distinction entre les sources.

Une structuration claire du contexte permet au modèle d'identifier précisément quelles informations sont pertinentes pour la question posée.

Contraintes sur la génération

Une autre approche consiste à imposer des contraintes strictes sur la génération de texte. Le modèle peut être limité à répondre uniquement en utilisant des informations explicitement présentes dans le contexte, sans ajout externe.

Dans certains systèmes, la génération est également contrainte à produire des citations obligatoires pour chaque affirmation importante, ce qui réduit indirectement les **hallucinations**.

Détection des hallucinations

Dans les systèmes **RAG** avancés, il est possible de mettre en place des mécanismes de détection des **hallucinations**. Ces systèmes analysent la réponse générée et vérifient si chaque affirmation est bien supportée par les documents sources.

Cette vérification peut être réalisée via des modèles secondaires ou des règles heuristiques basées sur la similarité sémantique.

Compromis entre créativité et fiabilité

La réduction des **hallucinations** implique souvent un compromis entre créativité et fiabilité. Un modèle trop contraint peut produire des réponses rigides ou incomplètes, tandis qu'un modèle trop libre augmente le risque d'erreur.

Dans un système **RAG** industriel, la priorité est généralement donnée à la fiabilité plutôt qu'à la créativité.

Conclusion

La réduction des **hallucinations** est un enjeu central dans la conception de systèmes **RAG** fiables. Elle repose sur une combinaison de techniques incluant l'amélioration du **retrieval**, la structuration du contexte, le prompt engineering et des mécanismes de contrôle de génération.

Un système **RAG** performant n'est pas celui qui génère les réponses les plus fluides, mais celui qui garantit que chaque information produite est correctement ancrée dans des sources vérifiables.

Section 19.3 — Réponses avec citations

Les réponses avec citations constituent un mécanisme essentiel pour renforcer la fiabilité et la traçabilité des systèmes **RAG** en production. Elles consistent à associer explicitement chaque information générée par le modèle de langage à une ou plusieurs sources issues du **retrieval**. Cette approche transforme la sortie du **LLM** en une réponse vérifiable, plutôt qu'en une simple génération textuelle.

Objectif des citations dans un système RAG

L'objectif principal des citations est de garantir que chaque affirmation importante produite par le modèle puisse être reliée à un document source. Cela permet à l'utilisateur de vérifier la provenance de l'information et d'évaluer sa fiabilité.

Dans un contexte industriel, cette transparence est essentielle pour réduire les risques d'erreurs critiques et renforcer la confiance dans le système.

Intégration des sources dans le prompt

Pour obtenir des réponses avec citations, le prompt doit inclure les documents récupérés sous une forme structurée, accompagnés d'identifiants clairs. Ces identifiants servent de référence pour relier les passages générés aux sources originales.

Le modèle est ensuite instruit de citer systématiquement les sources utilisées pour chaque élément de réponse, en respectant une structure cohérente.

Granularité des citations

La granularité des citations peut varier selon les besoins du système. Dans certains cas, une citation globale par paragraphe suffit, tandis que dans des systèmes plus exigeants, chaque phrase peut être associée à une source précise.

Plus la granularité est fine, plus la traçabilité est forte, mais plus la complexité de génération augmente.

Alignement entre retrieval et génération

La qualité des citations dépend directement de l'alignement entre les documents récupérés et la réponse générée. Si le **retrieval** est imprécis ou incomplet, les citations risquent d'être incorrectes ou trompeuses.

Il est donc essentiel que le **pipeline RAG** assure une cohérence forte entre les sources fournies et le contenu généré.

Vérification des citations

Dans les systèmes avancés, les citations peuvent être vérifiées automatiquement après génération. Un module de contrôle peut comparer les affirmations du modèle avec les documents sources pour s'assurer que les citations sont correctes et pertinentes.

Ce processus de validation renforce la robustesse du système et réduit les erreurs de correspondance entre texte et sources.

Impact sur l'expérience utilisateur

Les réponses avec citations améliorent fortement l'expérience utilisateur en rendant le système plus transparent et explicable. L'utilisateur peut consulter les sources directement et vérifier la validité des informations fournies.

Cela est particulièrement important dans des domaines sensibles comme le juridique, la santé ou la finance.

Contraintes de génération

La génération de réponses avec citations impose des contraintes supplémentaires au **LLM**. Le modèle doit non seulement produire une réponse cohérente, mais également maintenir une correspondance explicite avec les sources fournies.

Cette contrainte peut augmenter la complexité du prompt et nécessiter des modèles mieux entraînés ou des techniques de structuration avancées.

Conclusion

Les réponses avec citations constituent un élément fondamental des systèmes **RAG** en production. Elles permettent de renforcer la transparence, la fiabilité et la traçabilité des réponses générées par le **LLM**.

En reliant explicitement les réponses aux sources, elles transforment le système **RAG** en un outil explicable et exploitable dans des contextes professionnels exigeants.

Section 19.4 — Grounding strict

Le grounding strict est une approche de conception des systèmes **RAG** qui impose au modèle de langage de ne produire des réponses qu'à partir des informations explicitement fournies dans le contexte de **retrieval**. Il s'agit d'un niveau de contrainte supérieur aux simples réponses avec citations, car il vise à éliminer toute forme de génération libre non justifiée par des sources.

Dans un système **RAG** en production, le grounding strict est un mécanisme de sécurité et de fiabilité qui garantit que le **LLM** agit uniquement comme un moteur de synthèse, et non comme une source d'information autonome.

Principe du grounding strict

Le principe du grounding strict repose sur une règle simple : aucune information ne doit être générée si elle n'est pas présente dans les documents fournis au modèle. Le **LLM** n'a pas le droit

d'utiliser ses connaissances internes pour compléter ou inventer des éléments de réponse.

Cette contrainte transforme profondément le comportement du modèle, qui devient dépendant du **retrieval** pour toute information factuelle.

Encadrement du prompt

Le grounding strict est généralement mis en œuvre via un prompt fortement structuré qui interdit explicitement l'utilisation de connaissances externes. Le modèle est instruit de répondre uniquement à partir du contexte fourni et de signaler toute absence d'information.

Cette instruction doit être claire, non ambiguë et systématiquement appliquée dans toutes les requêtes du système **RAG**.

Dépendance totale au retrieval

Dans une configuration de grounding strict, la qualité du **retrieval** devient le facteur déterminant de la qualité des réponses. Si une information n'est pas récupérée, elle ne pourra pas être utilisée dans la génération.

Cela impose un niveau d'exigence très élevé sur le **chunking**, les **embeddings**, le ranking et la couverture de la base documentaire.

Réduction des hallucinations

Le grounding strict est l'une des méthodes les plus efficaces pour réduire les **hallucinations** dans les systèmes **RAG**. En empêchant le modèle d'utiliser ses connaissances internes, on limite fortement la possibilité de génération d'informations non vérifiées.

Cependant, cette approche peut également conduire à des réponses incomplètes si le **retrieval** est insuffisant.

Gestion des cas d'absence d'information

Un aspect important du grounding strict est la gestion des situations où aucune information pertinente n'est trouvée dans le contexte. Dans ce cas, le modèle doit explicitement indiquer l'absence de données plutôt que de tenter de compléter la réponse.

Cette transparence est essentielle pour maintenir la fiabilité du système.

Compromis entre flexibilité et sécurité

Le grounding strict impose un compromis entre flexibilité et sécurité. D'un côté, il améliore fortement la fiabilité des réponses. De l'autre, il limite la capacité du modèle à généraliser ou à inférer des informations implicites.

Dans les systèmes **RAG** industriels, ce compromis est généralement accepté au profit de la fiabilité.

Conclusion

Le grounding strict constitue une approche avancée de contrôle des systèmes **RAG**, visant à garantir que toutes les réponses produites sont entièrement ancrées dans des sources vérifiables.

En combinant cette approche avec le **retrieval**, le **reranking** et les réponses avec citations, il est possible de construire des systèmes **RAG** hautement fiables, adaptés à des environnements professionnels exigeants.

Partie V — Industrialisation avancée du RAG

Chapitre 20 — Systèmes de production à grande échelle

Le passage d'un prototype **RAG** à une plateforme industrielle ne consiste pas uniquement à ajouter davantage de documents. Lorsque le corpus dépasse plusieurs dizaines de millions de passages, de nouveaux problèmes apparaissent : temps de recherche, mémoire, coûts GPU, synchronisation des index, disponibilité, mises à jour continues et résilience aux pannes.

Les systèmes modernes reposent donc sur une architecture distribuée où plusieurs composants collaborent afin de garantir une faible latence tout en conservant une excellente qualité de recherche.

Section 20.1 — Millions de documents

Pourquoi le changement d'échelle est difficile

Un **RAG** développé localement fonctionne généralement sur quelques milliers ou centaines de milliers de documents.

À cette échelle :

- tout tient facilement en mémoire ;
- un seul index vectoriel suffit ;
- les mises à jour sont rares ;
- la recherche reste rapide.

Lorsque le volume atteint plusieurs millions de documents, la situation change complètement.

On rencontre alors plusieurs difficultés :

- mémoire insuffisante ;
- index gigantesques ;
- augmentation du temps de recherche ;

- coût du stockage vectoriel ;
- temps de reconstruction des index.

Par exemple :

- Ces chiffres concernent uniquement les vecteurs, sans compter :
- le texte original,
- les métadonnées,
- les index **ANN**,
- les sauvegardes.

On comprend rapidement qu'une simple base FAISS locale devient insuffisante.

Découpage des documents

Les documents volumineux ne sont jamais indexés tels quels.

Ils sont découpés en morceaux (chunks).

Par exemple :

- Livre

↓

Chapitre

↓

Section

↓

Paragraphe

↓

Chunks de 300–600 tokens

Chaque chunk devient une unité indépendante.

Chaque unité possède :

- son **embedding** ;
- son identifiant ;
- ses métadonnées ;

- son texte.

Cette granularité améliore fortement la précision du **RAG**.

Explosion du nombre de chunks

Un corpus peut sembler modeste mais produire un très grand nombre de vecteurs.

Exemple :

- 500 000 PDF

↓

200 pages par PDF

↓

10 chunks par page

↓

≈ 1 milliard de chunks

Le système ne recherche donc plus parmi quelques centaines de milliers de documents mais parmi plusieurs centaines de millions de vecteurs.

Stratégies de réduction

Pour limiter cette croissance, plusieurs techniques sont utilisées :

Chunking intelligent

Au lieu de découper tous les 500 caractères :

- respect des titres
- respect des paragraphes
- respect des listes
- respect des tableaux

Le contexte est mieux conservé.

Suppression des doublons

Les grandes entreprises possèdent souvent :

- plusieurs versions d'un même document ;
- des copies ;
- des archives ;
- des sauvegardes.

Avant l'indexation, un système de déduplication retire les contenus identiques.

Filtrage

Tous les documents ne méritent pas d'être indexés.

On peut ignorer :

- les pages blanches ;
- les signatures ;
- les pieds de page ;
- les menus ;
- les publicités.

Cela réduit fortement le nombre de chunks.

Compression des embeddings

Les vecteurs float32 peuvent être compressés.

Par exemple :

- float16
- int8

Product Quantization (PQ)

Scalar Quantization

On peut diviser l'espace mémoire par 4 à 16 avec une perte de précision limitée.

Ingestion continue

Dans une entreprise, le corpus évolue en permanence.

Chaque jour arrivent :

- nouveaux PDF ;
- nouveaux tickets ;
- nouvelles FAQ ;
- nouvelles procédures ;
- nouveaux contrats.

Le **pipeline RAG** fonctionne donc en continu :

- Nouveau document

↓

Extraction

↓

Nettoyage

↓

chunking

↓

embeddings

↓

Indexation

↓

Disponible pour les utilisateurs

Les index ne sont pratiquement jamais reconstruits intégralement.

Section 20.2 — Sharding vectoriel

Pourquoi répartir les vecteurs ?

À partir de plusieurs dizaines de millions de vecteurs, un seul serveur devient rapidement insuffisant.

Les limites apparaissent sur plusieurs ressources :

- mémoire RAM ;
- mémoire GPU ;
- puissance CPU ;
- bande passante disque ;
- temps de réponse.

Le **sharding** vectoriel consiste à répartir les **embeddings** sur plusieurs machines afin de partager la charge.

Principe général

Au lieu d'un unique index :

- Serveur unique

[Tous les vecteurs]

- on obtient :
- Shard 1

Vecteurs A

Shard 2

Vecteurs B

Shard 3

Vecteurs C

Shard 4

Vecteurs D

Chaque serveur ne contient qu'une partie du corpus.

Types de sharding

Sharding aléatoire

Les documents sont répartis de façon uniforme.

Avantages :

- équilibrage simple ;
- charge homogène.

Inconvénient :

- chaque requête doit interroger tous les shards.

Sharding par domaine

Exemple :

- Finance

RH

Juridique

Support

Marketing

Une question juridique n'interrogera que le shard juridique.

Le coût diminue fortement.

Sharding géographique

Utilisé pour les multinationales.

Exemple :

- Europe

Amérique

Asie

Afrique

Chaque région conserve ses propres données.

Sharding sémantique

Les documents proches sont regroupés grâce au clustering (k-means, HDBSCAN, etc.).

Les requêtes sont dirigées uniquement vers les groupes les plus pertinents.

Cette approche réduit considérablement le nombre de comparaisons.

Router de requêtes

Avant toute recherche, un composant décide quels shards interroger.

Question

↓

Router

↓

Shard A

Shard C

↓

Fusion des résultats

Ce routeur peut utiliser :

- des règles métiers ;
- des métadonnées ;
- un petit modèle de classification ;
- un **LLM** léger.

Avantages

Le **sharding** permet :

- d'augmenter la capacité totale ;
- de réduire la latence ;
- d'ajouter facilement de nouveaux serveurs ;
- d'améliorer la tolérance aux pannes.

Section 20.3 — Index distribués

Pourquoi distribuer les index ?

Lorsque le volume de données dépasse la capacité d'un seul serveur, il ne suffit plus de répartir les documents : les structures d'indexation elles-mêmes doivent être distribuées.

Chaque nœud conserve un sous-ensemble de l'index et participe à la recherche.

Architecture distribuée

Une architecture typique comprend :

- Clients

↓

API Gateway

↓

Router

↓

Cluster vectoriel

■■■ Nœud 1

■■■ Nœud 2

■■■ Nœud 3

■■■ Nœud 4

↓

Fusion des résultats

↓

LLM

Le routeur diffuse la requête, collecte les meilleurs candidats de chaque nœud, puis agrège les résultats avant leur transmission au modèle génératif.

Réplication

Pour assurer la disponibilité, chaque shard est généralement répliqué.

Shard A

↓

Copie 1

Copie 2

Copie 3

Si un serveur tombe en panne, un autre prend immédiatement le relais.

La **réplication** améliore également les performances en répartissant les lectures entre plusieurs copies.

Recherche parallèle

Les recherches sont exécutées simultanément sur plusieurs machines.

Chaque nœud renvoie ses **Top-K** résultats.

Le coordinateur effectue ensuite une fusion globale pour produire le classement final.

Cette stratégie permet de maintenir une faible latence même lorsque le corpus atteint plusieurs centaines de millions de vecteurs.

Mise à jour des index

Contrairement à un prototype, les données évoluent en permanence.

Les systèmes industriels utilisent souvent deux mécanismes complémentaires :

- Index incrémental : ajout immédiat des nouveaux vecteurs.

Réindexation périodique : reconstruction complète pendant les périodes de faible activité afin d'optimiser la structure **ANN**.

Cette approche garantit un bon équilibre entre fraîcheur des données et performances.

Tolérance aux pannes

Les clusters distribués doivent continuer à fonctionner malgré les défaillances matérielles.

Les bonnes pratiques incluent :

- **réplication** des shards ;
- sauvegardes régulières ;
- surveillance de l'état des nœuds ;
- bascule automatique (failover) ;
- reconstruction automatique des index dégradés.

Section 20.4 — Optimisation coût/performance

Le véritable défi

Dans un système **RAG** industriel, la qualité des réponses n'est pas le seul objectif.

Chaque requête possède un coût :

- calcul des **embeddings** ;
- recherche vectorielle ;
- **reranking** ;
- génération par le **LLM** ;
- stockage ;
- trafic réseau.

À grande échelle, quelques millisecondes gagnées ou quelques centimes économisés par millier de requêtes représentent des économies considérables.

Optimiser les embeddings

Toutes les requêtes ne nécessitent pas le modèle d'**embedding** le plus volumineux.

Une stratégie fréquente consiste à :

- utiliser un modèle compact pour les recherches courantes ;
- réserver les modèles plus performants aux cas complexes.

Cette approche réduit le temps de calcul et la consommation GPU.

Cache intelligent

Les questions fréquentes peuvent être mises en cache.

Le système peut mémoriser :

- les **embeddings** des requêtes ;
- les résultats de recherche ;
- les réponses du **LLM**.

Pour les questions répétitives, la réponse est alors renvoyée presque instantanément.

Recherche en plusieurs étapes

Une stratégie efficace consiste à procéder en cascade :

- Recherche **ANN** très rapide.

Sélection des 100 meilleurs candidats.

reranking par un modèle plus précis.

Envoi des 5 à 10 meilleurs passages au **LLM**.

Cette approche réduit fortement le coût tout en conservant une excellente pertinence.

Choix du matériel

Le matériel dépend du profil de charge.

CPU : ingestion, indexation, recherche légère.

GPU : calcul d'**embeddings**, **reranking**, génération **LLM**.

Stockage NVMe : accès rapide aux index volumineux.

Mémoire RAM : maintien des structures d'index en mémoire pour limiter les accès disque.

Un dimensionnement équilibré évite les goulots d'étranglement.

Mesures de performance

Les indicateurs les plus suivis sont :

- Le suivi continu de ces métriques permet d'ajuster l'infrastructure sans dégrader l'expérience utilisateur.

Vers des architectures hybrides

Les plateformes **RAG** les plus performantes combinent plusieurs techniques :

- recherche vectorielle pour la similarité sémantique ;
- recherche lexicale (**BM25**) pour les termes exacts ;
- filtres sur les métadonnées ;
- **reranking** neuronal ;
- cache multi-niveaux ;
- orchestration distribuée.

Cette combinaison offre un compromis optimal entre précision, rapidité et coût, permettant de faire évoluer un système **RAG** depuis quelques milliers de documents jusqu'à plusieurs milliards de passages indexés.

Chapitre 21 — Évaluation continue du système

Une fois qu'un système d'IA (particulièrement un **RAG** ou un **LLM**) est déployé, le plus dur commence. Contrairement au logiciel traditionnel, le comportement d'un modèle est stochastique et les données du monde réel évoluent constamment. L'évaluation ne s'arrête pas à la mise en production : elle devient continue.

Section 21.1 — Évaluation automatique en production

Évaluer un système en production nécessite de se passer de "ground truth" (vérité terrain), car vous ne disposez pas de réponses idéales rédigées par des humains pour chaque requête utilisateur. L'évaluation automatique repose donc sur trois piliers principaux :

- Les métriques de LLM-as-a-Judge : Utilisation de modèles puissants (comme GPT-4) pour évaluer à la volée les réponses du système selon des critères précis :

- Fidélité (Faithfulness) : La réponse est-elle entièrement basée sur les documents récupérés ? (Évite les **hallucinations**).

Pertinence de la réponse : Répond-elle directement à la question de l'utilisateur ?

Les signaux implicites des utilisateurs (Implicit Feedback) : * Taux de clics sur les sources proposées.

Boutons Thumbs Up/Down.

Temps de lecture ou copier-coller du texte généré.

La supervision des guardrails : Intégration d'outils (comme NeMo Guardrails ou Llama Guard) qui valident la toxicité, la confidentialité et la cohérence de l'input et de l'output en temps réel.

Section 21.2 — Tests de non-régression

Chaque mise à jour du système (changement de prompt, mise à niveau du **LLM**, modification de l'algorithme de **chunking**) peut corriger un bug tout en en créant dix autres. Les tests de non-régression (Regression Testing) sécurisent le **pipeline**.

Le dataset en or (Golden Dataset) : Constitution d'un jeu de données fixe et représentatif (de 100 à 1000 questions-réponses clés, validées par des experts métiers).

Automatisation CI/CD : À chaque Pull Request, le **pipeline** exécute le **RAG** sur ce Golden Dataset.

Évaluation différentielle : Comparaison des nouvelles réponses par rapport aux anciennes via des scores de similarité sémantique (BERTScore) ou via un LLM-as-a-Judge chargé de détecter si la nouvelle version est "meilleure", "pire" ou "équivalente" à la précédente.

Section 21.3 — A/B testing RAG

Le A/B testing pour un système **RAG** est plus complexe que pour une simple page web, car les variables influençant la performance sont nombreuses (stratégie de **chunking**, modèle d'**embedding**, **LLM** de génération).

Le routage des utilisateurs : Séparer les utilisateurs en deux groupes (A et B) de manière homogène. Le groupe A utilise la version de production actuelle, le groupe B utilise la variante à tester (ex: un nouveau modèle de **reranking**).

Métriques clés à surveiller :

- Métriques business/usage : Taux de rétention, taux d'engagement.

Métriques techniques : Temps de latence (Time to First Token - TTFT), coût par requête.

Interprétation des résultats : Utilisation de tests statistiques pour s'assurer que l'amélioration des scores du groupe B n'est pas le fruit du hasard, tout en surveillant l'absence de dégradation de l'expérience utilisateur.

Section 21.4 — Drift des embeddings

Le Data Drift (dérive des données) est un phénomène silencieux mais critique. Dans un système **RAG**, il se manifeste par le drift des **embeddings**, c'est-à-dire une perte de pertinence de l'espace vectoriel face aux nouvelles données du monde réel.

Origine du drift : * Évolution du vocabulaire : Apparition de nouveaux termes techniques, produits ou concepts que le modèle d'**embedding** n'a jamais vus lors de son entraînement.

Changement d'intention : Les utilisateurs se mettent à poser des questions sous une forme ou sur des sujets radicalement différents.

Détection : * Suivi de la distribution des distances cosinus lors des requêtes. Si la distance moyenne entre les requêtes et les documents les plus proches augmente significativement, le système perd en précision de recherche.

Visualisation et réduction de dimensionnalité (UMAP/t-SNE) pour repérer graphiquement des clusters de requêtes utilisateurs qui "s'éloignent" de la zone couverte par la base de connaissances.

Remèdes : Mise à jour (**fine-tuning**) du modèle d'**embedding** ou ré-indexation complète de la base de données vectorielle avec un modèle plus récent.

Chapitre 22 — Traçabilité et audit

Les systèmes **RAG** (Retrieval-Augmented Generation) sont de plus en plus utilisés dans des domaines où la fiabilité des réponses est essentielle : santé, droit, finance, assurance, industrie ou encore administration. Dans ces contextes, une réponse correcte ne suffit plus. Il devient indispensable de pouvoir expliquer d'où provient l'information, quelles données ont été utilisées, comment le modèle est arrivé à cette conclusion et comment reproduire exactement la génération.

La traçabilité transforme un système **RAG** en un système explicable, auditable et conforme aux exigences réglementaires modernes (**RGPD**, AI Act européen, ISO 42001, ISO 27001, etc.).

Section 22.1 — Sources et citations obligatoires

L'un des plus grands avantages du **RAG** est qu'il peut citer les documents ayant servi à construire la réponse. Contrairement à un **LLM** seul, le système ne s'appuie pas uniquement sur ses connaissances internes mais sur une base documentaire maîtrisée.

Pourquoi citer les sources ?

Une citation permet à l'utilisateur de :

- vérifier l'information ;
- consulter le document original ;
- détecter une éventuelle erreur ;
- approfondir le sujet.

Sans citation, une réponse reste difficilement vérifiable.

Les informations à conserver

Chaque document indexé devrait posséder des métadonnées complètes.

Exemple :

```
• {  
  "document_id": "DOC_1456",  
  "titre": "Guide Qualité 2026",  
  "version": "3.2",  
  "page": 18,  
  "paragraphe": 4,  
  "date": "2026-04-12",  
  "auteur": "Direction Qualité",  
  "url": "https://..."  
}
```

Ces métadonnées permettent ensuite de construire une citation précise.

Exemple de réponse :

- Selon le Guide Qualité 2026 (version 3.2, page 18), la température maximale autorisée est de 80°C.

Citation au niveau du chunk

Le document est généralement découpé en centaines de morceaux.

Chaque chunk possède son propre identifiant.

Document

■

■■■■ Chunk 001

■■■■ Chunk 002

■■■■ Chunk 003

■■■■ Chunk 004

Le moteur **RAG** retourne directement les chunks les plus pertinents.

Chaque chunk conserve :

- son document d'origine ;
- sa page ;
- sa position ;
- sa version.

Ainsi, la citation reste extrêmement précise.

Exemple de réponse enrichie

Réponse :

Le contrat peut être résilié avec un préavis de trois mois.

Sources :

- Contrat Commercial V4
 - page 27
 - paragraphe 3
- Conditions Générales
 - article 12

L'utilisateur peut immédiatement vérifier l'information.

Plusieurs sources simultanément

Les meilleurs systèmes **RAG** combinent souvent plusieurs documents.

Exemple :

- Réponse :

Le produit nécessite une maintenance annuelle.

Sources :

[1] Manuel Technique

- page 42

[2] Procédure Maintenance

- chapitre 7

[3] Notice Constructeur

- section 4.2

La réponse devient alors beaucoup plus crédible.

Gestion des versions

Un document évolue au fil du temps.

Exemple :

- Guide RH V1

↓

Guide RH V2

↓

Guide RH V3

Le système doit toujours connaître la version utilisée lors de la génération.

Cela évite qu'une réponse produite aujourd'hui cite une version qui n'est plus en vigueur.

Citation des documents externes

Si le **RAG** interroge Internet, il est recommandé de conserver :

- URL complète ;
- date de consultation ;
- date de publication ;
- titre de la page ;
- auteur.

Exemple :

- Source :
- `https://...`
- consultée le 14 mai 2026

Cette pratique est indispensable pour garantir la reproductibilité des résultats.

Section 22.2 — Logging des contextes LLM

La simple conservation des sources ne suffit pas. Pour comprendre pourquoi un **LLM** a produit une réponse, il est nécessaire d'enregistrer tout le contexte de génération.

On parle de **LLM** Logging ou Prompt Logging.

Les informations à journaliser

Chaque requête peut générer un journal complet.

Exemple :

```
• {  
  "request_id": "abc123",  
  "timestamp": "...",  
  "user": "client42",  
  "model": "GPT-5.5",  
  "temperature": 0.2,  
  "top_p": 0.95,  
  "max_tokens": 1200
```

```
}
```

Conserver le prompt système

Le prompt système influence fortement le comportement du modèle.

Exemple :

- Tu es un assistant juridique.

Réponds uniquement à partir des documents fournis.

Ne fais jamais d'hypothèse.

Une modification de quelques mots peut changer complètement la réponse.

Le conserver permet de reproduire exactement une génération.

Journaliser le prompt utilisateur

Exemple :

- Quels sont les délais de rétractation ?

Le prompt d'origine est indispensable lors d'un audit.

Journaliser le contexte RAG

Le système conserve les chunks envoyés au modèle.

Top K = 5

Chunk 1

Chunk 2

Chunk 3

Chunk 4

Chunk 5

Ainsi, il est possible de savoir exactement quelles informations étaient disponibles au moment de la génération.

Conserver les scores de similarité

Chaque chunk possède un score.

Chunk 14

Score : 0.93

Chunk 85

Score : 0.91

Chunk 230

Score : 0.88

Ces scores permettent d'expliquer pourquoi un document a été sélectionné plutôt qu'un autre.

Enregistrer les paramètres du modèle

Les principaux paramètres influençant la génération sont :

- modèle utilisé ;
- température ;
- top_p ;
- fréquence de pénalisation ;
- présence de pénalisation ;
- longueur maximale ;
- seed (si disponible).

Ces paramètres rendent les expériences comparables et facilitent les investigations en cas d'anomalie.

Journaliser la réponse finale

Le système archive également :

- la réponse générée ;
- les citations ;
- les documents utilisés ;
- le temps de génération ;
- le nombre de tokens ;

- le coût de l'appel.

Exemple :

```
• {  
  "latence_ms": 1450,  
  "input_tokens": 3200,  
  "output_tokens": 450,  
  "cout": 0.012  
}
```

Détection des erreurs

Le logging permet également d'identifier :

- **hallucinations** ;
- absence de contexte ;
- contexte contradictoire ;
- dépassement de contexte ;
- erreurs de récupération.

Ces informations servent à améliorer progressivement le système.

Section 22.3 — Auditabilité complète

L'auditabilité consiste à pouvoir reconstituer intégralement une génération, même plusieurs mois ou plusieurs années après son exécution.

C'est une exigence croissante dans les secteurs réglementés.

La chaîne complète de décision

Une requête **RAG** peut être représentée comme une succession d'étapes.

Question utilisateur

■



embedding



Recherche vectorielle



Chunks retrouvés



Re-ranking



Prompt final



LLM



Réponse

Chaque étape doit être enregistrée.

Les éléments à conserver

Pour chaque réponse, un système d'audit robuste archive notamment :

- identifiant de la requête ;
- identité ou rôle de l'utilisateur ;
- date et heure ;
- modèle d'**embedding** ;
- version de la base vectorielle ;

- version des documents ;
- paramètres de recherche (**Top-K**, seuils, filtres) ;
- résultats de la recherche ;
- scores de similarité ;
- modèle **LLM** ;
- prompt système ;
- prompt utilisateur ;
- contexte injecté ;
- paramètres de génération ;
- réponse produite ;
- citations affichées ;
- coût ;
- temps de calcul.

L'ensemble constitue une preuve technique du processus de génération.

Rejouer une génération

Une bonne architecture permet de reproduire une réponse en rechargeant :

- les mêmes documents ;
- les mêmes **embeddings** ;
- le même index vectoriel ;
- le même prompt ;
- les mêmes paramètres du modèle.

Cette capacité de replay est précieuse pour le débogage, la validation qualité et les audits de conformité.

Conformité réglementaire

L'auditabilité répond à plusieurs objectifs :

- démontrer l'origine des informations fournies ;
- justifier les décisions prises par un système d'IA ;
- faciliter les audits internes et externes ;
- répondre aux exigences de conservation des preuves ;
- améliorer la gouvernance des modèles d'IA.

Ces mécanismes sont particulièrement importants dans le cadre du **RGPD**, de l'AI Act européen, des normes ISO 42001 (management de l'IA), ISO 27001 (sécurité de l'information) et ISO 9001 (management de la qualité).

Bonnes pratiques

Pour terminer ce chapitre, voici quelques recommandations pour concevoir un système **RAG** pleinement traçable :

- Conserver les métadonnées de chaque document et de chaque chunk.

Afficher systématiquement les sources utilisées dans les réponses.

Versionner les documents, les index et les modèles afin de garantir la reproductibilité.

Journaliser l'intégralité du **pipeline** : requête, recherche, contexte, paramètres et réponse.

Mettre en place une politique de rétention des journaux, conforme aux exigences légales et à la confidentialité des données.

Permettre le rejeu d'une génération pour faciliter les audits et les analyses d'incidents.

Un système **RAG** moderne ne doit pas seulement produire des réponses pertinentes : il doit être capable de prouver leur origine, expliquer leur construction et reproduire leur génération. La traçabilité et l'auditabilité constituent ainsi des piliers essentiels des applications d'IA générative destinées aux environnements professionnels et réglementés.

Chapitre 23 — Pipeline complet production RAG

Le déploiement industriel d'un système de génération augmentée de documents (**RAG** - Retrieval-Augmented Generation) ne se résume pas à l'écriture d'un script Python connectant un orchestrateur comme LangChain ou LlamaIndex à une API de **LLM**. Passer à l'échelle en production requiert une architecture robuste, capable de gérer des flux de données asynchrones, de garantir une latence minimale, d'assurer la traçabilité des réponses et de s'adapter aux pannes d'infrastructure.

Ce chapitre détaille pas à pas l'implémentation et la conception d'un **pipeline** complet prêt pour la production.

Section 23.1 — Architecture finale

Pour garantir des performances optimales et une excellente tolérance aux pannes, l'architecture finale du système **RAG** en production est scindée en deux grands cycles indépendants :

- Le **pipeline** d'ingestion hors-ligne (Offline Pipeline) : Il gère la collecte, le nettoyage, le découpage, la vectorisation et le stockage des documents de manière asynchrone.

Le **pipeline** d'inférence en ligne (Online Pipeline) : Il gère la requête de l'utilisateur, la recherche sémantique hybride, le **reranking**, l'appel au **LLM** et la restitution des résultats avec citations en temps réel.

1. PDF / Multi-Data sources

Les sources de données d'une entreprise sont intrinsèquement hétérogènes : documents PDF numérisés ou textuels, fichiers Word/Excel, présentations, bases de connaissances internes (Confluence, Notion) ou bases de données relationnelles (SQL).

L'extraction des documents d'origine doit s'appuyer sur des connecteurs robustes connectés à des outils de stockage objets (Amazon S3, Google Cloud Storage, ou serveurs MinIO sur site). Elle s'accompagne d'un mécanisme de détection de changement (via une somme de contrôle MD5 ou des métadonnées de versioning) afin d'éviter de retraiter inutilement des fichiers inchangés.

2. Ingestion pipeline (Pipeline d'ingestion asynchrone)

Pour éviter les goulots d'étranglement et gérer de grands volumes de documents sans dégrader l'expérience utilisateur, l'ingestion ne doit jamais être synchrone.

Nous mettons en œuvre un **pipeline** asynchrone orchestré par Apache **Airflow** ou **Celery** avec RabbitMQ/Redis comme broker de messages. Les tâches sont découpées de manière hautement isolée :

- Détection et téléchargement du document source.

Extraction de texte brut.

Découpage sémantique.

Calcul des **embeddings** et indexation vectorielle.

Gestion automatique des reprises sur erreur (retries) avec un mécanisme de backoff exponentiel en cas d'échec d'un service tiers ou d'une API de modèle.

3. Cleaning / Normalization (Nettoyage et normalisation)

Le bruit dans les données brutes détruit directement la qualité des représentations vectorielles. L'étape de nettoyage effectue plusieurs traitements indispensables :

Extraction de structure : Utilisation de bibliothèques avancées comme Unstructured, Grobid ou des modèles de vision industrielle (comme LayoutLMv3) pour isoler les tableaux, ignorer les en-têtes et les pieds de page répétitifs, et reconstituer l'ordre de lecture réel d'un document multi-colonnes.

Normalisation du texte : Suppression des caractères non-imprimables, nettoyage des espaces et sauts de ligne multiples, normalisation Unicode (NFC).

Anonymisation (**RGPD**) : Passage automatique du texte dans un module de détection d'informations personnelles identifiables (PII) à l'aide de moteurs comme Microsoft Presidio pour masquer ou anonymiser les noms, adresses, adresses email et numéros de téléphone avant stockage.

4. Chunking avancé (Découpage sémantique)

Découper le texte par fenêtres fixes de caractères (ex: blocs de 500 caractères avec chevauchement) est insuffisant en production car cela brise le flux logique de la pensée. Nous implémentons ici un **chunking** sémantique basé sur la structure et le sens :

- Découpage structurel : Respect des frontières naturelles du texte (paragraphe, structures Markdown pour les listes et tables, blocs de code, balises HTML).

Segmentation sémantique : Analyse des variations de distance cosinus entre phrases successives. Un nouveau chunk est créé dès que la transition sémantique dépasse un certain seuil de rupture :

$$\Delta = 1 - \cos(\vec{u}_t, \vec{u}_{t+1}) > \tau$$

(Où $\vec{u}_t$ représente l'**embedding** de la phrase t et τ le seuil empirique de découpe).

Propagation des métadonnées : Chaque chunk final hérite de métadonnées cruciales pour le filtrage ultérieur (ID du document source, numéro de page, titre de section, date de dernière modification, tags de droits d'accès utilisateur).

5. Embeddings (Vectorisation)

Les chunks nettoyés sont convertis en vecteurs d'**embeddings** denses. En production, le choix du modèle d'**embedding** s'articule autour du compromis coût/latence/performance :

Modèles locaux : Déploiement de modèles performants (ex: **BGE-M3**, GTE-Large) sur notre propre infrastructure GPU à l'aide de runtimes optimisés comme ONNX Runtime ou TensorRT-LLM pour garantir une latence d'inférence extrêmement faible (inférieure à 15 ms).

APIs managées : Utilisation d'APIs spécialisées comme Cohere v3 Embed qui intègrent nativement des paramètres de type de requête (search_query vs search_document) pour maximiser l'alignement de l'espace vectoriel.

6. Vector DB (Milvus/Qdrant)

Le stockage des vecteurs requiert une base de données distribuée et hautement disponible. **Qdrant** ou **Milvus** sont privilégiés pour leurs performances à l'échelle et leurs fonctionnalités d'entreprise :

Indexation **HNSW** (Hierarchical Navigable Small World) : Configuration des hyperparamètres d'indexation (notamment `ef_construction` et `M`) pour équilibrer le temps de construction de l'index et le taux de rappel (recall).

Filtrage de métadonnées (**payload filtering**) : Application de filtres stricts en amont ou en cours de recherche vectorielle (ex: restreindre la recherche aux documents dont l'utilisateur possède spécifiquement les droits de lecture) sans dégradation des temps de réponse.

sharding & réplication : Répartition des index sur plusieurs nœuds physiques pour assurer la redondance et la répartition de la charge de requête.

7. Hybrid retrieval (Recherche hybride)

La recherche vectorielle pure (sémantique) échoue souvent sur les requêtes contenant des termes très spécifiques, des numéros de série, ou des acronymes précis. La production exige donc une recherche hybride combinant :

- La recherche dense : **similarité cosinus** sur les **embeddings** denses pour capturer le contexte sémantique global.

La recherche creuse (Sparse **retrieval**) : Indexation lexicale via l'algorithme **BM25** classique ou des approches d'apprentissage de représentations creuses comme **SPLADE**.

La fusion des résultats s'opère par l'algorithme de **Reciprocal Rank Fusion (RRF)**, qui classe les documents en sommant les inverses de leurs rangs respectifs dans les deux listes de résultats :

$$RRF_Score(d \in D) = \sum_{m \in M} \frac{1}{k + r_m(d)}$$

(Où M représente les méthodes de recherche [dense et creuse], $r_m(d)$ est le rang du document d dans la méthode m , et k est une constante de régularisation, généralement fixée à 60).

8. Reranking (Re-classement sémantique)

Pour optimiser la pertinence sémantique finale, les K meilleurs candidats (par exemple 50 documents issus de la fusion hybride **RRF**) sont passés dans un modèle de **reranking** (comme **Cohere Rerank** ou le modèle open-source **BGE-Reranker-Large**).

Contrairement aux modèles d'**embeddings** classiques (bi-encodeurs), le cross-encodeur de **reranking** calcule une attention croisée complète entre la question et chaque document candidat. Cette étape lourde en calcul permet de filtrer et de réduire le nombre de chunks finaux passés au **LLM** (par exemple de 50 à seulement 5 documents extrêmement pertinents), ce qui économise des tokens d'input et élimine le problème du "Lost in the Middle" (perte d'information située au milieu du contexte d'un long prompt).

9. Context builder (Générateur de contexte)

Le générateur de contexte assemble les chunks sélectionnés dans le prompt final envoyé au **LLM**. Il effectue de manière automatisée plusieurs tâches fondamentales :

- Contrôle du budget de tokens : Tronquer dynamiquement le contexte pour ne jamais saturer la fenêtre de contexte maximale supportée par l'instance **LLM**.

Formatage structuré du contexte : Présentation uniforme de chaque source incluse pour aider le **LLM** à analyser les informations. Exemple :

```
[Contenu textuel du chunk...]
```

Instructions de cloisonnement : Injection de consignes de prompt strictes forçant le **LLM** à s'appuyer uniquement sur le contexte XML fourni pour répondre et à refuser de répondre s'il ne trouve pas l'information (afin d'éviter les **hallucinations**).

10. LLM inference (Inférence optimisée)

En production, l'inférence du **LLM** doit être optimisée pour maximiser le débit (throughput) tout en réduisant la latence au minimum. Nous utilisons un moteur d'inférence dédié comme **vLLM** ou Triton Inference Server hébergé sur des instances GPU dédiées (ex: NVIDIA L4, L40S ou H100) :

Continuous Batching & **PagedAttention** : Optimisations de **vLLM** qui éliminent le gaspillage de mémoire lié aux clés-valeurs d'attention (KV Cache) et permettent de multiplier par 10 le débit de requêtes concurrentes.

Streaming des réponses (SSE) : Envoi de la réponse sous forme de flux Server-Sent Events (SSE) au client pour abaisser drastiquement le temps d'attente perçu (Time to First Token - TTFT < 100ms).

11. Response with citations (Réponse et citations d'ancrage)

L'explicabilité est essentielle pour instaurer la confiance avec les utilisateurs. Le **LLM** est instruit par prompt engineering ou par contraintes de schéma (Structured Outputs / JSON Mode) à produire ses affirmations en y adjoignant des citations ancrées dans le texte de base.

La réponse attendue lie chaque fait à l'identifiant de la source :

"La température critique de fonctionnement du réacteur est de 450°C [doc_001]. En cas de dépassement de ce seuil, les vannes d'urgence s'activent automatiquement dans un délai de 3 secondes [doc_003]."

Le système extrait ensuite ces identifiants pour afficher, au clic de l'utilisateur, le document source original ouvert précisément à la bonne page.

12. API FastAPI

Toutes ces étapes sont orchestrées derrière une API construite avec **FastAPI** (Python). FastAPI est retenu pour ses performances asynchrones (via uvicorn), sa validation automatique des types de données avec Pydantic v2, et sa génération automatique de documentation interactive OpenAPI (/docs). L'API expose des routes clés :

- POST /api/v1/query : Interrogation synchrone et asynchrone (streaming) du **RAG**.

POST /api/v1/documents : Gestion et ajout manuel de documents à la base de connaissances.

GET /api/v1/health : Validation de l'état de santé de l'infrastructure et des connexions aux bases de données.

13. Cache + monitoring (Optimisation et observabilité)

Pour assurer la viabilité économique et la stabilité opérationnelle du système à grande échelle :

Cache sémantique (**Semantic Cache**) : Implémentation d'un cache basé sur **Redis**. Avant d'exécuter la recherche hybride et l'appel **LLM**, l'API recherche dans Redis s'il existe une question similaire précédemment posée dont la distance d'**embedding** est extrêmement proche (ex: seuil cosinus > 0.96). Si c'est le cas, la réponse mise en cache est directement renvoyée, abaissant la latence à quelques millisecondes et réduisant les coûts de jetons d'appel de 90 %.

Monitoring et observabilité : Collecte des métriques système (CPU, VRAM, taux d'erreur, latence, débit) via Prometheus et affichage sur des tableaux de bord Grafana.

Traçabilité **LLM** (LLM Tracing) : Enregistrement exhaustif des étapes de recherche et des appels LLM dans un outil de tracing comme Langfuse ou Arize Phoenix afin de debugger les **hallucinations** ou les mauvaises réponses.

14. Docker + orchestration

L'ensemble du système est conteneurisé à l'aide de **Docker** pour garantir l'homogénéité entre les environnements de développement et de production. On utilise Docker Compose pour les déploiements mononœuds (environnements de staging ou serveurs indépendants) et Kubernetes (K8s) pour la mise à l'échelle automatique et la haute disponibilité.

Un fichier Dockerfile type pour notre API **FastAPI** de production est optimisé avec des builds multi-étapes (multi-stage builds) afin de minimiser le poids de l'image finale :

```
FROM python:3.11-slim AS builder
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends build-essential gcc
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

FROM python:3.11-slim AS runner
WORKDIR /app
```

```
COPY --from=builder /root/.local /root/.local
COPY . .
ENV PATH=/root/.local/bin:$PATH
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "4"]
```

15. VPS / cloud deployment (Choix d'infrastructure)

Le choix de l'infrastructure de déploiement dépend des exigences de conformité et de budget de l'entreprise :

Chapitre 24 — Cas d'usage réels

Section 24.1 — RAG documentaire entreprise

Section 24.2 — RAG juridique

Section 24.3 — RAG industriel

Section 24.4 — RAG support client

Voici une version grandement enrichie, densifiée et académique de votre conclusion. Ce texte est structuré pour apporter du relief à votre fin d'ouvrage, en insistant sur la rigueur technique, la bascule culturelle que représente le passage à la production, et la posture de l'architecte face aux systèmes d'IA critiques.

Conclusion — De l'ingénieur RAG au système critique

Au terme de ce voyage au cœur de la génération augmentée par récupération, une réalité s'impose : le fossé qui sépare un prototype de recherche (Proof of Concept) d'un système de production à tolérance de panne minimale est un gouffre d'ingénierie. Coder un script d'une cinquantaine de lignes connectant une bibliothèque d'orchestration standard à une API tierce est à la portée de n'importe quel développeur. En revanche, bâtir, sécuriser et faire évoluer une infrastructure capable

de répondre de manière déterministe, traçable et souveraine à des milliers d'utilisateurs concurrents est l'apanage de l'architecte de systèmes d'IA critiques.

Ce manuel a été conçu pour vous faire franchir ce cap. À travers les concepts théoriques, les formules mathématiques et les choix d'infrastructure que nous avons analysés, vous avez consolidé un ensemble de compétences hautement spécialisées qui redéfinissent votre rôle dans l'entreprise.

I. Un arsenal de compétences théoriques et pratiques acquises

Le parcours de cet ouvrage vous a permis de structurer et d'assimiler les piliers fondamentaux de l'ingénierie des données et de l'intelligence artificielle appliquée

Conception d'architecture **RAG** complète de bout en bout : Vous avez appris à rejeter les architectures monolithiques au profit de systèmes asynchrones hautement découplés. Vous maîtrisez désormais la séparation stricte entre le **pipeline** d'ingestion hors-ligne (chargé du traitement lourd des données) et le pipeline d'inférence en ligne (optimisé pour la latence et le débit). Cette asynchronicité garantit qu'une mise à jour de la base documentaire n'impactera jamais la disponibilité de vos services pour l'utilisateur final.

Ingestion industrielle multi-sources : Les données d'entreprise réelles sont sales, fragmentées et hétérogènes. Vous êtes désormais capable d'orchestrer des files d'attente distribuées (**Celery**, RabbitMQ, Apache **Airflow**) pour ingérer et normaliser des flux massifs provenant de stockages objets (S3, GCS) ou de bases de données relationnelles. L'utilisation d'outils de vision documentaire et d'extraction de structure vous permet de transformer des PDF complexes ou des tableaux croisés en flux d'informations exploitables, tout en automatisant le masquage des informations personnelles (**RGPD**) en amont grâce à des moteurs de détection de PII.

retrieval optimisé à grande échelle : Vous avez dépassé la simple recherche vectorielle naïve. En combinant l'indexation de graphes de proximité (**HNSW**) au sein de bases vectorielles comme **Qdrant** ou Milvus avec la recherche lexicale classique (**BM25**) et les représentations creuses (SPLADE), vous savez concevoir des moteurs de recherche hybrides résilients. La maîtrise de l'algorithme **Reciprocal Rank Fusion (RRF)** et l'intégration systématique de modèles de **reranking** sémantique (cross-encoders) vous permettent d'extraire la substantifique moelle de vos corpus documentaires en quelques millisecondes, éliminant ainsi le bruit informationnel.

Architectures multi-utilisateurs et cloisonnement : La sécurité des données en entreprise ne souffre aucune approximation. Vous savez configurer des filtres de métadonnées dynamiques (**payload filtering**) exécutés au plus près du calcul vectoriel afin d'appliquer à la volée les listes de contrôle d'accès (ACL). Cette architecture garantit qu'un utilisateur n'obtiendra jamais de réponse basée sur un document pour lequel il ne possède pas explicitement les droits de lecture.

Déploiement agile (Cloud vs. VPS) : Vous maîtrisez l'art de la conteneurisation optimisée. Grâce à des builds **Docker** multi-étapes et à la gestion de runtimes d'inférence haut de gamme (**vLLM**, Triton), vous savez dimensionner vos charges de travail sur GPU. Vous êtes capable d'arbitrer rationnellement entre la **scalabilité** élastique des clouds publics (AWS, GCP, Azure) et la prédictibilité financière d'infrastructures souveraines dédiées (Hetzner, Scaleway, OVHcloud),

évitant ainsi l'explosion des coûts liés à la bande passante et au calcul.

Monitoring, observabilité et traçabilité : Une IA en production est un système vivant. Vous avez appris à instrumenter vos applications pour collecter des métriques d'infrastructure (Prometheus/Grafana) et à implémenter un traçage complet de chaque étape de génération (Langfuse, Arize Phoenix). Cet enregistrement systématique des traces d'exécution vous permet d'auditer instantanément les requêtes lentes, de debugger les échecs de **retrieval**, et de surveiller l'évolution sémantique des requêtes de vos utilisateurs.

Réduction drastique et scientifique des hallucinations : C'est le cœur de la promesse du **RAG**. En couplant un **chunking** sémantique précis à un prompt engineering ultra-directif, en structurant vos contextes d'entrée par balisage XML strict et en forçant le **LLM** à produire des sorties typées (Structured Outputs) contenant des citations d'ancrage vérifiables, vous avez transformé un modèle de langage probabiliste en un moteur de restitution de faits fiable et auditable.

II. Du stochastique au déterministe : La quête du système critique

Déployer un système **RAG** critique demande une bascule culturelle profonde. Les ingénieurs logiciels traditionnels sont habitués au déterminisme rigide des algorithmes classiques : une même entrée produit invariablement la même sortie. Les modèles de langage, de par leur nature stochastique, introduisent une variabilité qui effraie les directions métiers et les responsables de la sécurité.

Votre rôle, en tant qu'ingénieur système critique, est de concevoir des ceintures de sécurité autour de cette incertitude :

Le contrôle par les métriques : Vous n'évaluez plus vos modèles à l'intuition. Vous mettez en place des processus d'évaluation continue (Ragas, TruLens) agissant comme des juges automatiques en production pour quantifier la fidélité de la réponse aux documents sources (faithfulness) et sa pertinence par rapport à la question initiale.

L'architecture défensive : L'usage de caches sémantiques (**Redis**) permet non seulement de réduire les coûts opérationnels de 90 %, mais agit également comme un bouclier limitant la variabilité des requêtes répétitives. Les guardrails en temps réel (NeMo, Llama Guard) interceptent les entrées malveillantes (injections de prompts) et filtrent les sorties hors-sujet ou toxiques avant qu'elles n'atteignent l'utilisateur final.

III. L'Architecte RAG : Pilier de l'entreprise cognitive

À l'ère de l'intelligence artificielle générative, la véritable valeur d'une organisation ne réside plus seulement dans ses serveurs ou ses codes sources, mais dans sa mémoire collective — cette accumulation de manuels, de rapports, de discussions et de bases de données qui constituent son patrimoine intellectuel.

En maîtrisant l'intégralité du **pipeline** de production **RAG**, vous n'êtes plus un simple consommateur d'APIs tierces. Vous êtes le concepteur de l'infrastructure cognitive de votre

entreprise. Vous êtes celui qui permet à une organisation d'interroger son propre savoir de manière sécurisée, instantanée et vérifiable.

Les compétences que vous avez acquises dans ce livre constituent le socle de l'informatique de demain. Armé de ces concepts, de ces architectures rigoureuses et de cette éthique de production, vous disposez désormais de tous les outils nécessaires pour transformer des téraoctets de données brutes et silencieuses en un copilote robuste, intelligent et digne de confiance. Le chemin vers des systèmes critiques et fiables est désormais tracé : il ne vous reste plus qu'à le déployer.